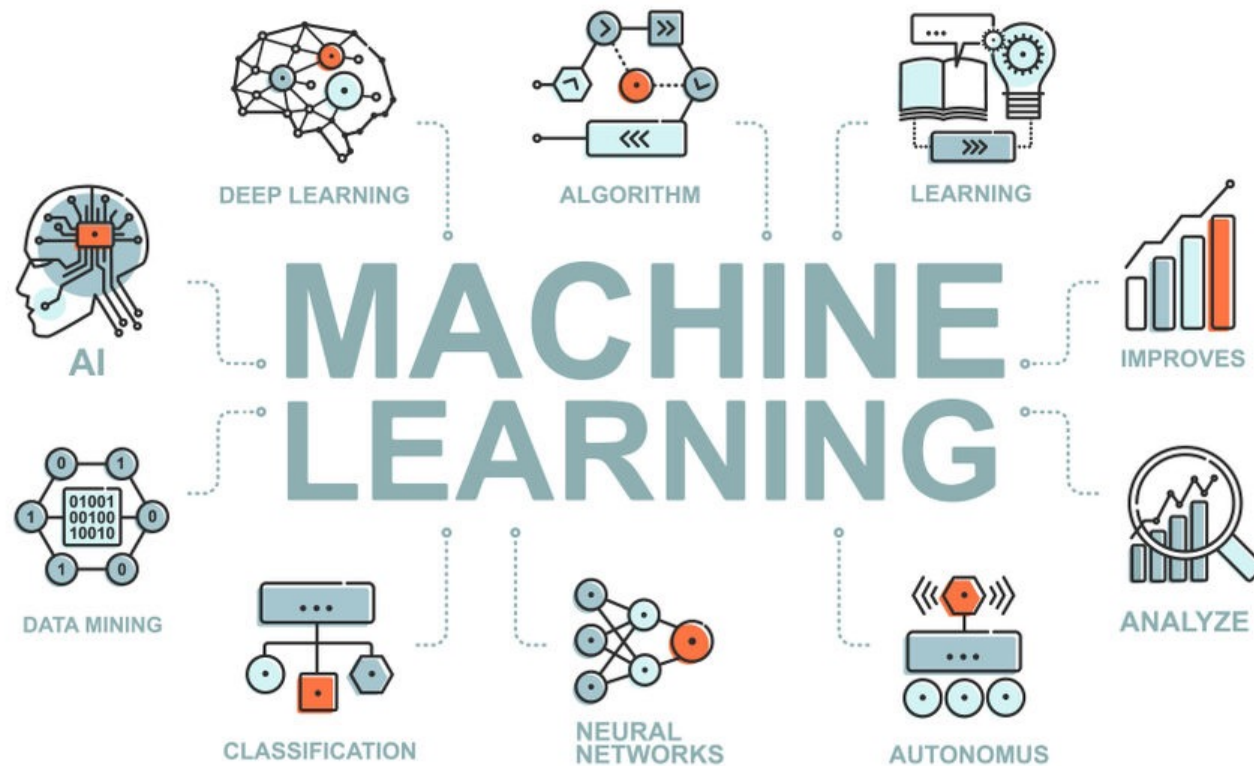
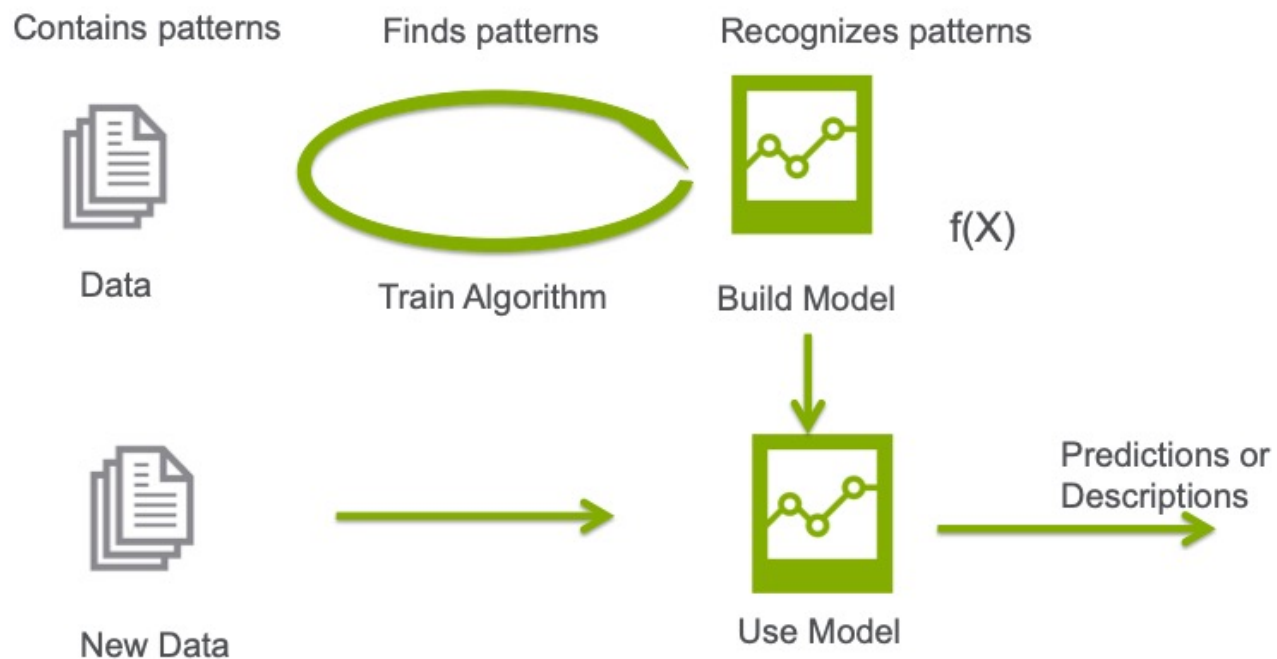


Introduction to Machine Learning



What is Machine Learning?

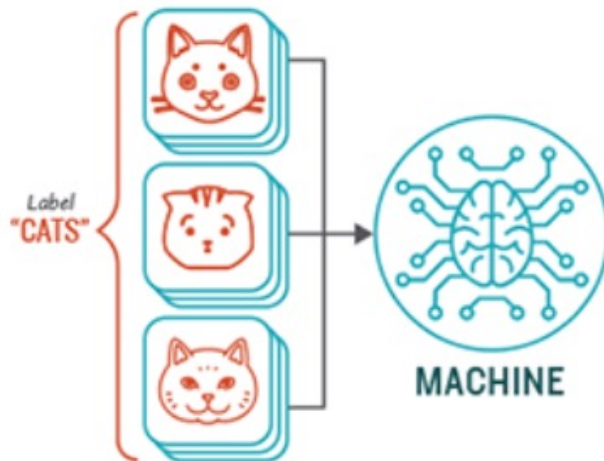
- Employs **algorithms** and **statistical models**
- Enables computer systems **to find patterns**
- Uses a model that **recognizes patterns** to **make predictions**



How Machines Learn?

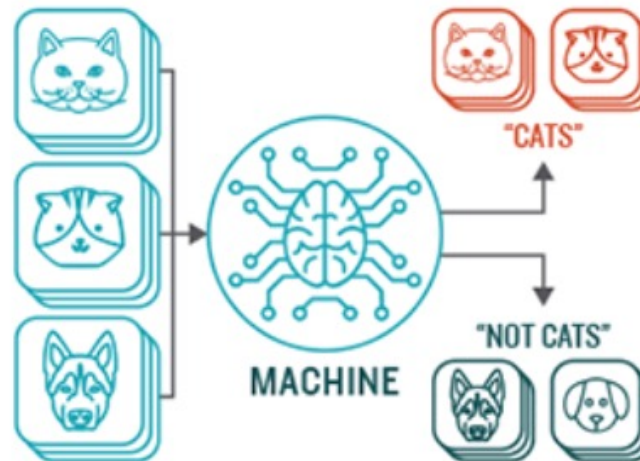
STEP 1

Provide the machine learning algorithm categorized or "labeled" input and output data from to learn

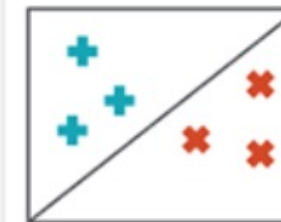


STEP 2

Feed the machine new, unlabeled information to see if it tags new data appropriately. If not, continue refining the algorithm

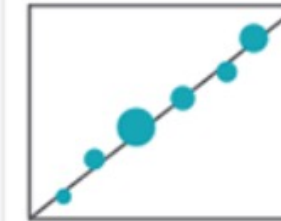


TYPES OF PROBLEMS TO WHICH IT'S SUITED



CLASSIFICATION

Sorting items into categories



REGRESSION

Identifying real values (dollars, weight, etc.)



Learning Input-Output Functions

- Learning input-output functions in machine learning refers to the **process of building models** that map inputs (features) to outputs (labels or predictions).
 - **Key Concepts** in Learning Input-Output Functions:
 - **Input (Features)**
 - **Definition:** Input data points (features) are representations of real-world data, typically in numerical form.
 - **Examples:** In an image recognition task, pixel values; in a sales prediction task, historical sales data.
-



Learning Input-Output Functions

- **Output (Labels/Targets)**
 - **Definition:** The output is what the model aims to predict or classify.
 - **Types:**
 - **Continuous:** Regression tasks (e.g., predicting house prices).
 - **Categorical:** Classification tasks (e.g., identifying whether an email is spam or not).
 - **Model (Function Approximation)**
 - **Definition:** The model approximates the relationship between input and output.
 - **Function:** $f(x; \theta) = \{y\}$, where x is input, θ are model parameters, and $\{y\}$ is the predicted output.
-

Example: Learning Input-Output Functions

Input | Single

Output | Single



Here the concept to learn is x^2

Instance #	Input	Output
1	1	1
2	2	4
3	3	9
4	4	16



Well Posed Learning

Definition # 1

Arthur Samuel (1959) coined the term machine learning and defined it as: ‘the field of study that gives computers the ability to learn without being explicitly programmed.’ This is an informal and old definition of machine learning.

Definition # 2

In 1998, Tom Mitchell redefined the concept of machine learning as ‘**A computer program is said to learn from experience E with respect to some class of tasks T and performance measures P ; if its performance at tasks in T , as measured by P , improves with experience E .**’



Well Posed Learning

- A well-posed learning problem in machine learning refers to a problem that is **carefully and clearly defined**, ensuring that the machine learning process is likely to produce meaningful and actionable results.

 - Criteria for a Well-Posed Learning Problem:
 - Existence of a Solution
 - Uniqueness of the Solution
 - Stability of the Solution
-



Well Posed Learning Problem

- Classification of emails as spam or not spam --- Task T
 - Tracking user marking emails as spam or not spam --- Experience E
 - The number of emails correctly classified as spam or not spam --- Performance P
-



Well Posed Learning Problem

- **Problem Statement:** Predict whether a bank customer will default on a loan.
 - **Input Data:**
 - Customer's financial history, income, credit score, loan amount, etc.
 - **Output (Target):**
 - Binary classification: 1 (default) or 0 (no default).
 - **Mapping Function:**
 - Logistic regression or a decision tree classifier.
 - **Evaluation Metrics:**
 - Accuracy, Precision, Recall, and AUC-ROC.
-



Designing a Learning System

- Define the Problem
 - Understand the Data
 - Choose the Learning Approach
 - Select a Model
 - Train the Model
 - Evaluate the Model
 - Optimize the Model
 - Deploy the Model
 - Monitor and Maintain
-



Designing a Learning System: Example

- **Define the Problem:** Task: Classify emails as spam or not spam.
- **Understand the Data:**
 - Inputs: Email text, sender's domain, email subject.
 - Output: Spam (1) or Not Spam (0).
- **Choose the Learning Approach:** Supervised Learning (classification).
- **Select a Model:** Naive Bayes or Logistic Regression.
- **Train the Model:**
 - Use labeled email data (spam vs. not spam).
 - Preprocess text (tokenization, stop word removal, TF-IDF vectorization).
- **Evaluate the Model:** Metrics: Precision, Recall, F1 Score, Accuracy.
- **Optimize the Model:** Tune hyperparameters.
- **Deploy the Model:** Implement as a real-time spam filter.
- **Monitor and Maintain:** Update with new labeled emails periodically.



Statistical Learning Framework

- It provides a **formal foundation** for understanding and designing machine learning models.
 - It defines the **process of learning** from data using a **mathematical** and **statistical lens**.
 - This framework helps quantify **how well a learning algorithm generalizes** from a sample to unseen data.
-

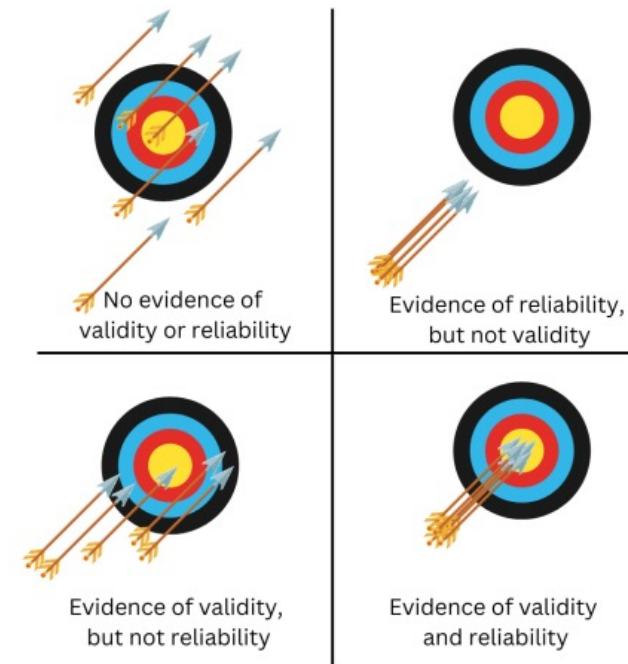


Statistical Learning Framework

- **Data Generation Process:** Data points are sampled from an unknown distribution
 - **Model and Hypothesis Space:** The ground-truth mapping
 - **Loss Function:** Measures the error between predicted output $f(x)$ & true output y
 - **Risk Function:** The average loss over the entire distribution
 - **Learning Objective:** Minimizes the expected risk
 - **Generalization:** How well the model performs on unseen data
 - **Overfitting and Underfitting**
 - **Overfitting:** The model captures noise in the training data, leading to poor generalization.
 - **Underfitting:** The model is too simple to capture the underlying patterns.
-

Empirical Risk Minimization (ERM)

- ERM aims to find a **model that minimizes the average loss** on a given training dataset.
- **Goal:** to throw a dart as close to the center of the target as possible.
 - **The Target:** Represents the true underlying function
 - **The Darts:** Represent different models, each with its own set of parameters.
 - **The Bullseye:** Represents the optimal model, which minimizes the loss.





ERM with Inductive Bias

- The Overfitting can be avoided by:
 - **Limiting** the number of possible hypothesis
 - **Increasing** the number of training data
 - **Definition:** Inductive bias refers to the **assumptions a learning algorithm makes to generalize** from the training data to unseen data.
 - It helps prioritize certain hypotheses in $\{H\}$ over others.
 - **Need:** Without inductive bias, ERM can overfit the training data, leading to poor generalization to unseen data.
-

ERM with Inductive Bias: Example

House Size (sq ft)	Price (50000 INR)
1000	150
1500	200
2000	250
2500	300

- The goal is to find a function $f(x)$ that predicts house prices y given the size x .
- In this case, the model $f(x)$ will be a linear function:
- $f(x) = w \cdot x + b$
- The ERM framework helps us learn the parameters w and b by minimizing the empirical risk.

Probably Approximately Correct (PAC) Learning

- PAC learning is a framework:
 - Provides a formal analysis of **how much data is needed** for a learning algorithm to perform well on unseen data.
 - Ensures with **high probability** δ , the learned model will have **low error ϵ on new data**.
 - Goal is to find a **learning algorithm** that can learn any function from a given hypothesis space with **high probability and low error**, given enough training data.
-



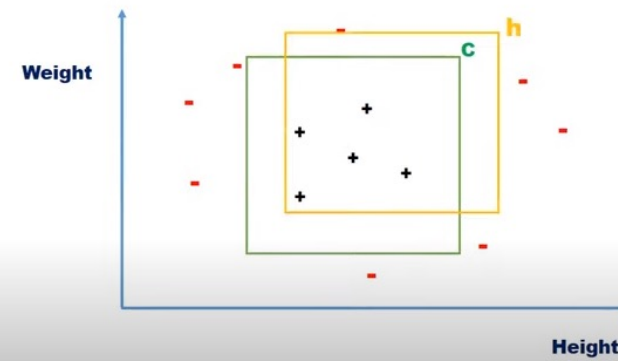
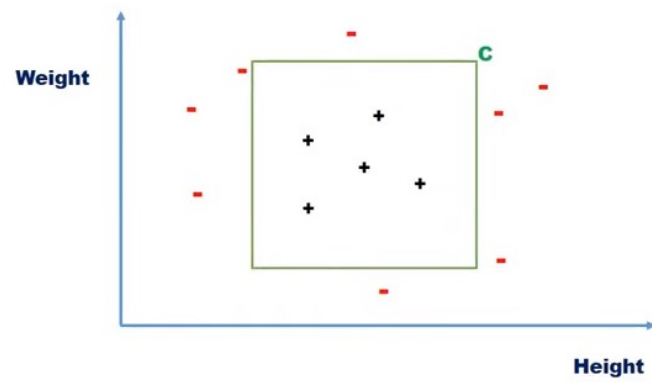
Definition

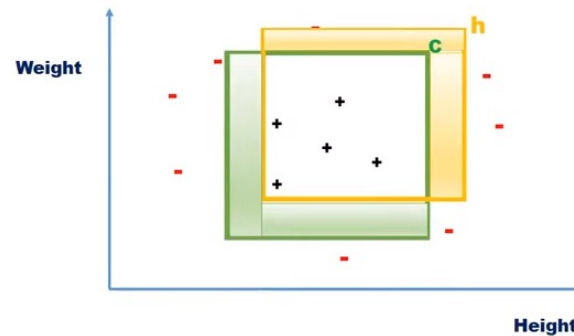
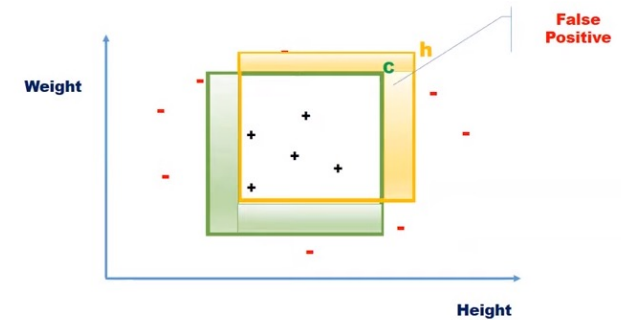
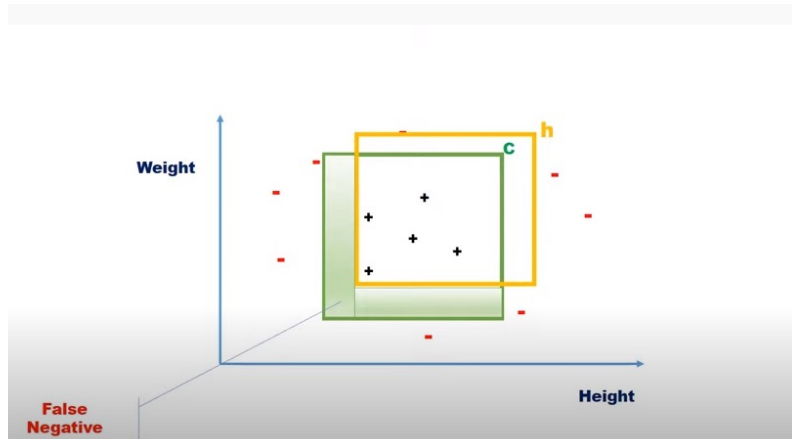
- A **concept class C** is **PAC-learnable** if there exists a learning algorithm such that, for every $\epsilon > 0$, $\delta > 0$, and every distribution D :
 - The algorithm outputs a **hypothesis $h \in H$** such that h is ϵ -accurate with probability at least $1 - \delta$.
 - The number of **labeled examples** required is polynomial in $1/\epsilon$, $1/\delta$, and the description size of c (the target concept).



Example: Medium Built Person

- **Training Set:** Height and Weight of m individuals.
 - **Target:** Whether the person is medium built or not.
-





$$P(C \text{ XOR } h) \leq \epsilon$$

- Learner needs to learn the concept from a **small representation of instance space**.
- Hypothesis is approximately correct, if **error is $\leq \epsilon$** .



Approximately Correct

- A hypothesis is said to be approximately correct, if the error is less than or equal to ϵ , where $0 \leq \epsilon \leq \frac{1}{2}$.
 - i.e. $P(c \oplus h) \leq \epsilon$

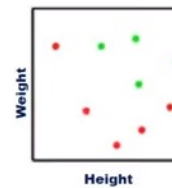
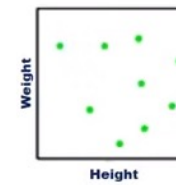
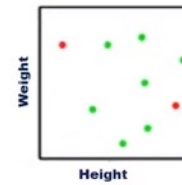
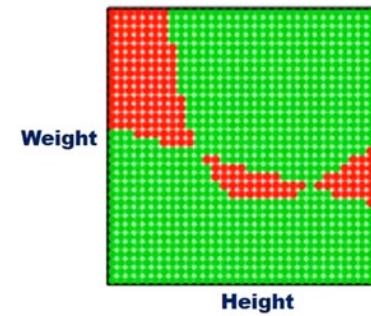
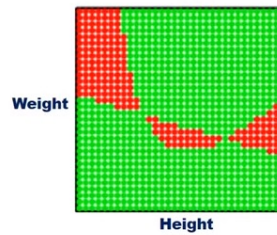


Probably Approximately Correct

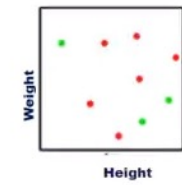
- The goal is to achieve low generalization error with high probability.

$$\Pr(\text{Error}(h) \leq \epsilon) > 1 - \delta$$

Instance Space

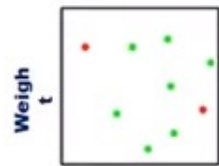


...

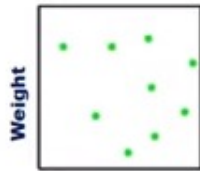




Error(H1)

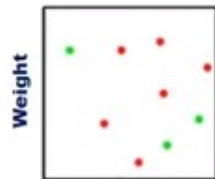


Height



Height

⋮

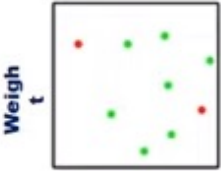
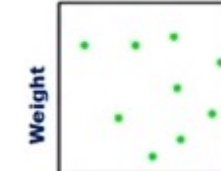


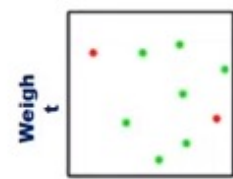
Height

$$\epsilon = 0.05$$

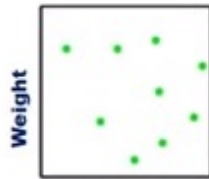
$$\delta = 0.20$$



Error(H1)		
	0.04	$\epsilon = 0.05$
	0.03	
	0.09	
	0.06	
	0.025	
	0.049	$\delta = 0.20$
	0.04	
	0.03	
	0.05	
	0.043	

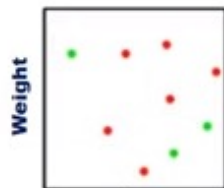


Height



Height

⋮



Height

Error(H1)

0.04

0.03

0.09

0.06

0.025

0.049

0.04

0.03

0.05

0.043

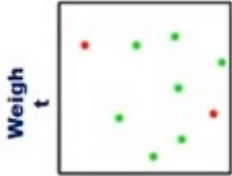
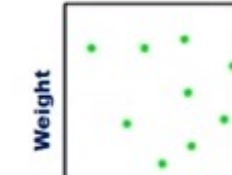
$\epsilon = 0.05$

$\delta = 0.20$

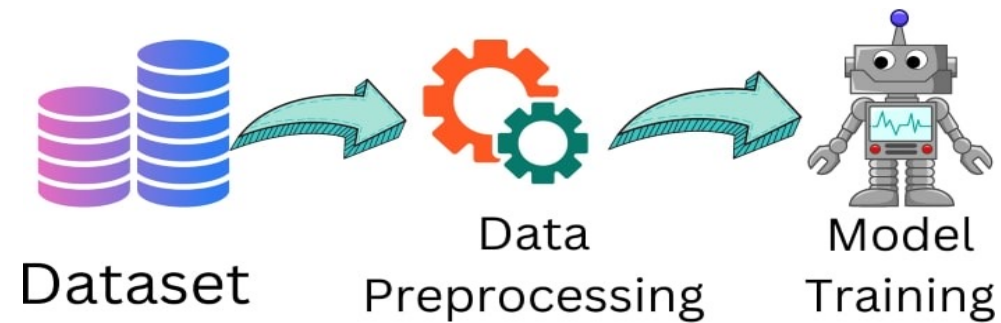
$P(H1) = 8/10 = 0.80$

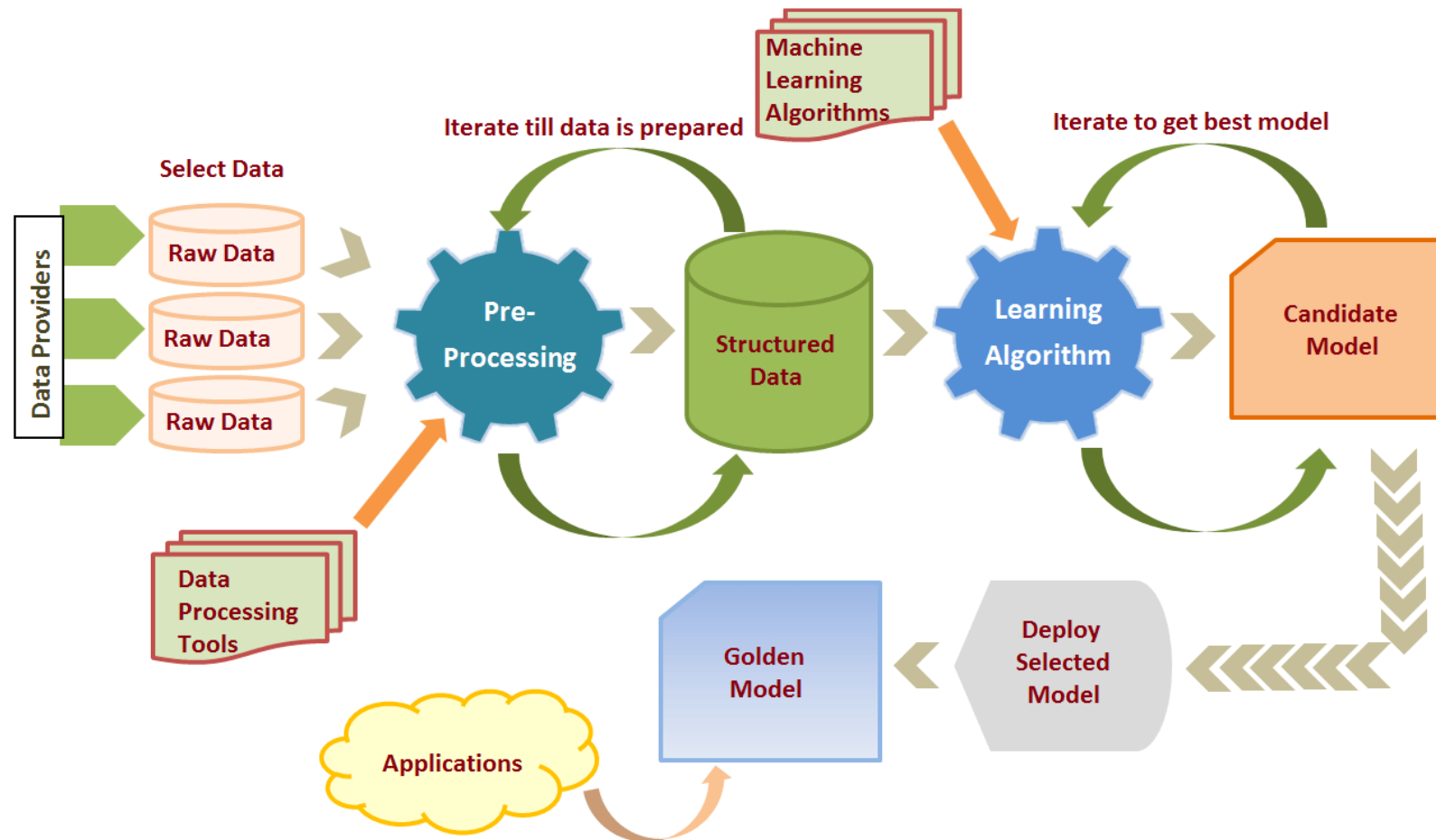
$P(H1) = 8/10 = 0.80 \geq 1 - 0.20$

Hence H1 is probably approximately correct

	Error(H1)	Error(h2)	
	0.04	0.04	$\epsilon = 0.05$
	0.03	0.035	$\delta = 0.20$
	0.09	0.039	$P(H1) = 8/10 = 0.80$
	0.06	0.06	$P(H1) = 8/10 = 0.80 \geq 1 - 0.20$
	0.025	0.025	Hence H1 is probably approximately correct
	0.049	0.059	$P(H2) = 7/10 = 0.70$
	0.04	0.04	$P(H2) = 7/10 = 0.70 < 1 - 0.20$
	0.03	0.03	Hence H2 is not probably approximately correct
	0.05	0.55	
	0.043	0.043	

Building Good Training Sets







L P U

NAAC
GRADE **A++**

Data Preprocessing



From Raw to Refined



Data Preprocessing

- Prepares data for machine learning models
 - Removes inconsistencies and irrelevant noise
 - Improves model reliability and accuracy
 - Includes cleaning, transforming, feature selection
 - Helps understand data characteristics deeply
 - Essential for real-world data challenges
-



Data Processing

Data Collection

Data Collection

Label

Ingest (Streaming, Batch)

Aggregate

Data Preparation (Wrangling during Interactive Data Analysis) Leverage Visualization for Exploratory Data Analysis (EDA)

Data Preprocessing

Clean (Replace, Impute,
Remove Outliers, Duplicates)

Partition (Train, Validate, Test)

Scale(Normalize, Standardize)

Unbias, Balance
(Detection & Mitigation)

Augment

Feature Engineering

Feature Selection

Feature Transformation

Feature Creation
(Encoding, Binning)

Feature Extraction
(Automated in Deep Learning)

Data Types

Qualitative (Categorical)

Ordinal
(ordered)

Example:
Test grade

Nominal
(not ordered)

Example:
Nationality

Quantitative

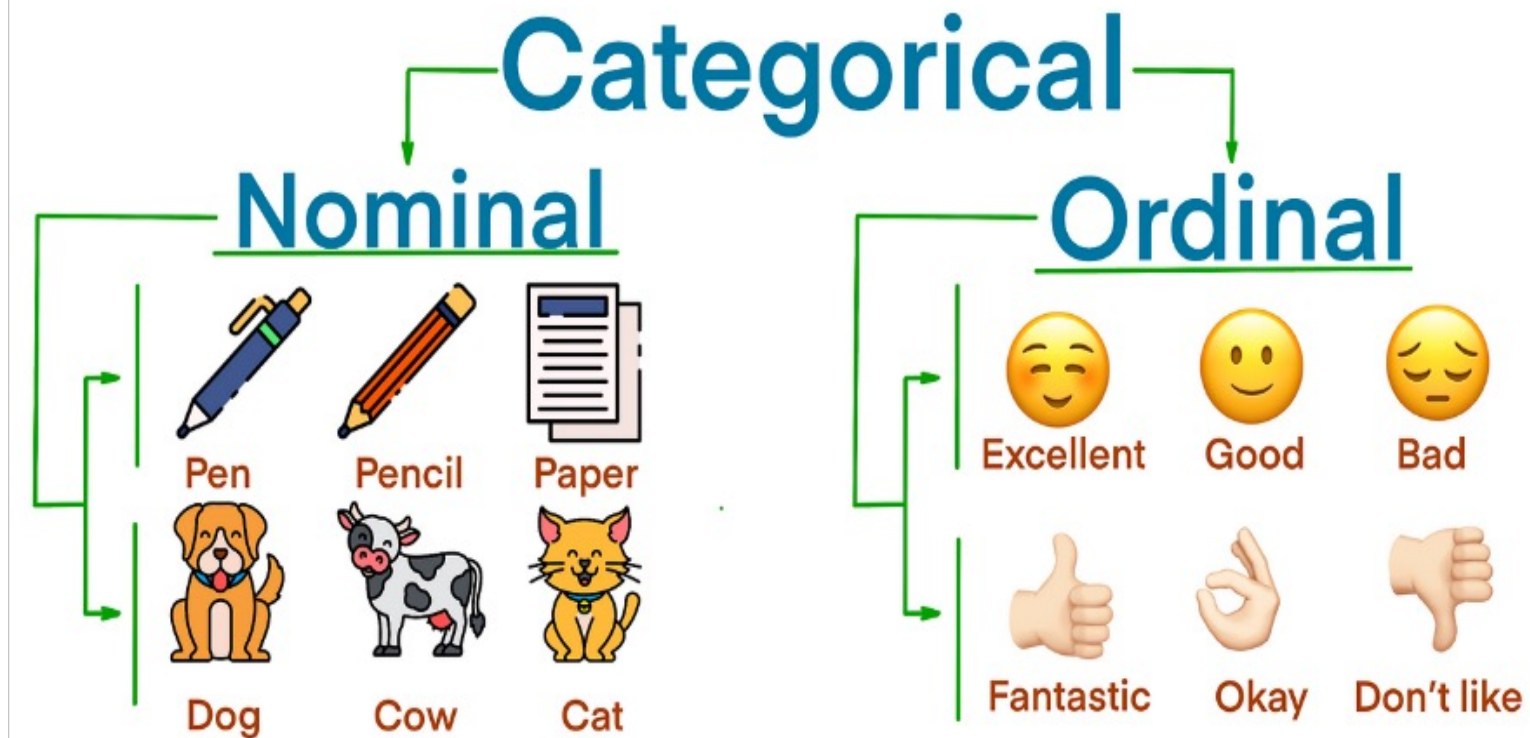
Continuous
(can be divided)

Example:
Distance

Discrete
(can't be divided)

Example:
Cats

Categorical Data





Dealing with Missing Values

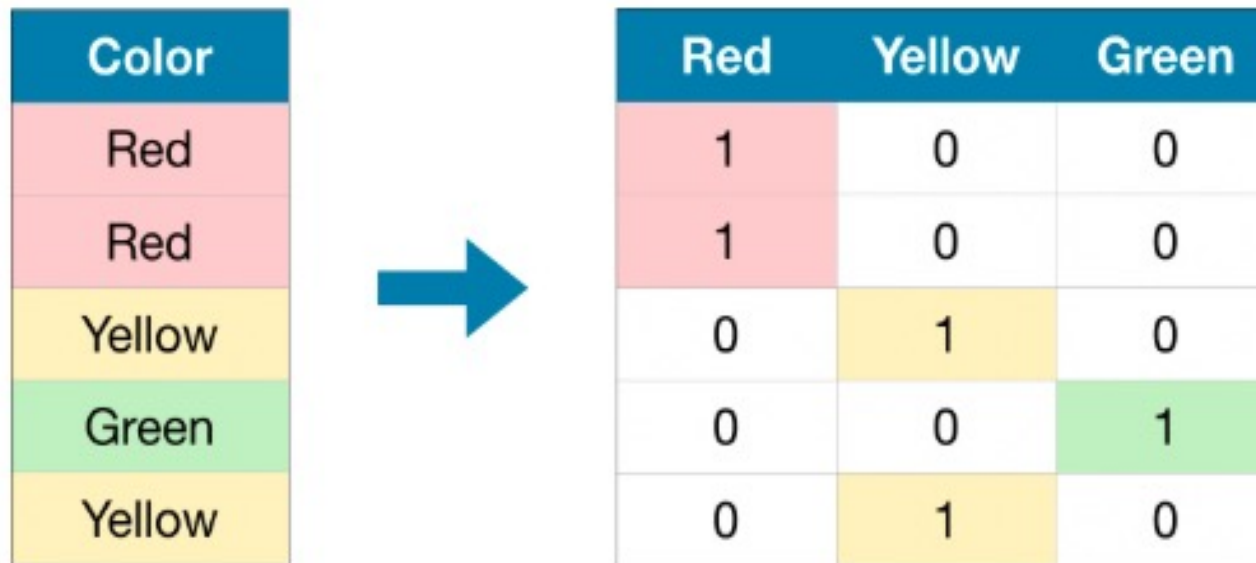
	School ID	Name	Address	City	Subject	Marks	Rank	Grade
0	101.0	Alice	123 Main St	Los Angeles	Math	85.0	2	B
1	102.0	Bob	456 Oak Ave	New York	English	92.0	1	A
2	103.0	Charlie	789 Pine Ln	Houston	Science	78.0	4	C
3	NaN	David	101 Elm St	Los Angeles	Math	89.0	3	B
4	105.0	Eva	NaN	Miami	History	NaN	8	D
5	106.0	Frank	222 Maple Rd	NaN	Math	95.0	1	A
6	107.0	Grace	444 Cedar Blvd	Houston	Science	80.0	5	C
7	108.0	Henry	555 Birch Dr	New York	English	88.0	3	B



Handling Categorical Data

- Label encode ordinal categorical values
 - Use one-hot encoding for independence
 - Map text to numerical equivalents
 - Manage unseen categories during testing
 - Avoid creating highly sparse matrices
 - Check for potential feature redundancies
-

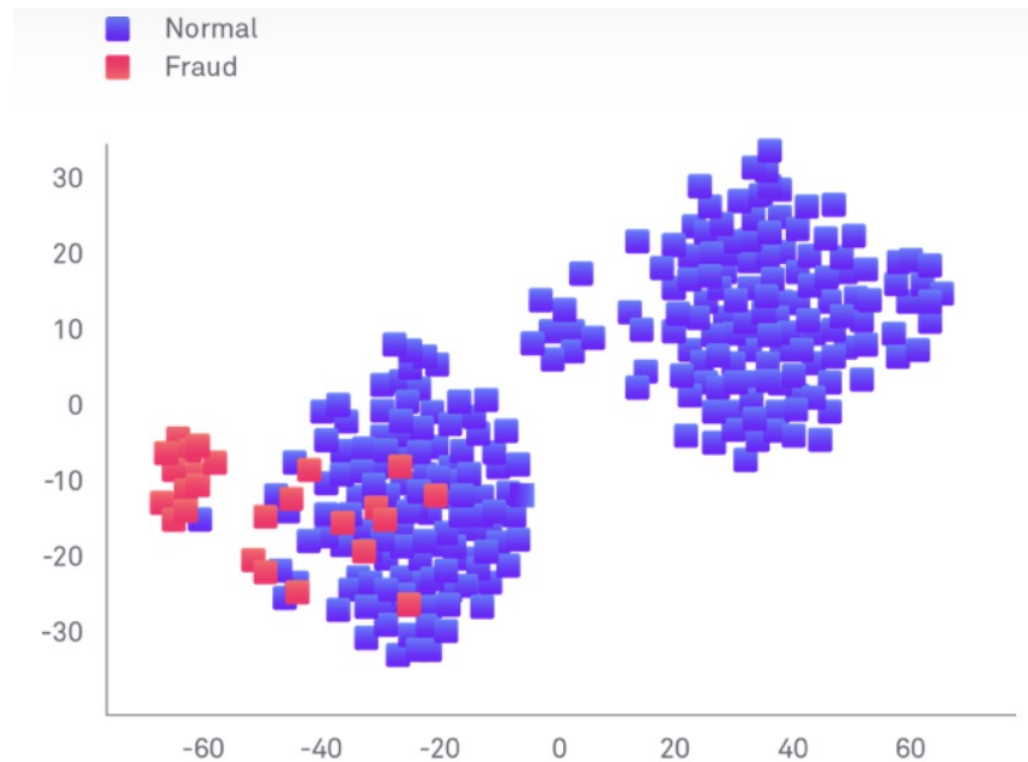
Handling Categorical Data



Color
Red
Red
Yellow
Green
Yellow

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1
0	1	0

Imbalanced Datasets

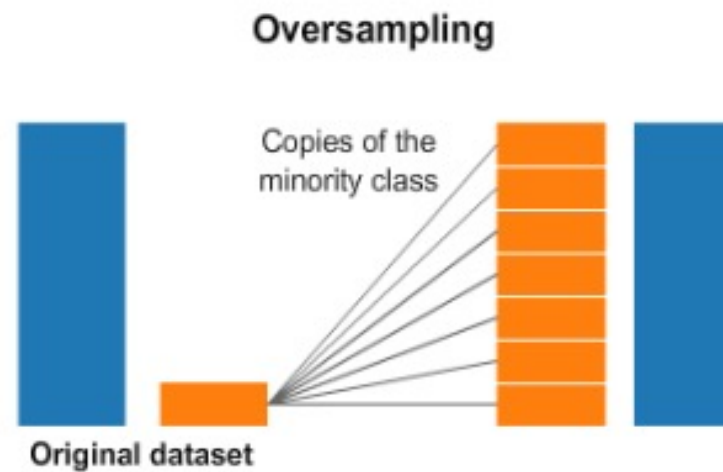
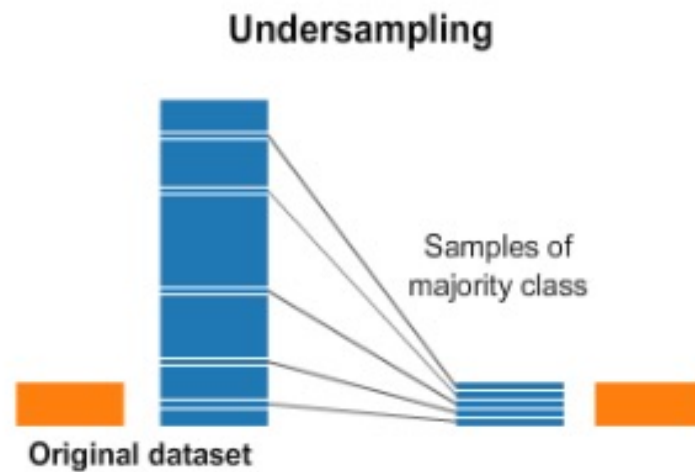




Handling Imbalanced Datasets

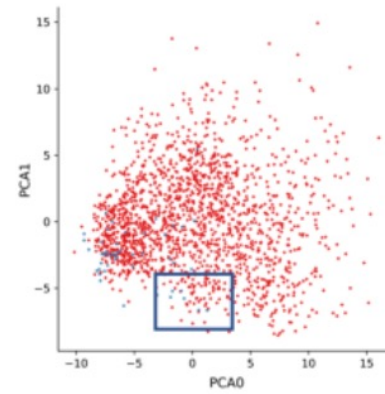
- Random Under-Sampler (RU)
 - Random Over-Sampler (RO)
 - Synthetic Minority Oversampling Technique (SMOTE)
 - Borderline SMOTE
 - SVM-SMOTE
 - Adaptive Synthetic Sampling (ADASYN)
-

Handling Imbalanced Datasets

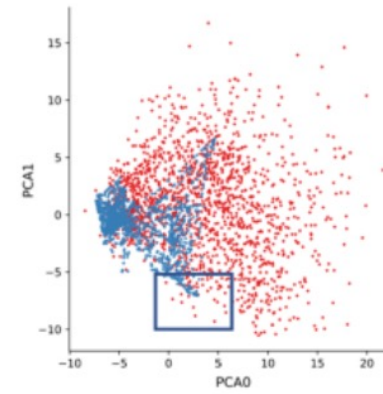




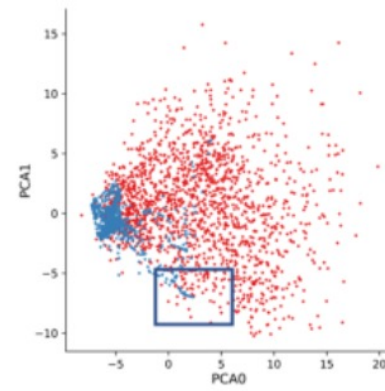
(a) Baseline



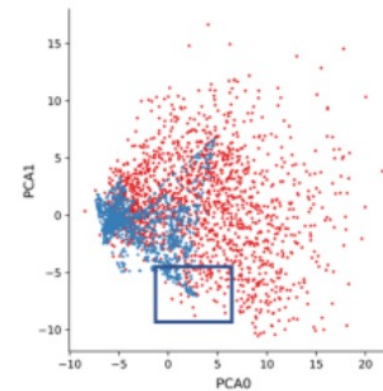
(b) SMOTE



(d) SVM SMOTE



(e) ADASYN





Handling Imbalanced Datasets: Random Under-Sampler (RU)

- **Concept:** Randomly removes samples from the majority class to balance the dataset.
 - **Pros:**
 - Reduces dataset size, making training faster.
 - Helps prevent overfitting to the majority class.
 - **Cons:**
 - Risk of losing important data points, potentially affecting model performance.
 - May not work well if the dataset is already small.
 - **Example:**
 - If Class 1 has 1000 samples and Class 2 has 100, RU randomly removes 900 samples from Class 1 to match Class 2.
-



Handling Imbalanced Datasets: Random Over-Sampler (RO)

- **Concept:** Randomly duplicates samples from the minority class to balance the dataset.
 - **Pros:**
 - Prevents loss of information.
 - Helps the model learn from more balanced data.
 - **Cons:**
 - Can lead to overfitting as the model sees repeated data points.
 - Does not create new informative examples.
 - **Example:**
 - If Class 1 has 1000 samples and Class 2 has 100, RO duplicates 900 samples from Class 2 to match Class 1.
-

Handling Imbalanced Datasets: Synthetic Minority Oversampling Technique (SMOTE)

- **Concept:** Instead of duplicating existing samples, SMOTE generates **synthetic samples** by interpolating between existing minority class instances.
 - **How it Works:**
 - Selects a minority class sample randomly.
 - Identifies its k-nearest neighbors (typically k=5).
 - Randomly selects one of these neighbors.
 - Creates a new synthetic data point along the line connecting the original sample and the selected neighbor.
 - **Pros:**
 - Reduces overfitting by introducing new synthetic data.
 - Better generalization compared to random oversampling.
 - **Cons:**
 - Can generate unrealistic synthetic samples.
 - Does not consider class distribution near decision boundaries.
-



Handling Imbalanced Datasets: Borderline SMOTE

- **Concept:** A modified version of SMOTE that focuses on **creating synthetic samples near the decision boundary**.
 - **How it Works:**
 - Identifies “**borderline**” **minority samples** (those close to majority class samples).
 - Generates synthetic samples **near these borderline points**.
 - Enhances the learning of hard-to-classify instances.
 - **Pros:**
 - Improves classifier performance in complex datasets with overlapping classes.
 - Helps models learn better decision boundaries.
 - **Cons:**
 - May still generate unrealistic samples if not tuned properly.
 - Does not work well if borderline samples are misclassified.
-



Handling Imbalanced Datasets: SVM-SMOTE

- **Concept:** Uses Support Vector Machines (SVM) to identify “support vectors” in the minority class and generates synthetic samples near them.
 - **How it Works:**
 - Trains an SVM model to find the decision boundary.
 - Identifies support vectors from the minority class.
 - Generates synthetic samples along the margin.
 - **Pros:**
 - Creates more informative synthetic samples.
 - Helps models learn more meaningful patterns.
 - **Cons:**
 - Computationally expensive due to SVM training.
 - May not work well in very high-dimensional spaces.
-



Handling Imbalanced Datasets: Adaptive Synthetic Sampling (ADASYN)

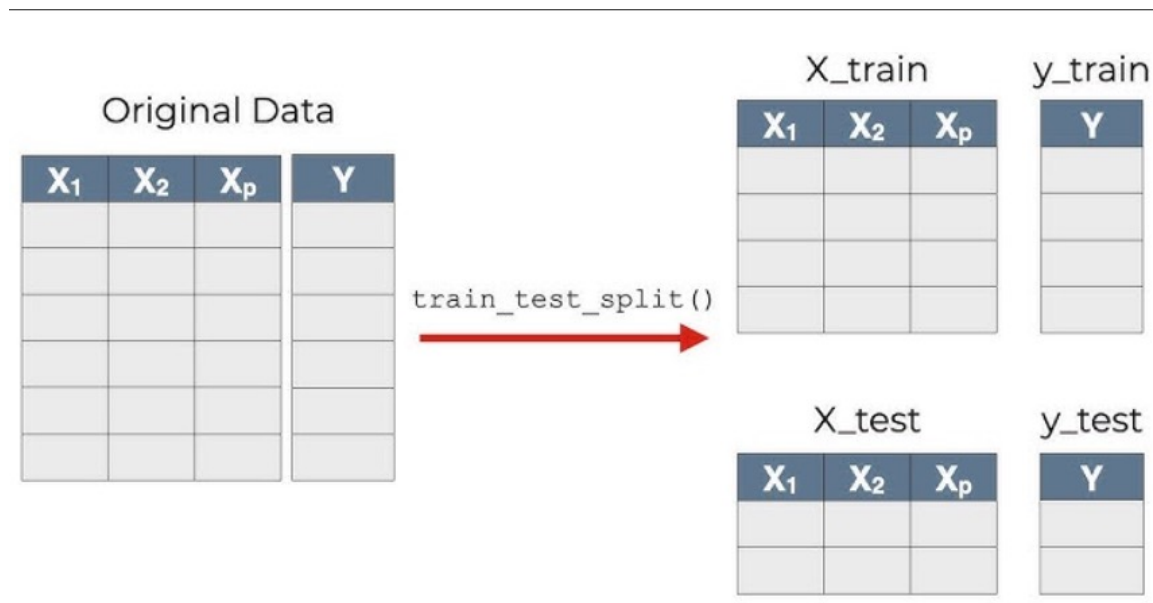
- **Concept:** An extension of SMOTE that adaptively generates synthetic samples based on the **difficulty of learning minority samples**.
 - **How it Works:**
 - Assigns **weights to minority class samples based on their difficulty**.
 - Generates more synthetic samples for difficult-to-classify points.
 - Balances the dataset dynamically based on data complexity.
 - **Pros:**
 - Focuses more on hard-to-classify minority samples.
 - Reduces the chance of over-generalization.
 - **Cons:**
 - Can create noise if difficult samples are actually outliers.
 - Requires careful tuning to avoid excessive synthetic samples.
-

Comparison of Imbalance Techniques

Technique	Approach	Advantage	Disadvantage
Random Under-Sampler	Removes majority samples	Faster training, reduces bias	Loss of information
Random Over-Sampler	Duplicates minority samples	Preserves all data	Risk of overfitting
SMOTE	Creates synthetic samples	Reduces overfitting	Can generate unrealistic samples
Borderline SMOTE	Focuses on borderline samples	Improves decision boundary learning	May create unrealistic samples
SVM-SMOTE	Uses SVM to find important samples	Generates more meaningful data	Computationally expensive
ADASYN	Generates samples adaptively	Focuses on hard-to-classify points	Can introduce noise



Partitioning Dataset





Partitioning Dataset

- Split data into training and testing sets
 - Use validation set for model tuning
 - Ensure proportional representation in splits
 - Avoid data leakage during splitting process
 - Test set remains unseen during training
-



Partitioning Dataset

a) Train-Test Split

- **Training Set:** Used to train the model. The model learns patterns and relationships from this data.
- **Test Set:** Used to evaluate the final model's performance on unseen data.

✓ Typical Split:

- **80% Training, 20% Testing** (or)
 - **70% Training, 30% Testing**
-



Partitioning Dataset

b) Train-Validation-Test Split

- **Training Set:** Used to train the model.
- **Validation Set:** Used for hyperparameter tuning and model selection.
- **Test Set:** Used for final performance evaluation.
-  **Typical Split:**
 - **70% Training, 15% Validation, 15% Testing**
 - **60% Training, 20% Validation, 20% Testing**

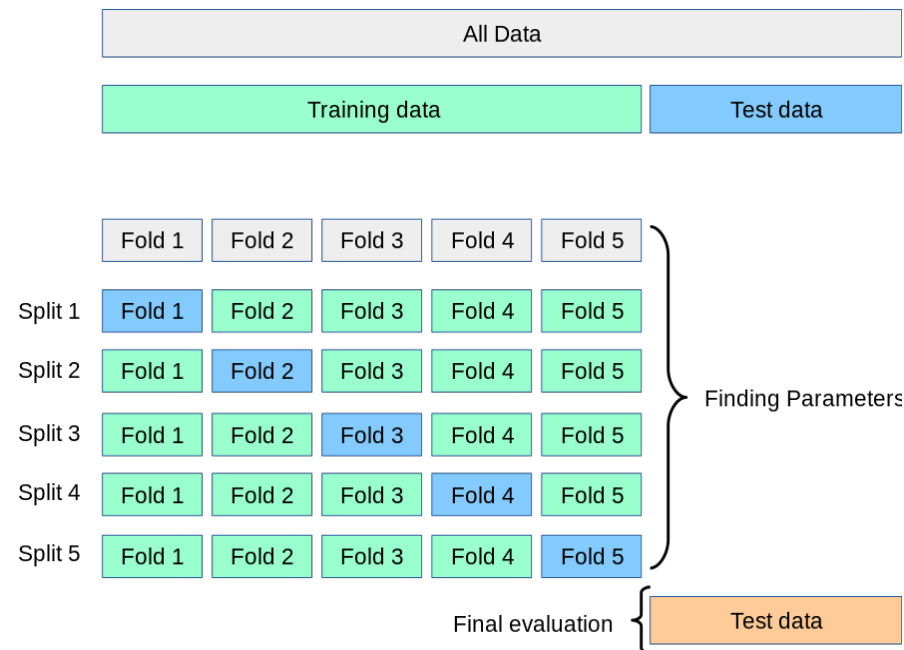


Partitioning Dataset

- **from** sklearn.model_selection **import** train_test_split
- # Assume X (features) and y (labels)
- X_train, X_test, y_train, y_test = train_test_split(X, y, **test_size=0.2**, random_state=42)

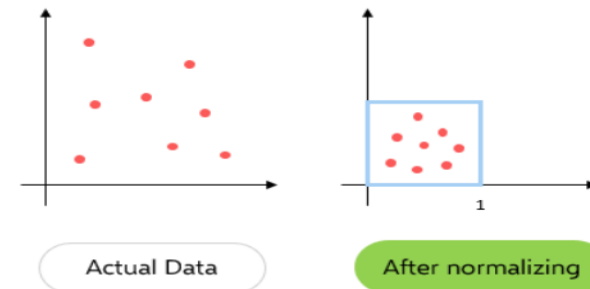
Partitioning Dataset: **K-Fold Cross Validation**

- ▶ **K-fold cross-validation** is a technique used to assess the performance of a machine learning model and ensure that it **generalizes well to unseen data**.
- ▶ It helps to **prevent overfitting or underfitting** by evaluating the model on different portions of the dataset.



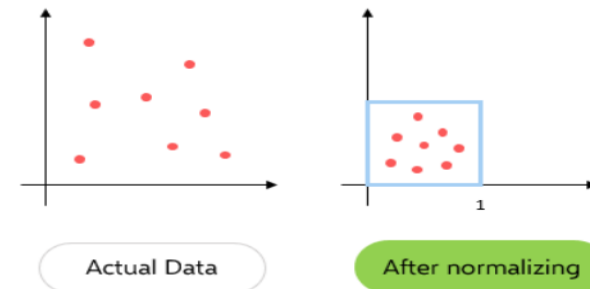
Normalization

- Scale data for consistent feature ranges
- Min-Max scaling for bounded values
- Standardize to zero mean and unit variance
- Improve convergence rates for optimization
- Essential for distance-based algorithms like KNN
- Avoid scale-related bias in models

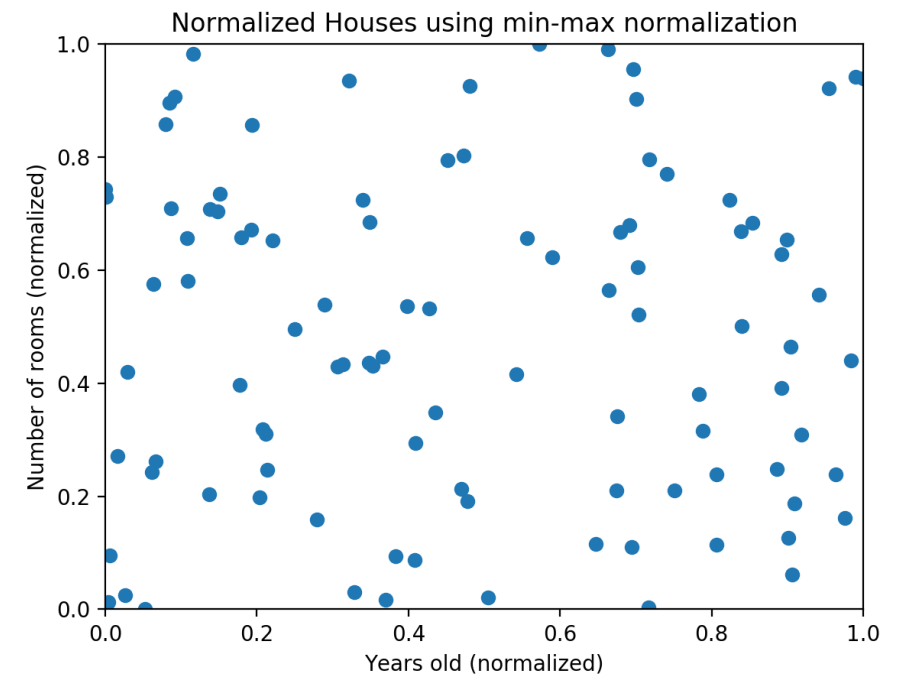
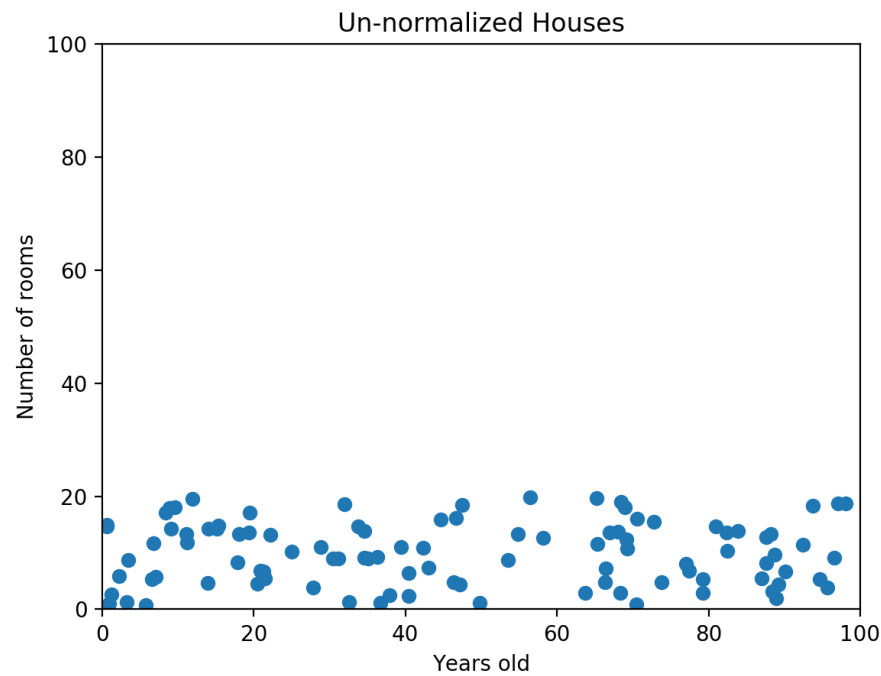


Normalization

- Scale data for consistent feature ranges
- Min-Max scaling for bounded values
- Standardize to zero mean and unit variance
- Improve convergence rates for optimization
- Essential for distance-based algorithms like KNN
- Avoid scale-related bias in models



Normalization



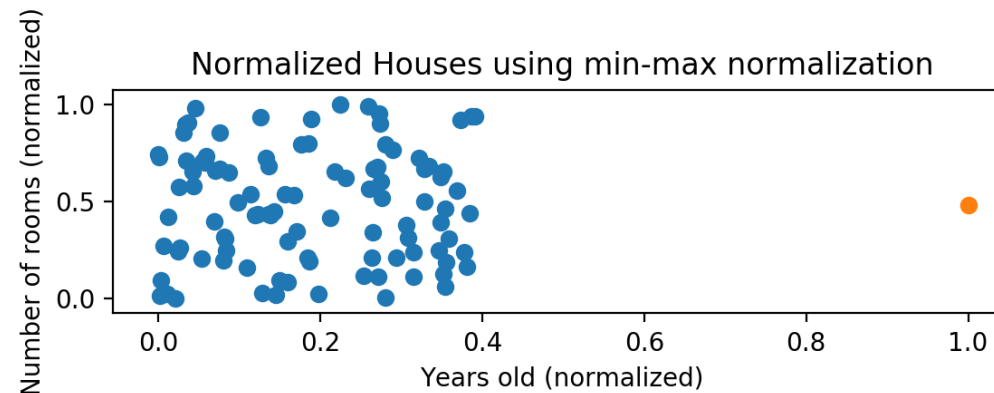
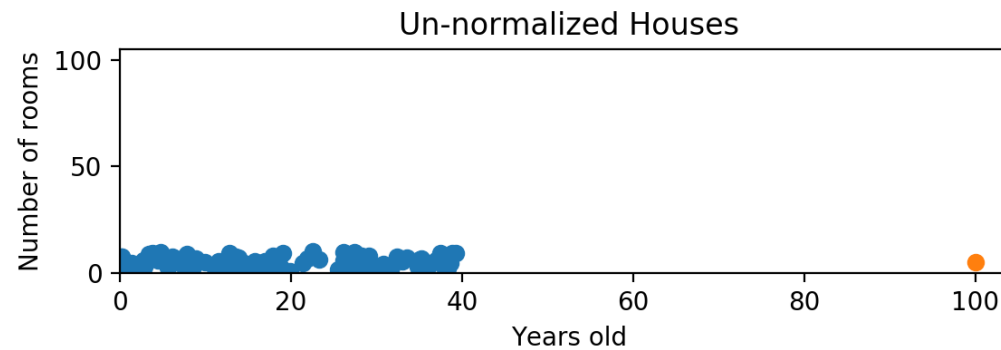


Normalization: Min-Max Normalization

- Min-max normalization is one of the **most common ways** to normalize data.
- For every feature, the **minimum value** of that feature gets **transformed into a 0**, the **maximum value** gets transformed into a **1**, and every other value gets transformed into a decimal between 0 and 1.
- For example, if the minimum value of a feature was 20, and the maximum value was 40, then 30 would be transformed to about 0.5 since it is halfway between 20 and 40. The formula is as follows:

$$(Value - Min / Max - Min)$$

Normalization: Min-Max Normalization





Normalization: Z-Score Normalization

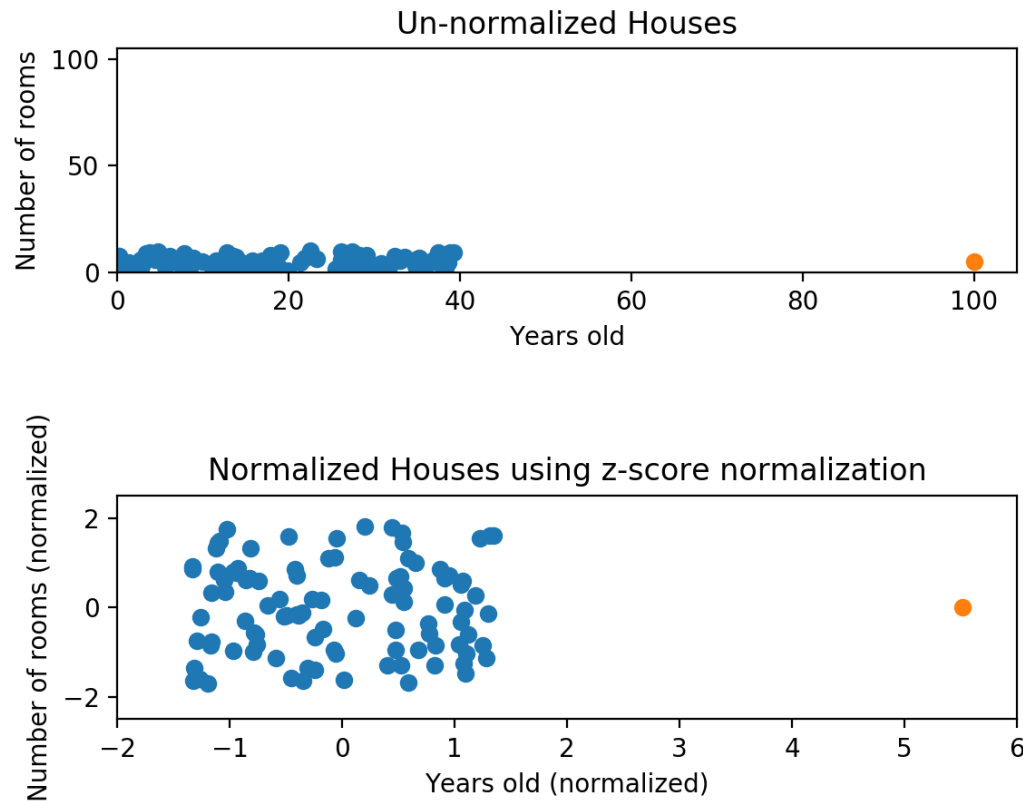
- Z-score normalization is a strategy of normalizing data that **avoids this outlier issue**. The formula for Z-score normalization is below:

$$\frac{(\text{Value} - \mu)}{\sigma}$$

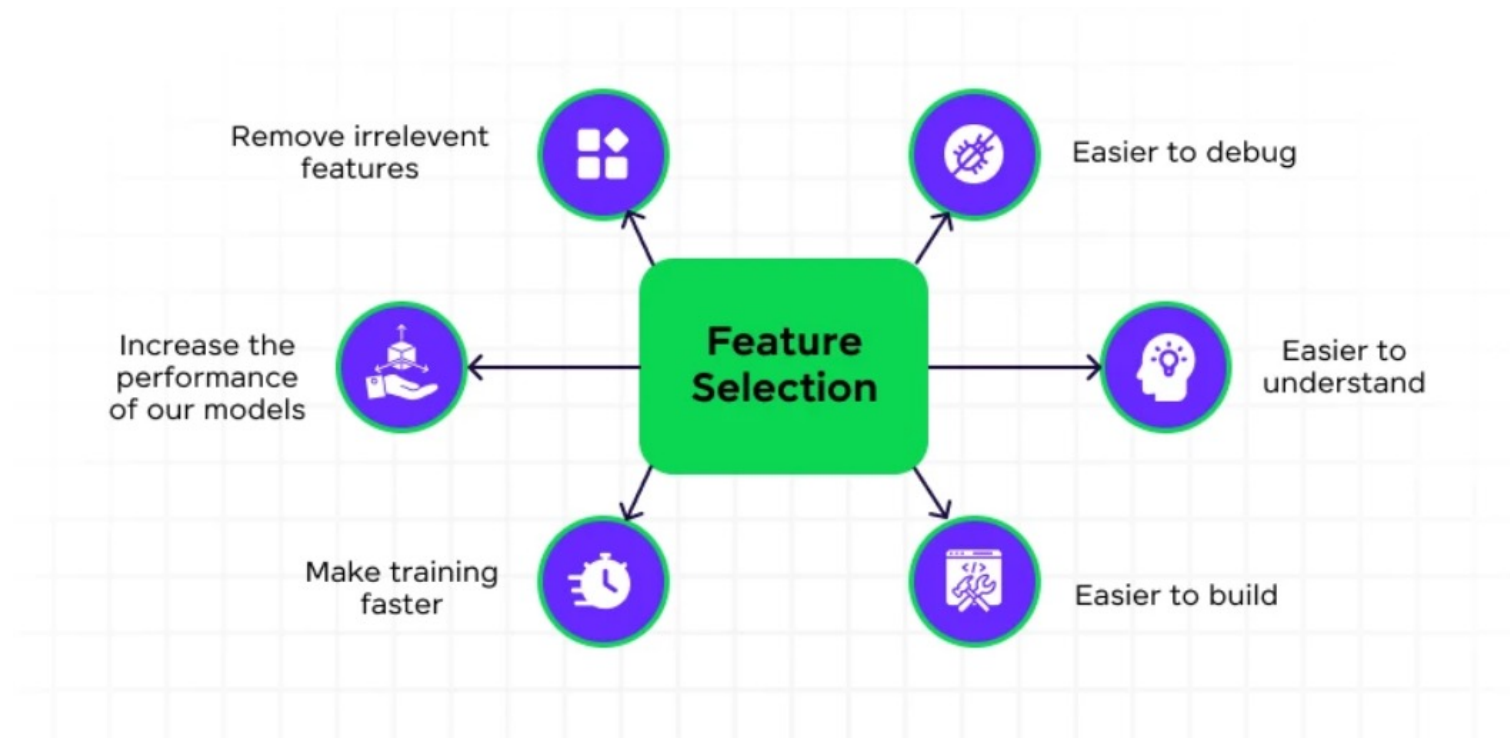
- Here, μ is the mean value of the feature and σ is the standard deviation of the feature.
- If a value is exactly equal to the mean of all the values of the feature, **it will be normalized to 0**.
- If it is below the mean, it will be a **negative number**, and if it is above the mean it will be a **positive number**.
- The **size of those negative and positive numbers** is determined by the standard deviation of the original feature.



Normalization: Z-Score Normalization

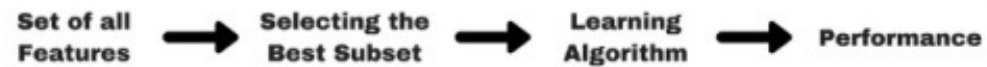


Feature Selection

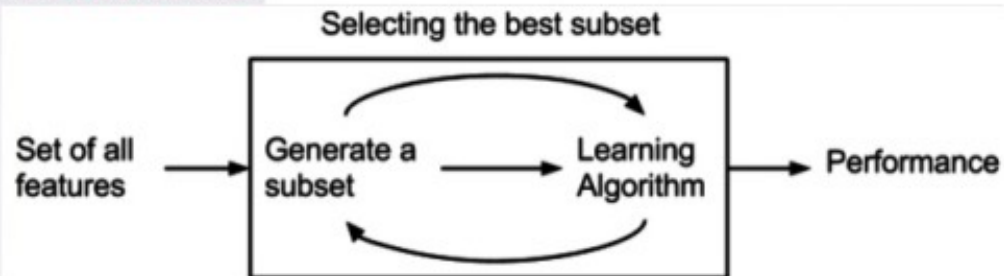


Feature Selection Methods

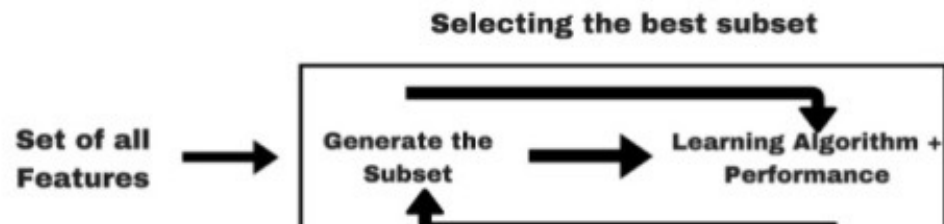
Filter Methods



Wrapper Methods



Embedded Methods





Feature Selection Methods

- **Filter Methods** → When you have **high-dimensional data** and need a quick selection.
 - **Wrapper Methods** → When **accuracy** is a priority and you can afford high computational costs.
 - **Embedded Methods** → When you want a **balance** between speed and accuracy with model-driven feature selection.
-

Dimensionality Reduction



A technique in
machine learning and
data analysis

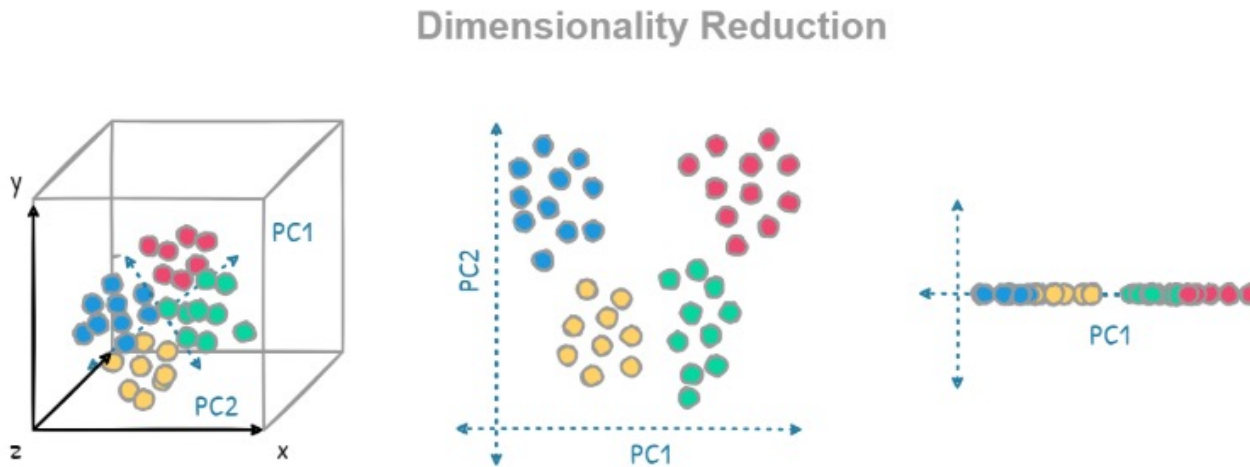


Reduces the number of
variables in a dataset
while retaining the
critical information



Helps in simplifying
complex datasets

Dimensionality Reduction



Dimensionality Reduction

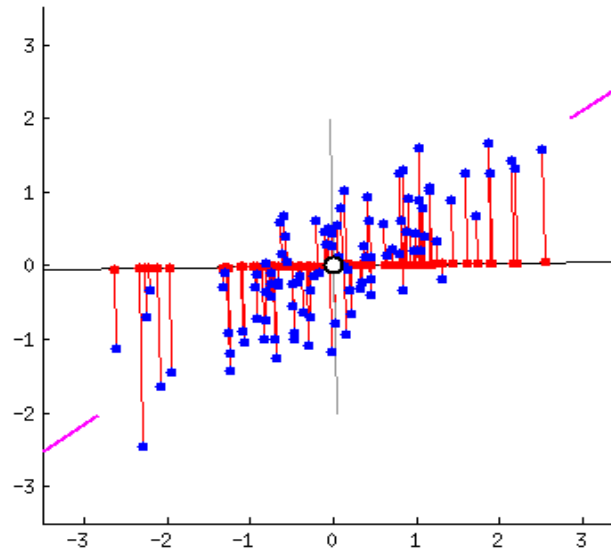
- ▶ There are two components of dimensionality reduction:
- ▶ **Feature selection**: In this, we try to find a subset of the original set of variables, or features, to get a smaller subset which can be used to model the problem. It usually involves three ways:
 - Filter
 - Wrapper
 - Embedded
- ▶ **Feature extraction**: This reduces the data in a high dimensional space to a lower dimension space, i.e. a space with lesser no. of dimensions.
- ▶ **Methods of Dimensionality Reduction**: The various methods used for dimensionality reduction include:
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - Generalized Discriminant Analysis (GDA)

Dimensionality Reduction: PCA

- ▶ Imagine you're analyzing a dataset with dozens of features, like a customer survey with age, income, purchase history, and website behavior.
 - ▶ While rich, this high dimensionality can be a curse: complex models, slower training, and even irrelevant info hiding the good stuff. That's where **Principal Component Analysis (PCA)** comes in as your dimensionality reduction hero!
 - ▶ **The Gist:** PCA takes your high-dimensional data and squishes it into a lower-dimensional space, capturing the most important information but ditching the redundancy. **Think of it like summarizing a long lecture into key points – you lose some detail, but the core meaning remains.**
-

Dimensionality Reduction: PCA

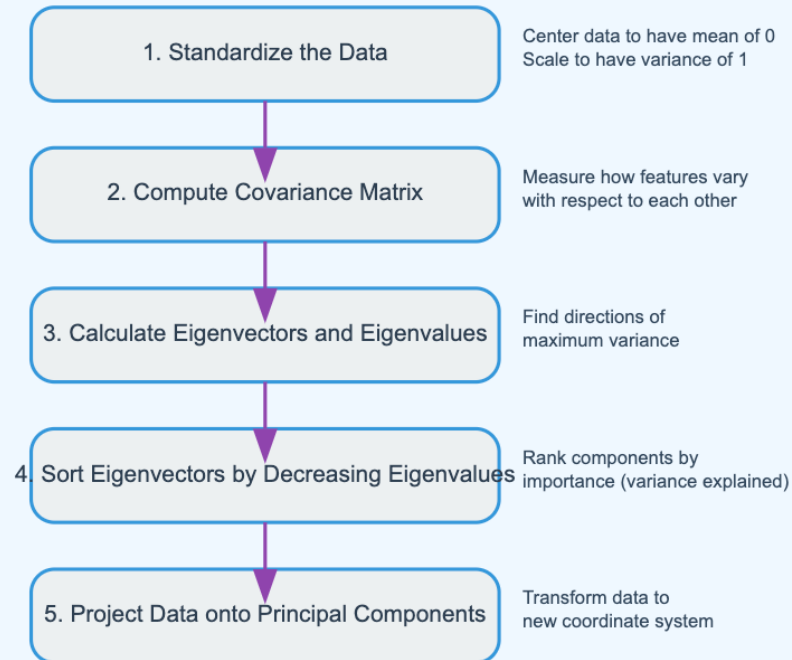
- ▶ For example, let's assume that the scatter plot of our data set is as shown below, can we guess the first principal component ?
- ▶ It's approximately the line that matches the purple marks because it goes through the origin and it's the line in which the projection of the points (red dots) is the most spread out. Or mathematically speaking, it's the line that maximizes the variance (the average of the squared distances from the projected points (red dots) to the origin).



Principal Component Analysis (PCA)

PCA Algorithm Flowchart

Step-by-Step Process of Principal Component Analysis



Key Points:

1. PCA is an iterative process that transforms the original data.
2. The goal is to find a new basis that best expresses the data's variance.
3. The resulting principal components are orthogonal to each other.



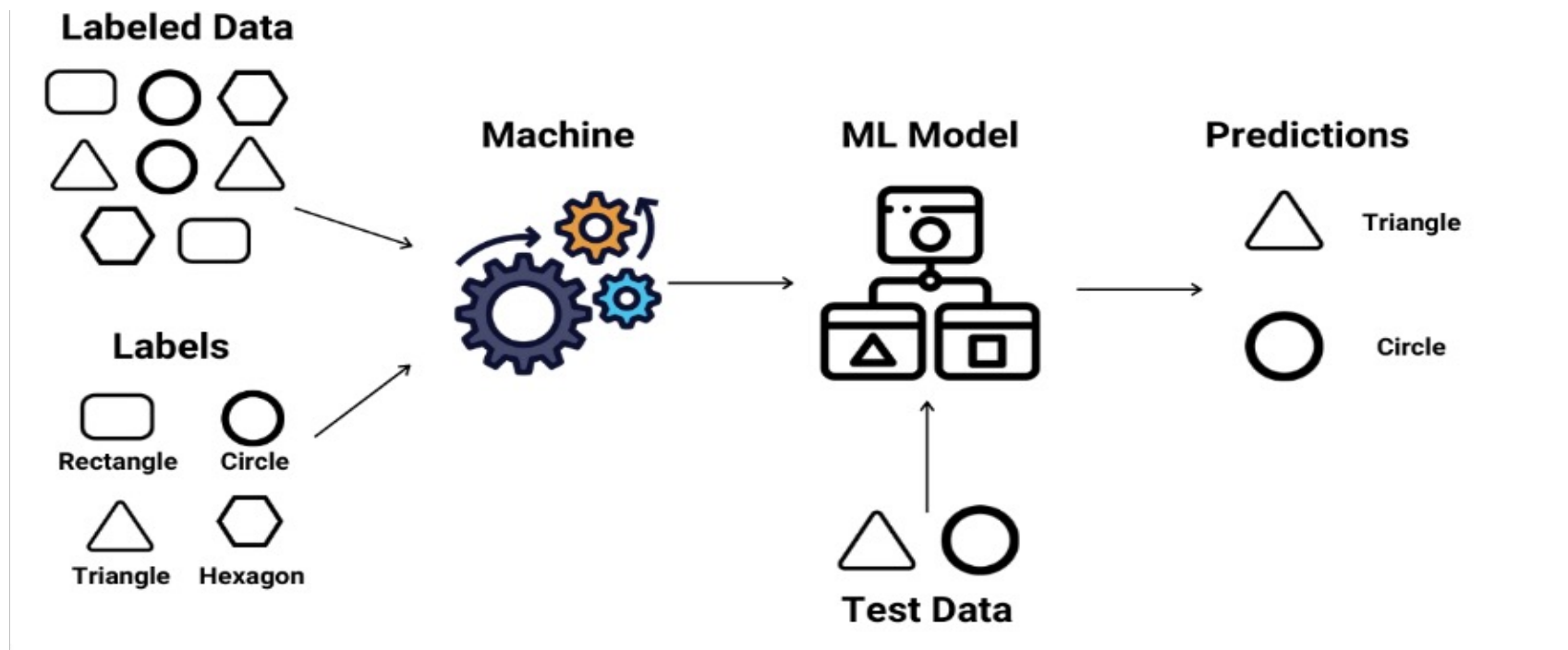
L P U

NAAC
GRADE **A++**

Machine Learning Classifier-I



Overview of Supervised Learning





Regression Vs Classification

Regression	Classification
The task of predicting a continuous quantity is known as regression.	The classification process involves anticipating a discrete class label.
The task is to map the input value (x) to the continuous output variable (y).	The task is to map the input value(x) to the discrete output variable (y).
A regression model can predict a discrete value, but only in the form of an integer quantity.	A classification model can predict a continuous value, but it is in the form of a probability for a class label.
The output variable in regression must be of continuous value.	The output variable in Classification must be discrete.

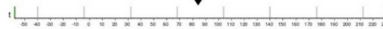
Regression Vs Classification

Regression



What will be the temperature tomorrow?

84°



Fahrenheit

Classification



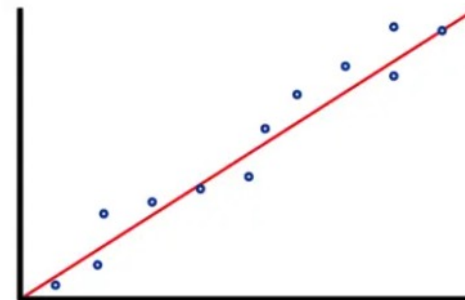
Will it be hot or cold tomorrow?

COLD

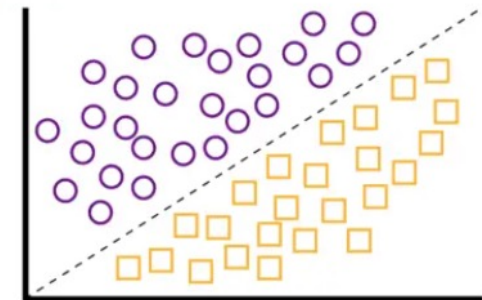
HOT



Fahrenheit



Regression



Classification



L P U

NAAC
GRADE **A++**

Real-life examples of Supervised Learning Techniques



Classifying emails as spam or not spam



Categorizing patients into different disease categories based on symptoms and test results



Identifying objects in images



Determining sentiments of text data



Predicting the grade of student in ML



Predicting that whether India is going to win the match or not



Student is going to get the placement or not

Classification Examples



Estimating the price of house based on price, location and area



Predicting future sales of product based on historical data



Forecasting the future price of stock based on financial indicators



Predicting temperature, rainfall and other weather conditions



Predicting creditworthiness of an individual or company based on financial history



Predicting product quality based on manufacturing process variables



Predict student performance based on attendance, grades and test scores

Regression Examples



Choosing a Classification Algorithm

- **a) Small dataset (<10,000 samples)** → Classical ML models (Logistic Regression, SVM, Decision Tree)
 - **Large dataset (>10,000 samples)** → Deep learning models (CNN, LSTM, Transformer)
 - **b) Number of Features (Dimensionality)**
 - **Low-dimensional data (<20 features)** → Logistic Regression, Naïve Bayes, Decision Tree
 - **High-dimensional data (>1000 features)** → SVM (with kernel trick), Deep Learning, Feature Selection needed
-



Choosing a Classification Algorithm

- **c) Type of Features**
 - **Numerical** → Logistic Regression, Random Forest, XGBoost, Neural Networks
 - **Categorical** → Decision Trees, Random Forest, XGBoost
 - **Mixed (Numerical + Categorical)** → CatBoost, XGBoost, Decision Trees
 - **d) Data Imbalance**
 - **Balanced** → Any model works well
 - **Imbalanced** → Consider:
 - Resampling (Oversampling, SMOTE, Undersampling)
 - Weighted loss function (XGBoost, CatBoost, Deep Learning)
-



Choosing a Classification Algorithm

- **a) Interpretability**
 - **High Interpretability Required** → Logistic Regression, Decision Trees, Naïve Bayes
 - **Low Interpretability (but High Accuracy)** → Random Forest, XGBoost, Deep Learning
 - **b) Computational Complexity**
 - **Low Complexity (Fast Training & Inference)** → Logistic Regression, Naïve Bayes, SVM (linear)
 - **Medium Complexity** → Decision Trees, Random Forest, XGBoost
 - **High Complexity (Slow Training, High GPU Requirements)** → Deep Learning (CNN, LSTM, Transformers)
-



Choosing a Classification Algorithm

- **a) Accuracy vs Speed Trade-off**
 - **High Accuracy Required** → XGBoost, Random Forest, Deep Learning
 - **Fast Computation Needed** → Logistic Regression, Naïve Bayes
 - **b) Sensitivity to Noise**
 - **Robust to Noise** → Random Forest, XGBoost, Deep Learning
 - **Sensitive to Noise** → SVM, Decision Trees (prone to overfitting)
 - **c) Overfitting Risk**
 - **Low Risk** → Random Forest, XGBoost (with regularization), Ridge/Lasso Regression
 - **High Risk** → Decision Trees (without pruning), Deep Learning (without dropout)
-



Choosing a Classification Algorithm

- **a) Sequential or Time-Series Data**
 - **Yes** → LSTM, GRU, Transformer, 1D-CNN
 - **No** → Use traditional ML models
 - **b) Multiclass vs Binary**
 - **Binary Classification** → Logistic Regression, SVM, Decision Trees, Deep Learning
 - **Multiclass (≥ 3 classes)** → Random Forest, XGBoost, Neural Networks (softmax activation)
 - **c) Streaming / Real-time Processing**
 - **Yes** → Online Learning Models (Incremental Learning with SGD, Adaptive Boosting, Streaming Decision Trees)
 - **No** → Batch Training Methods (Deep Learning, XGBoost)
-

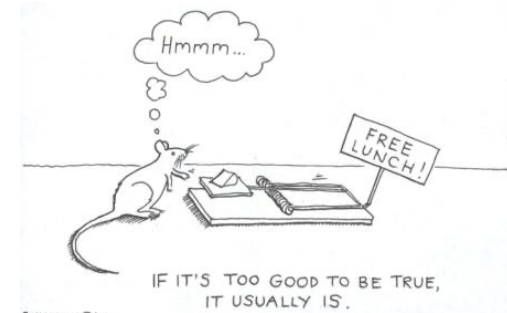


Choosing a Classification Algorithm

Use Case	Recommended Algorithm
Text Classification	Naïve Bayes, Transformer (BERT), LSTM
Image Classification	CNN, ResNet, EfficientNet, Vision Transformer
Tabular Data Classification	XGBoost, Random Forest, Logistic Regression
Anomaly Detection	Isolation Forest, Autoencoders, One-Class SVM
Medical Diagnosis	XGBoost, Random Forest, CNN for Images
Fraud Detection	Random Forest, XGBoost, Autoencoders

No Free Lunch Theorem

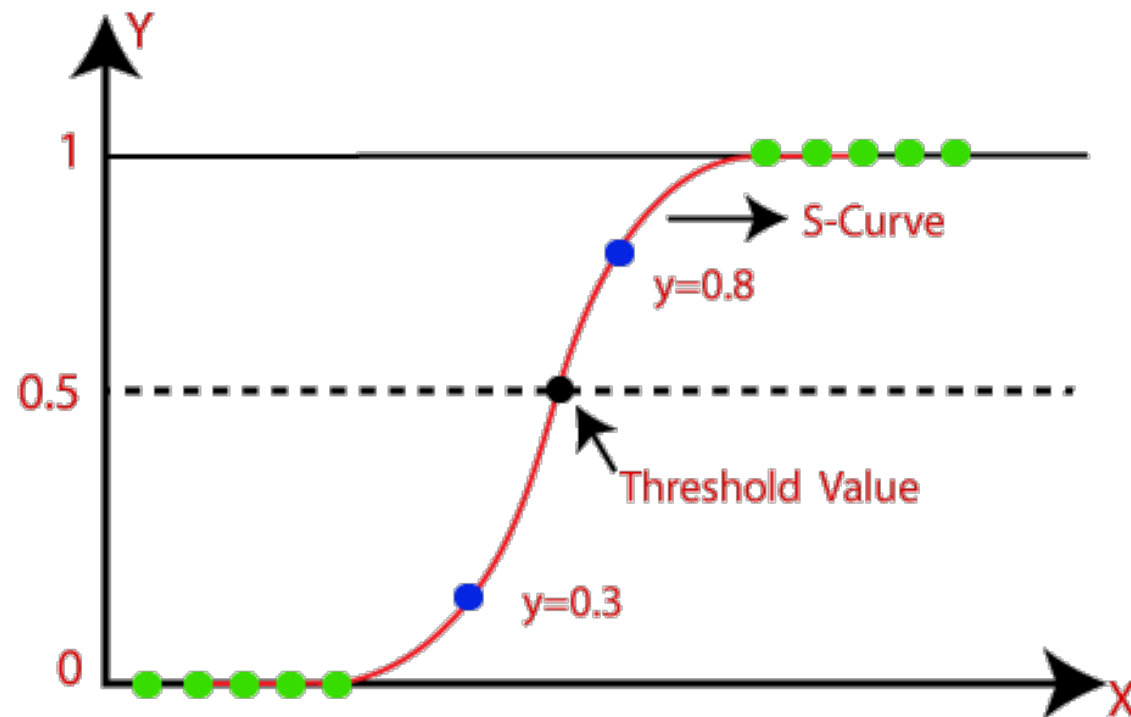
- **No algorithm** best for all problems.
- Performance depends on **problem context**.
- **Data matters** more than the algorithm
- Trade-offs exist: **Accuracy vs. Efficiency**.
- Evaluate models using **cross-validation methods**.
- Experiment with **hyperparameters for optimization**.
- Choose algorithm based on **specific requirements**.



Logistic Regression

- Logistic regression predicts binary class probabilities.
 - Uses sigmoid function to map outputs.
 - Optimized using cross-entropy loss function.
 - Works well for linearly separable data.
 - **Extension:** Multinomial logistic for multiple classes.
 - Widely used for classification problems.
-

Logistic Regression



Logistic Regression Algorithm

Step 1: Compute Linear Combination of Features

Logistic Regression starts with a **weighted sum of input features**:

$$Z = w_1X_1 + w_2X_2 + \dots + w_nX_n + b$$

Step 2: Apply the Sigmoid Activation Function

Instead of predicting **any real number**, we transform Z using the **sigmoid function**:

$$\sigma(Z) = \frac{1}{1 + e^{-Z}}$$

where:

- $\sigma(Z)$ **squashes the output between 0 and 1**, making it a probability.
 - If $\sigma(Z) > 0.5$, classify as **1 (Positive Class)**.
 - If $\sigma(Z) \leq 0.5$, classify as **0 (Negative Class)**.
-

Logistic Regression Algorithm

Step 3: Define the Cost Function (Log Loss)

Instead of Mean Squared Error (MSE), we use **Log Loss (Binary Cross-Entropy)**:

$$J(w) = -\frac{1}{m} \sum_{i=1}^m [y_i \log y_i^{\wedge} + (1 - y_i) \log(1 - y_i^{\wedge})]$$

- **Minimizing Log Loss** ensures that the model maximizes the probability of correct predictions.

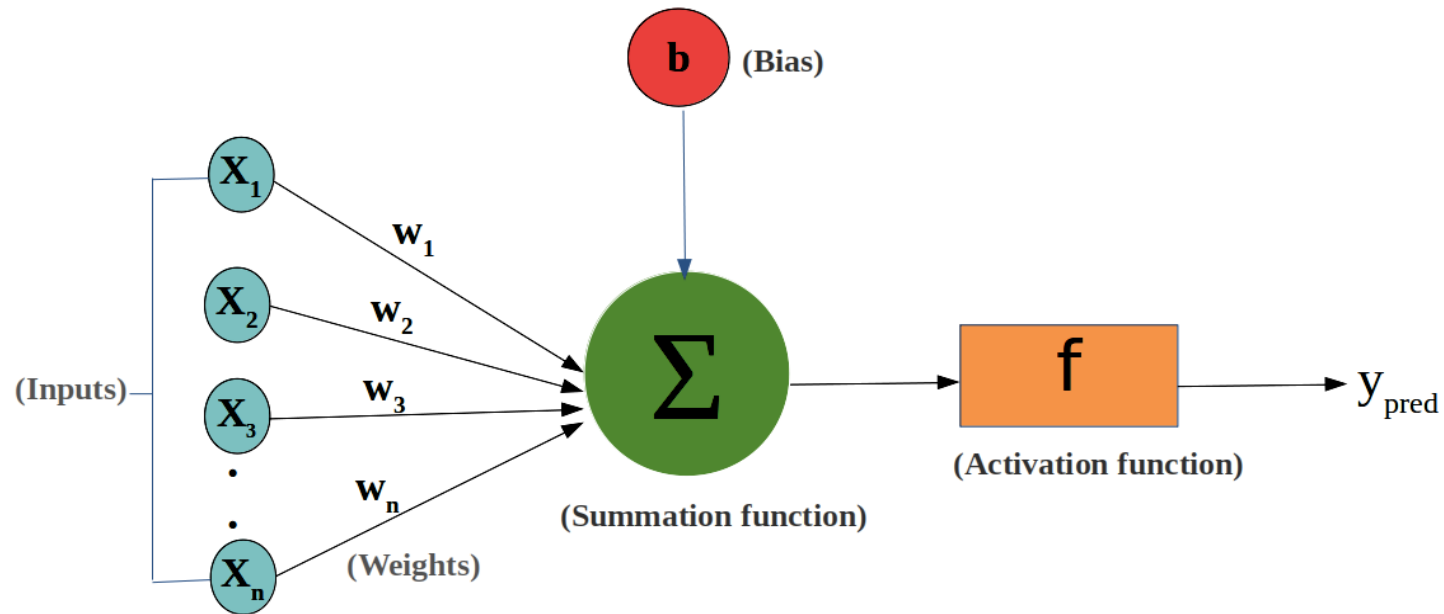
Step 4: Optimize Weights Using Gradient Descent

Logistic Regression updates weights using **Gradient Descent**:

$$w_j = w_j - \alpha \left(\frac{\partial J}{\partial w_j} \right)$$

- **Gradient Descent** ensures the cost function is minimized.

Perceptron





Perceptron Algorithm

- **Step 1: Initialize Weights & Bias**
 - . Set all weights (w_i) and bias (b) to small random values or zeros.
- **Step 2: Compute the Output**
- For each training sample (X, y):
 - . Compute the weighted sum: $Z = \sum w_i X_i + b$
 - . Apply the **step activation function**:
 - $y_{\text{pred}} = \{1: \text{if } Z \geq 0, 0: \text{if } Z < 0\}$



Perceptron Algorithm

- **Step 3: Update Weights Using the Perceptron Rule**

- If the predicted output y_{pred} **matches** the actual label y , do nothing.
- If **incorrect**, update the weights using the rule:
 - $w_i = w_i + \eta (y - y_{\text{pred}}) X_i$
 - $b = b + \eta (y - y_{\text{pred}})$

- **Step 4: Repeat Until Convergence**

- Iterate over the dataset multiple times (epochs) until:
 - All samples are classified correctly.
 - A stopping criterion (max iterations or minimal error) is met.



Decision Tree Classifier

- Models decisions as **tree-like structures**.
 - Splits nodes using **feature thresholds recursively**.
 - Handles both **categorical and numerical data**.
 - Overfitting mitigated using **pruning techniques**.
 - Easy to interpret, **lacks generalization sometimes**.
 - Algorithms include **ID3, CART, and C4.5**.
-



Decision Tree Classifier

- A Decision Tree might split the data as follows:
 - **First Split on Credit Score:**
 - If **Credit Score > 700** $\rightarrow$ **No Default**
 - If **Credit Score ≤ 700** $\rightarrow$ Check Age.
 - **Second Split on Age:**
 - If **Age ≤ 30** $\rightarrow$ **Default**
 - If **Age > 30** $\rightarrow$ **No Default**
 - This **hierarchical decision-making** forms a **tree-like structure**.
-

Decision Tree Classifier

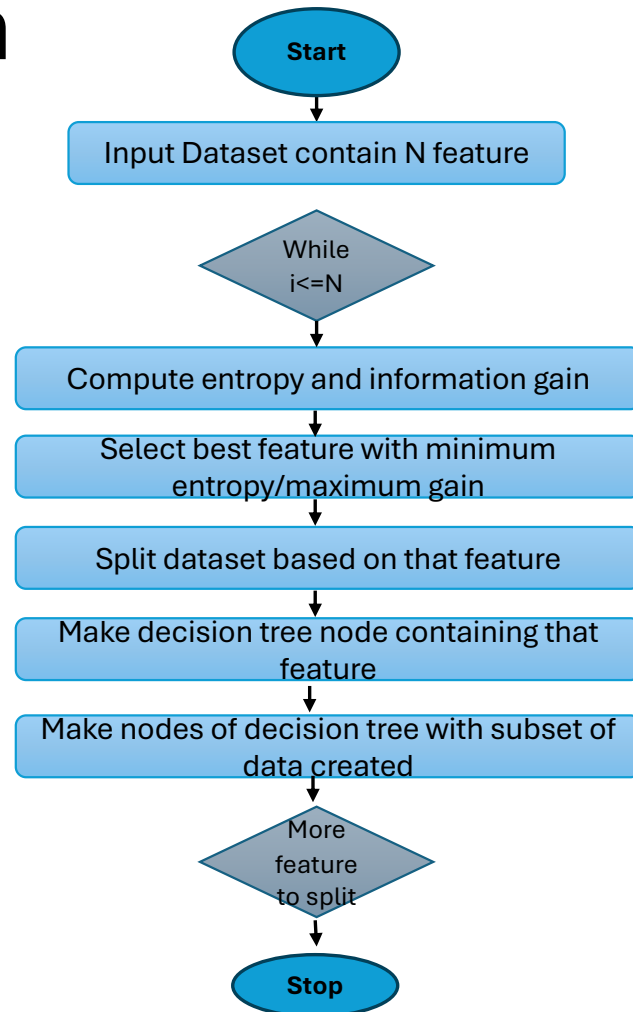
- **Root Node:** The **first decision point** (e.g., Credit Score).
- **Internal Nodes:** Decision points based on feature splits.
- **Leaves (Terminal Nodes):** Final classification or prediction outcome.
- The algorithm determines the **best feature** to split on using:
- **Gini Impurity** → Used in **CART (Classification and Regression Trees) Algorithm**, measures how “impure” a node is.
- **Entropy (Information Gain)** → Used in **ID3 Algorithm**, calculates reduction in uncertainty.
- **Pruning:** Removes unnecessary branches to avoid overfitting.
 - **Pre-Pruning (Stopping early):** Limits tree depth.
 - **Post-Pruning (Trimming later):** Removes nodes after training.



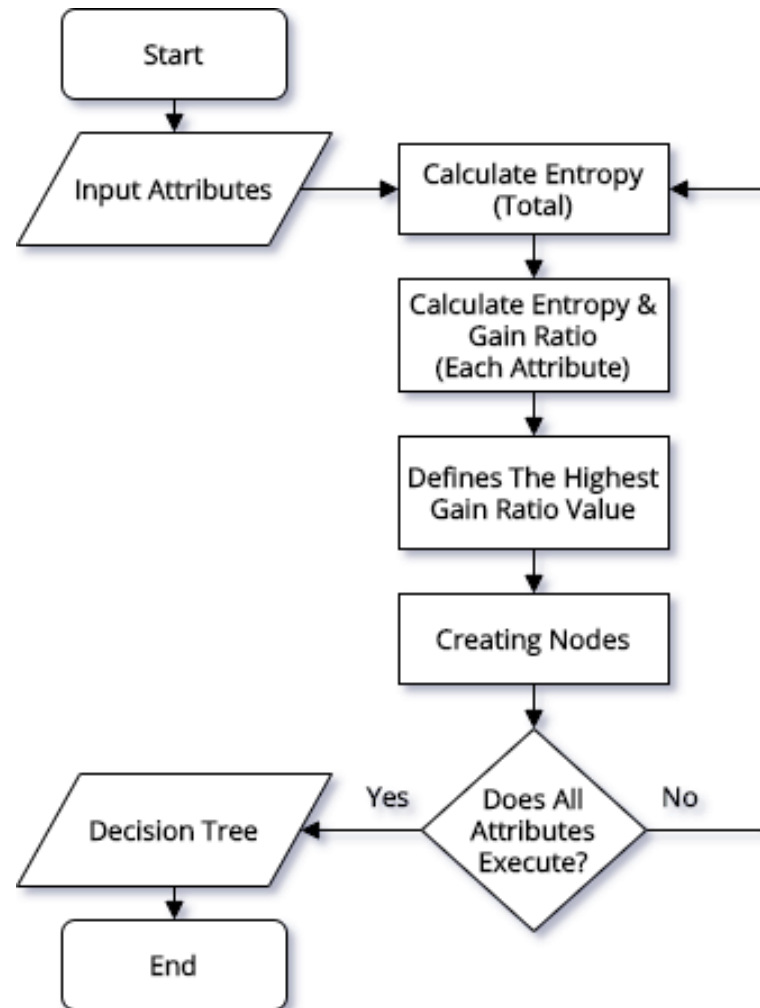
ID3 Algorithm

- ID3 builds trees using **information gain metric**.
 - Splits data based on **entropy reduction**.
 - Suitable for **categorical feature data splitting**.
 - Prone to **overfitting with noisy data**.
 - Iteratively chooses **best attribute for split**.
 - Simpler than **CART and C4.5 algorithms**.
-

ID3 Algorithm



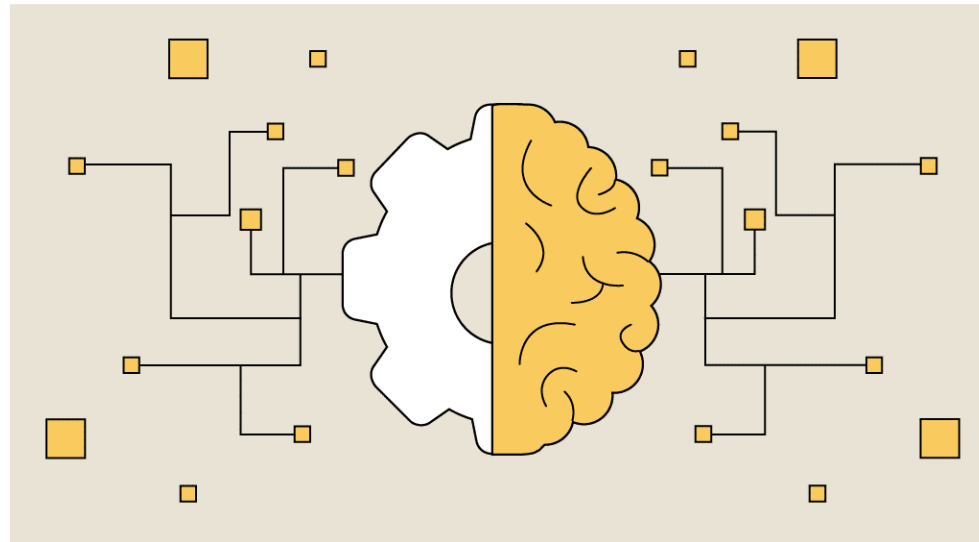
C4.5 Algorithm



Comparison of ID3 and C4.5

Feature	ID3 (Iterative Dichotomiser 3)	C4.5 (Successor of ID3)
Splitting Criterion	Uses Entropy & Information Gain to choose the best feature for splitting.	Uses Entropy & Gain Ratio , which normalizes Information Gain to prevent bias towards multi-valued attributes.
Handling Continuous Features	Cannot handle numerical (continuous) attributes; requires discretization. (e.g., "High Credit Score" vs. "Low Credit Score").	Can handle continuous (numerical) attributes by creating threshold splits. (e.g., "Credit Score $\leq$ 680").
Handling Missing Values	Cannot handle missing values.	Can handle missing values by assigning probabilities to different attribute values.
Tree Pruning	No pruning ; leads to overfitting.	Uses post-pruning to remove branches that do not improve accuracy, reducing overfitting.
Bias Towards Multi-Valued Attributes	Yes , because Information Gain favors attributes with more unique values.	No , because Gain Ratio corrects this bias.
Output	Produces a large tree with possible overfitting.	Produces a simpler tree with better generalization.
Efficiency	Faster but less accurate due to lack of pruning.	Slower than ID3 but more accurate due to pruning and handling continuous values.

Machine Learning Classifier-II



Introduction to Support Vector Machines

- A support vector machine (SVM) is a supervised machine learning model that uses classification algorithms for two-group classification problems.
- Support Vector Machines is a fast and dependable classification algorithm that performs very well with a limited amount of data.
- In machine learning, support-vector machines (SVMs), also support-vector networks are supervised learning models with associated learning algorithms that analyze data used for classification and regression analysis.

Introduction to Support Vector Machines

- SVM finds the **optimal decision boundary (hyperplane)** that maximizes the margin between two classes.
- An SVM model is a **representation of the examples as points in space**, mapped so that the examples of the separate categories are divided by a clear gap that is as wide as possible. New examples are then mapped into that same space and predicted to belong to a category based on the side of the gap on which they fall.
- SVM algorithm can be used for **face detection, image classification, text categorization, etc.**

Types of SVM

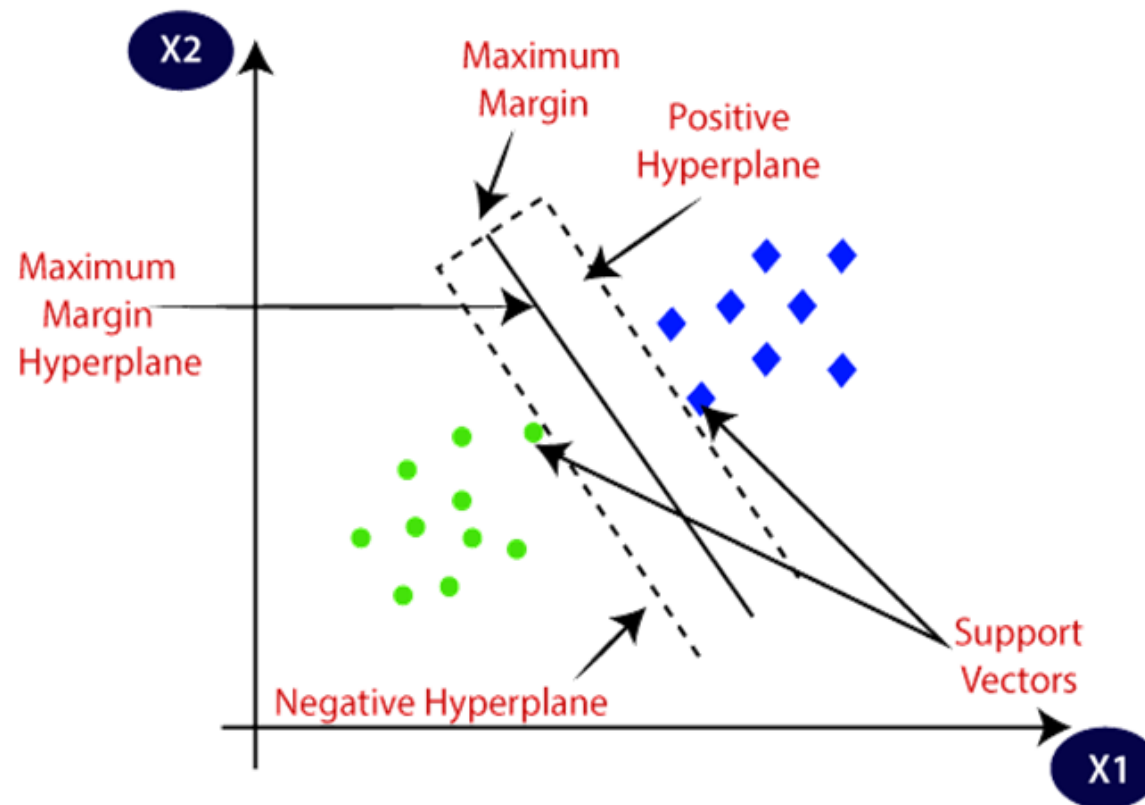
- **Linear SVM**

- Linear SVM is used for **linearly separable data**, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.

- **Non-linear SVM**

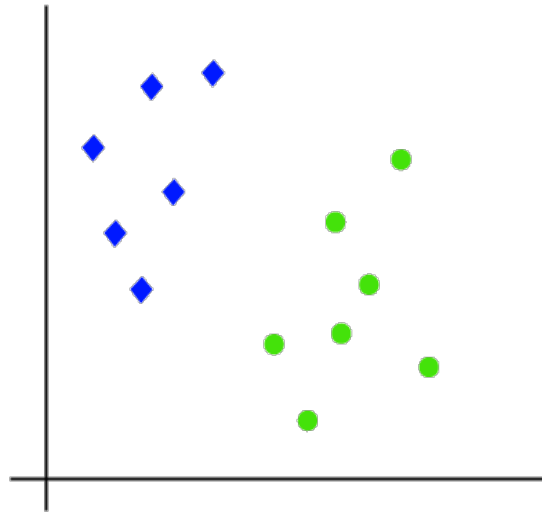
- Non-Linear SVM is used for **non-linearly separated data**, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Example



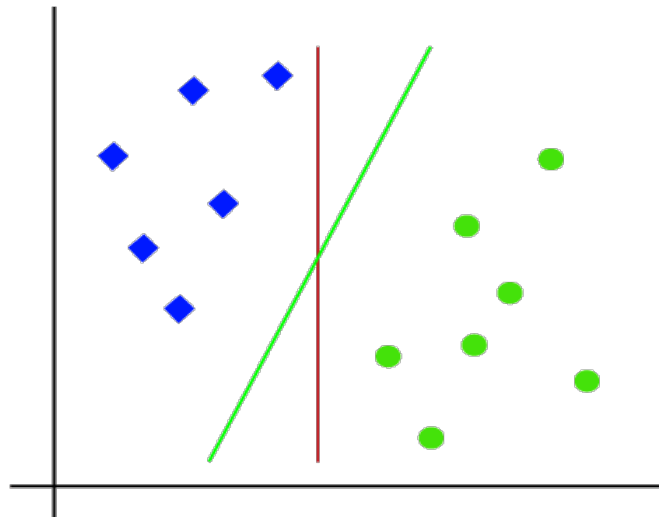
Linear SVM

- The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below image:



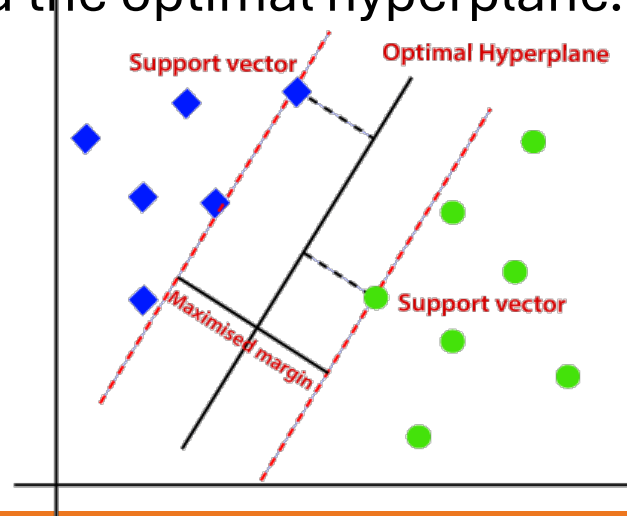
Linear SVM

- So, as it is 2-d space **so by just using a straight line**, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:



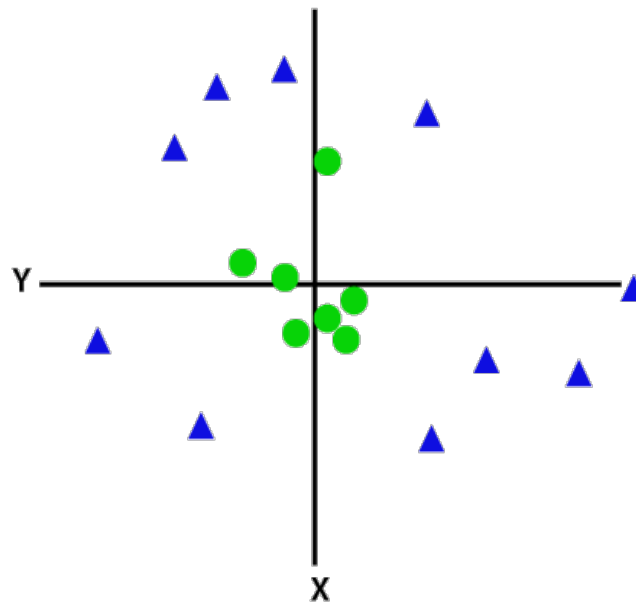
Linear SVM

- Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is **called as a hyperplane**.
- SVM algorithm finds the closest point of the lines from both the classes. These points are **called support vectors**.
- The distance between the vectors and the hyperplane is **called as margin**. And the goal of SVM is to maximize this margin. The hyperplane with maximum margin is called the optimal hyperplane.



Non-Linear SVM

- If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, **we cannot draw a single straight line**. Consider the below image:

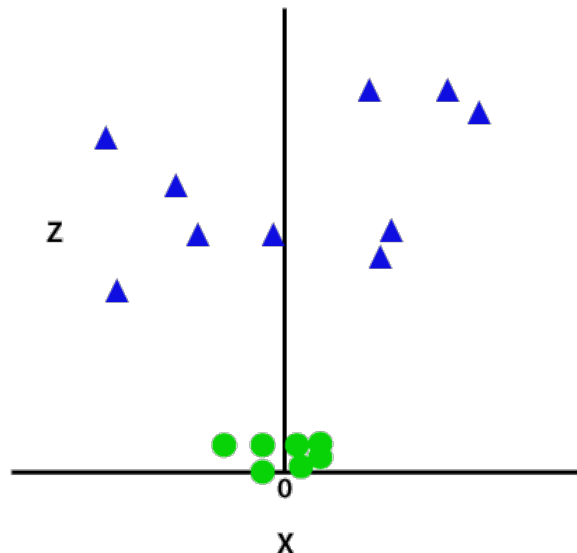


Non-Linear SVM

- So, to separate these data points, we need to add **one more dimension**. For linear data, we have used two dimensions x and y, so for non-linear data, **we will add a third-dimension z**. It can be calculated as:

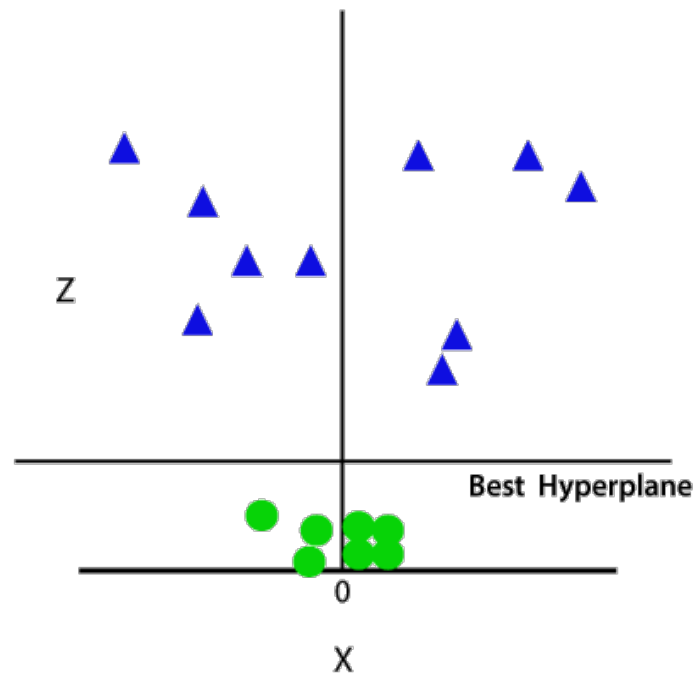
$$z = x^2 + y^2$$

- By adding the third dimension, the sample space will become as below image:



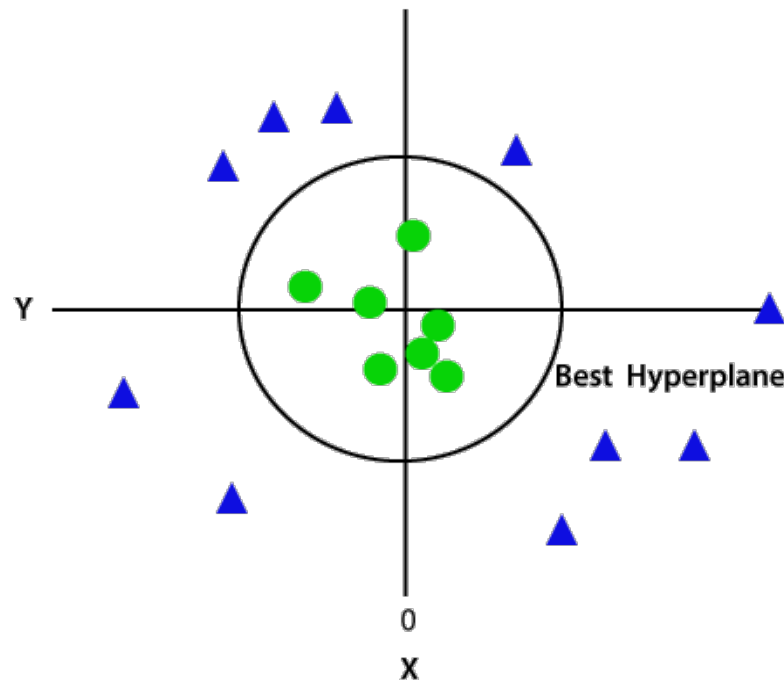
Non-Linear SVM

- So now, SVM will divide the datasets into classes in the following way. Consider the below image:



Non-Linear SVM

- Since we are in **3-D Space**, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with $z=1$, then it will become as:



SVM Formula (Linear)

- For **linearly separable data**, the equation of the hyperplane is:

$$w.x+b=0$$

where:

- w = weight vector (normal to the hyperplane),
 - x = input features,
 - b = bias term.
-
- SVM finds w and b such that the **margin is maximized**.

SVM Formula (Non-Linear)

1. **Uses Kernel Tricks** to map data into a **higher-dimensional space** where it becomes linearly separable.
2. Common **kernel functions**:
 - **Linear Kernel** $\rightarrow K(x, y) = x \cdot y$
 - **Polynomial Kernel** $\rightarrow K(x, y) = (x \cdot y + c)^d$
 - **Radial Basis Function (RBF) Kernel** $\rightarrow K(x, y) = e^{\{-\gamma ||x - y||^2\}}$
(Most commonly used)

K-Nearest Neighbour(KNN) Algorithm

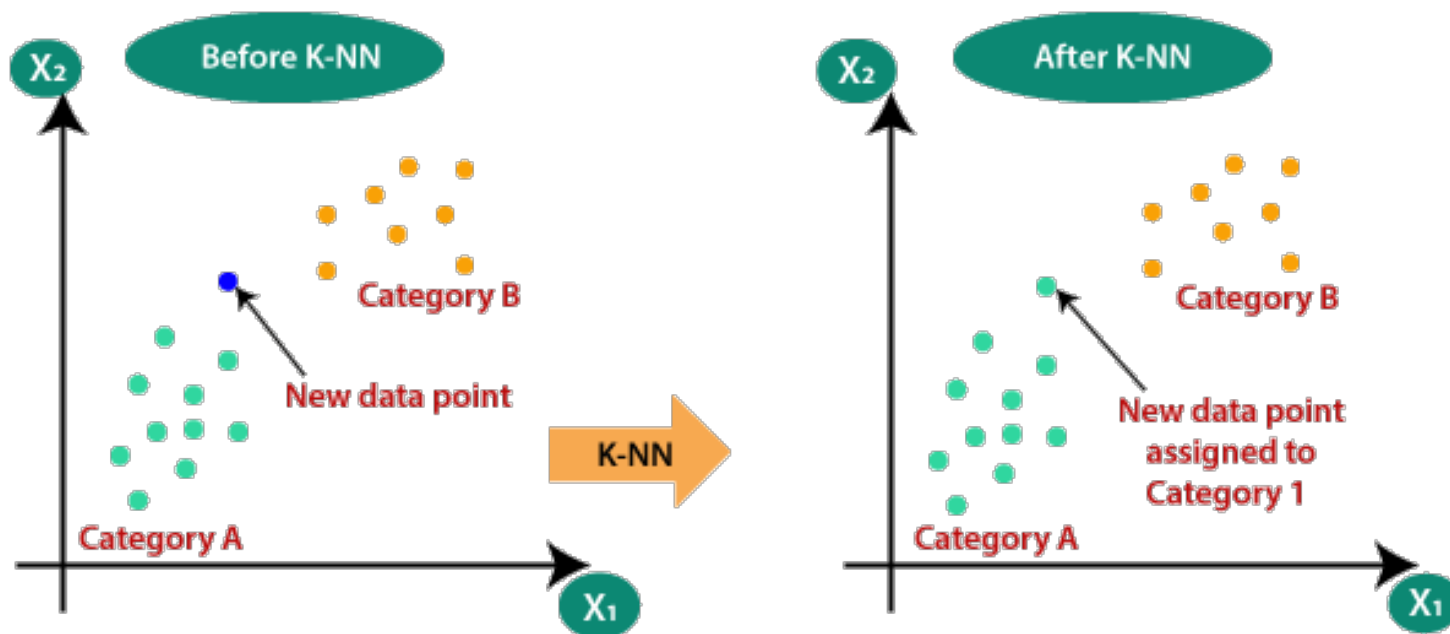
- K-NN algorithm assumes the **similarity between the new case/data and available cases** and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and **classifies a new data point based on the similarity**. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for **Regression as well as for Classification** but mostly it is used for the **Classification problems**.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.

KNN Algorithm: Example

- **Example:** Suppose, we have an image of a creature that **looks similar to cat and dog**, but we want to know either it is a cat or dog. So, for this identification, we can use the KNN algorithm, as it works on a similarity measure.
- Our KNN model will find the **similar features of the new data set** to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.



KNN Algorithm: Example



KNN Algorithm: Algorithm

- **Step-1:** Select the **number K of the neighbours**
- **Step-2:** Calculate the **Euclidean distance** of K number of neighbours
- **Step-3:** Take the K nearest neighbours as per the **calculated Euclidean distance**.
- **Step-4:** Among these k neighbours, **count the number of the data points in each category**.
- **Step-5:** Assign the new data points to that category for **which the number of the neighbour is maximum**.
- **Step-6:** Our model is ready.

How to Choose the Value of K in the K-NN Algorithm

- There is **no particular way** of choosing the value K, but here are some common conventions to keep in mind:
 - Choosing a **very low value** will most likely lead to inaccurate predictions.
 - The commonly used **value of K is 5**.
 - Always use an **odd number as the value of K**.

KNN Algorithm: Advantages and Disadvantages

Advantages:

- It is **simple to implement**.
- It is **robust to the noisy training data**.
- It can be more effective **if the training data is large**.

Disadvantages:

- Always needs to **determine the value of K** which may be complex some time.
- The **computation cost is high** because of calculating the distance between the data points for all the training samples.

KNN Algorithm: Working example

BRIGHTNESS	SATURATION	CLASS
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue

KNN Algorithm: Working example

BRIGHTNESS	SATURATION	CLASS
20	35	?

We have a new entry but it doesn't have a class yet. To know its class, we have to calculate the distance from the new entry to other entries in the data set using the Euclidean distance formula.

Here's the formula: $\sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$

Where:

- X_2 = New entry's brightness (20).
- X_1 = Existing entry's brightness.
- Y_2 = New entry's saturation (35).
- Y_1 = Existing entry's saturation.

KNN Algorithm: Working example

Distance #1

For the first row, d1:

BRIGHTNESS	SATURATION	CLASS
40	20	Red

$$\begin{aligned}d1 &= \sqrt{(20 - 40)^2 + (35 - 20)^2} \\&= \sqrt{400 + 225} \\&= \sqrt{625} \\&= 25\end{aligned}$$

We now know the distance from the new data entry to the first entry in the table. Let's update the table.

KNN Algorithm: Working example

Distance #2

For the second row, d2:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
50	50	Blue	?

$$\begin{aligned}d2 &= \sqrt{(20 - 50)^2 + (35 - 50)^2} \\&= \sqrt{900 + 225} \\&= \sqrt{1125} \\&= 33.54\end{aligned}$$

KNN Algorithm: Working example

Here's what the table will look like after all the distances have been calculated:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
40	20	Red	25
50	50	Blue	33.54
60	90	Blue	68.01
10	25	Red	10
70	70	Blue	61.03
60	10	Red	47.17
25	80	Blue	45

KNN Algorithm: Working example

Let's rearrange the distances in ascending order:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
10	25	Red	10
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17
70	70	Blue	61.03
60	90	Blue	68.01

KNN Algorithm: Working example

Since we chose 5 as the value of **K**, we'll only consider the first five rows.

That is:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
10	25	Red	10
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17

As you can see above, the majority class within the 5 nearest neighbors to the new entry is **Red**. Therefore, we'll classify the new entry as **Red**.

KNN Algorithm: Working example

Here's the updated table:

BRIGHTNESS	SATURATION	CLASS
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue
20	35	Red



Naïve Bayes Classifier

- Based on Bayes' Theorem
 - Assumes features are independent
 - Effective for text classification tasks
 - Requires small training data size
 - Handles multi-class classification well
 - Simple and computationally efficient
-

Naïve Bayes Classifier

- Naïve Bayes is based on **Bayes' Theorem**, which describes the probability of an event given prior knowledge:

$$P(A|B) = \frac{\{P(B|A) \cdot P(A)\}}{\{P(B)\}}$$

Where:

- $P(A|B)$ = Probability of event A (class) given B (features) → **Posterior Probability**.
- $P(B|A)$ = Probability of B (features) given A (class) → **Likelihood**.
- $P(A)$ = Prior probability of class A.
- $P(B)$ = Prior probability of feature B.



Naïve Bayes Classifier

- The algorithm **assumes that all features are independent (Naïve Assumption)**.
- In real-world data, features are often **correlated**, but **Naïve Bayes still works well**.
- **Example:**
 - To predict whether a **loan applicant will default**, features like **Credit Score, Income, Loan Amount** are used.
 - **Naïve Bayes assumes** these features are **independent**, but in reality, they may be correlated.

Types of Naïve Bayes Classifier

1. Gaussian Naïve Bayes (GNB)

- Assumes that numerical features follow a **normal (Gaussian) distribution**.
- Used for **continuous data** (e.g., age, income, temperature).

2. Multinomial Naïve Bayes (MNB)

- Used for **discrete data** (e.g., word counts in text classification).
- Commonly used for **spam detection, sentiment analysis**.

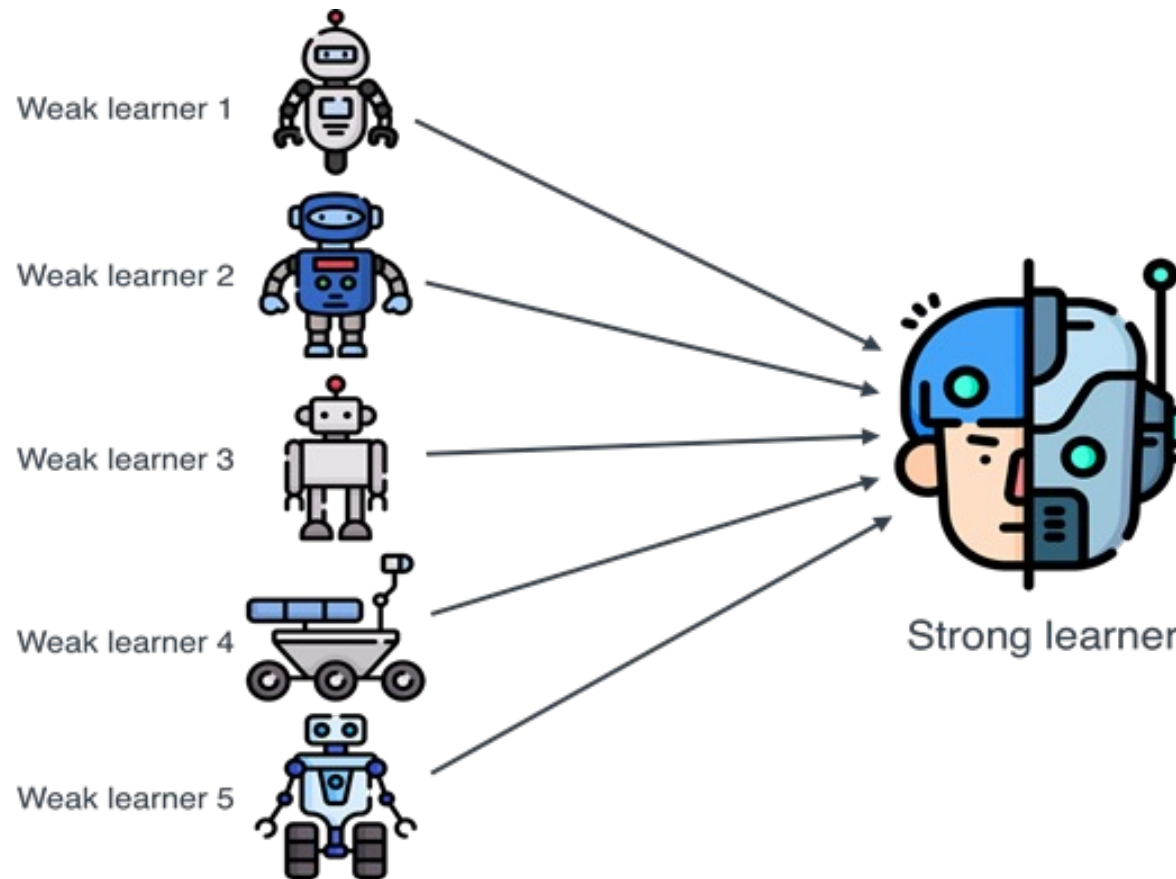
3. Bernoulli Naïve Bayes (BNB)

- Used for **binary features** (e.g., presence or absence of words in text).
 - Works well for **document classification**.
-

Introduction to Ensemble Learning

- Combines predictions from **multiple models**
 - Improves **accuracy** and model **robustness**
 - **Reduces overfitting** in supervised tasks
 - Used in **classification and regression**
 - **Voting, bagging, and boosting** methods
 - Ensemble **outperforms individual models**
-

Introduction to Ensemble Learning

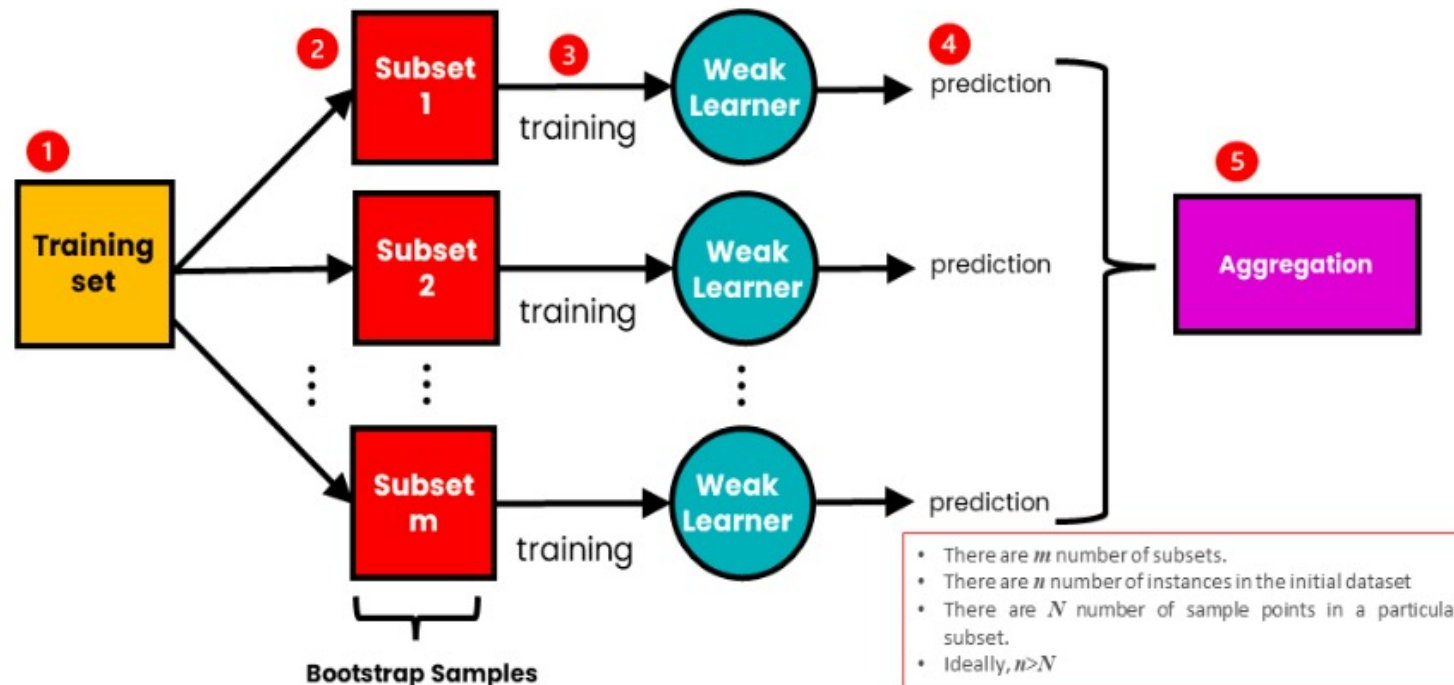


Bagging: Bootstrap Aggregating

- Combines models trained on bootstraps
 - Reduces variance and improves accuracy
 - Works well with high-variance models
 - Uses random subsets of training data
 - Parallel computation for faster training
 - Bootstrap=Random sampling with replacement
 - Example: Random Forest algorithm
-

Bagging: Bootstrap Aggregating

The Process of Bagging (Bootstrap Aggregation)

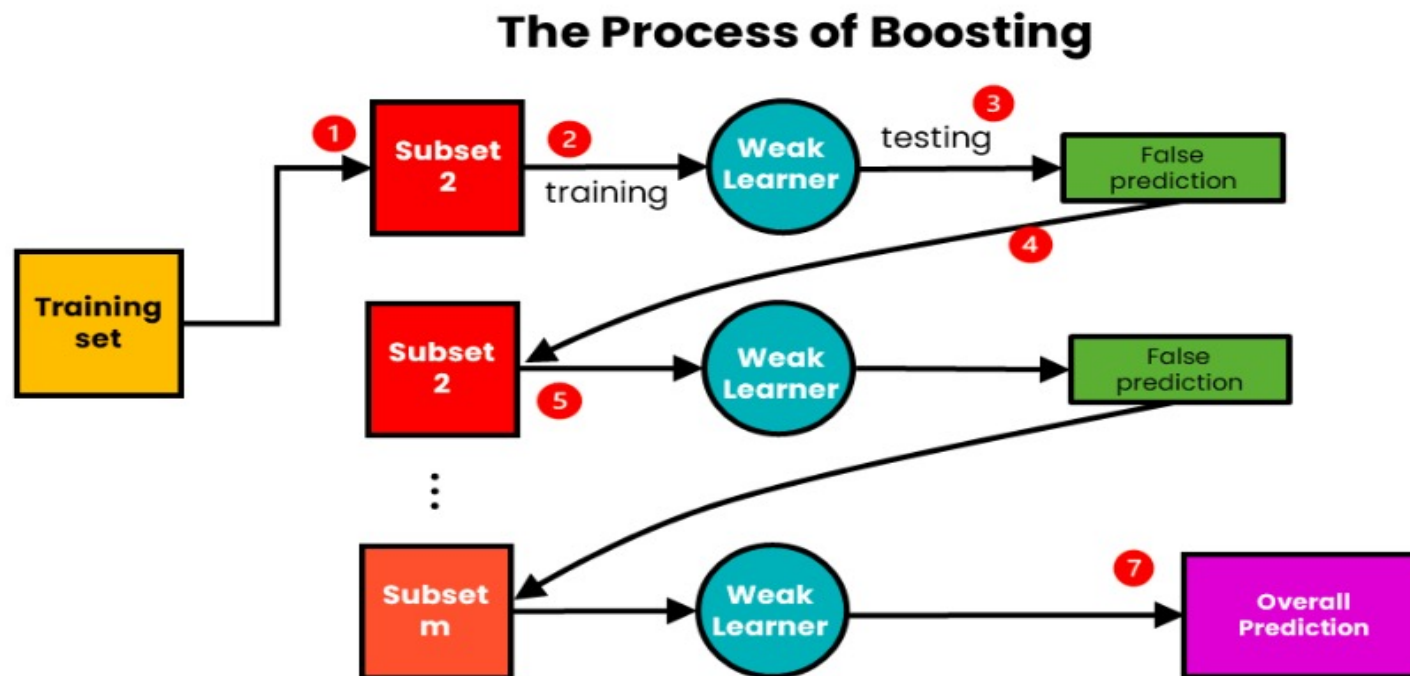




Boosting: Sequential Improvement

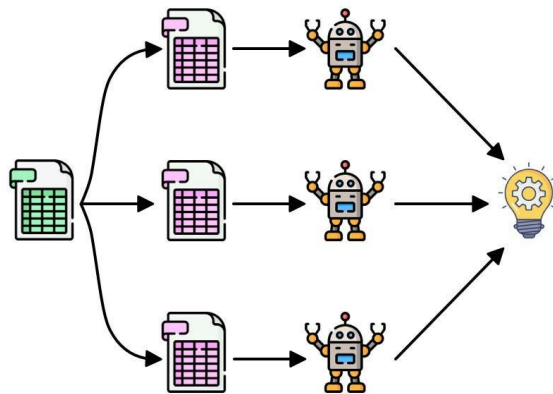
- Builds models sequentially to reduce error
 - Focuses on misclassified training samples
 - Reduces bias and variance effectively
 - Uses weak learners to form strong model
 - Sensitive to noisy data and overfitting
 - Examples: AdaBoost, Gradient Boosting
-

Boosting: Sequential Improvement



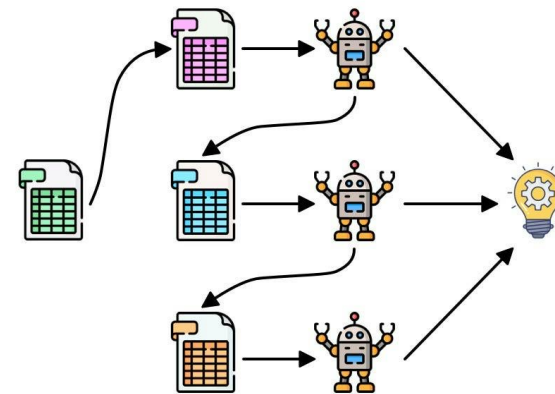
Comparison Bagging Vs Boosting

Bagging



Parallel

Boosting



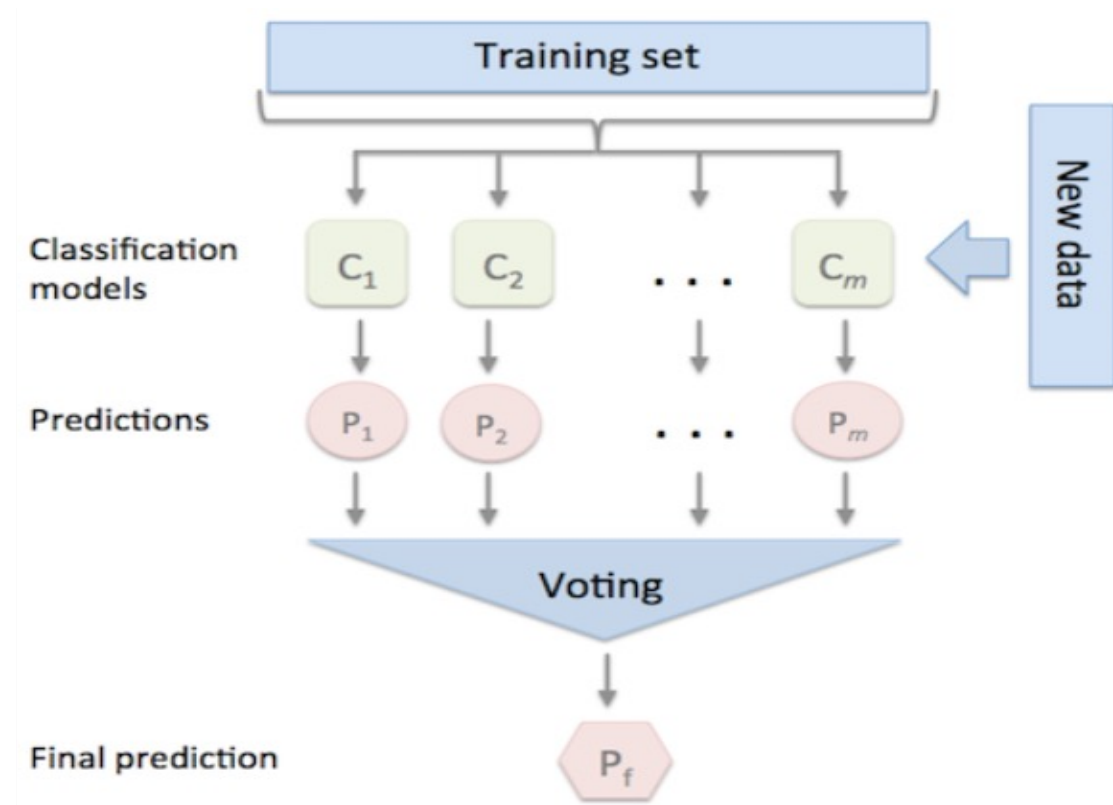
Sequential



Majority Voting Classifier

- Combines multiple classifiers' predictions
 - Class with majority votes is chosen
 - Works with bagging and boosting methods
 - Effective for diverse model combinations
 - Simple and interpretable ensemble approach
 - Requires diverse base classifiers
-

Majority Voting Classifier



Majority Voting Classifier: Types

1. Hard Voting (Majority Vote)

- Each model votes for a class.
- The class with the most votes is the final prediction.

✓ Example:

- If models predict **[Yes, No, Yes]**, the majority vote is **Yes**.

✓ Use Case: When all models **predict discrete class labels**.

Majority Voting Classifier: Types

2. Soft Voting (Weighted Probability Vote)

- Each model gives a **probability score** for each class.
- The class with the **highest average probability** is selected.

✓ Example:

If models give the following probabilities for “Default” (Yes):

- **Logistic Regression** → 80%
- **Decision Tree** → 40%
- **SVM** → 70%

Average = 0.63

Since **63% > 50%**, the model predicts “Yes” (Default).

✓ **Use Case:** When classifiers output **probabilities** instead of direct labels.

Majority Voting Classifier: Code

```
from sklearn.ensemble import VotingClassifier
```

```
logistic_clf = Pipeline([('preprocessor', preprocessor), ('classifier',  
LogisticRegression())])
```

```
tree_clf = Pipeline([('preprocessor', preprocessor), ('classifier',  
DecisionTreeClassifier(max_depth=5))])
```

```
svm_clf = Pipeline([('preprocessor', preprocessor), ('classifier',  
SVC(kernel='linear', probability=True))])
```

```
voting_clf = VotingClassifier(estimators=[  
('logistic', logistic_clf),  
('tree', tree_clf),  
('svm', svm_clf)], voting='hard')
```

Bias-Variance Tradeoff

1 What is Bias?

- **Bias** is the error introduced by making **simplifying assumptions** in the model.
- **High Bias:** The model **underfits** the data, meaning it is too simple to capture patterns.
- **Low Bias:** The model **fits the data well**.

✓ Example (Loan Default Prediction)

- A **Linear Regression model** predicting **Loan Default (Yes/No)** based only on **Income** (ignoring Credit Score, Loan Amount, etc.).
 - This model **oversimplifies** the problem and has **high bias**.
 - It may **always predict No Default** if it sees a high income.
 - ◆ **Result:** Poor performance on both training and test data.
-

Bias-Variance Tradeoff

2 What is Variance?

- **Variance** is the error introduced when a model is **too sensitive to the training data**.
- **High Variance:** The model **overfits**, capturing noise in the data.
- **Low Variance:** The model **generalizes well** to unseen data.

✓ Example (Loan Default Prediction)

- A **Decision Tree with max depth = 20** trained on the loan dataset.
- It learns **every minor pattern**, including noise.
- It classifies **almost all training samples correctly** but fails on new data.
- ◆ **Result:** High accuracy on training data but poor performance on test data.

Bias-Variance Tradeoff

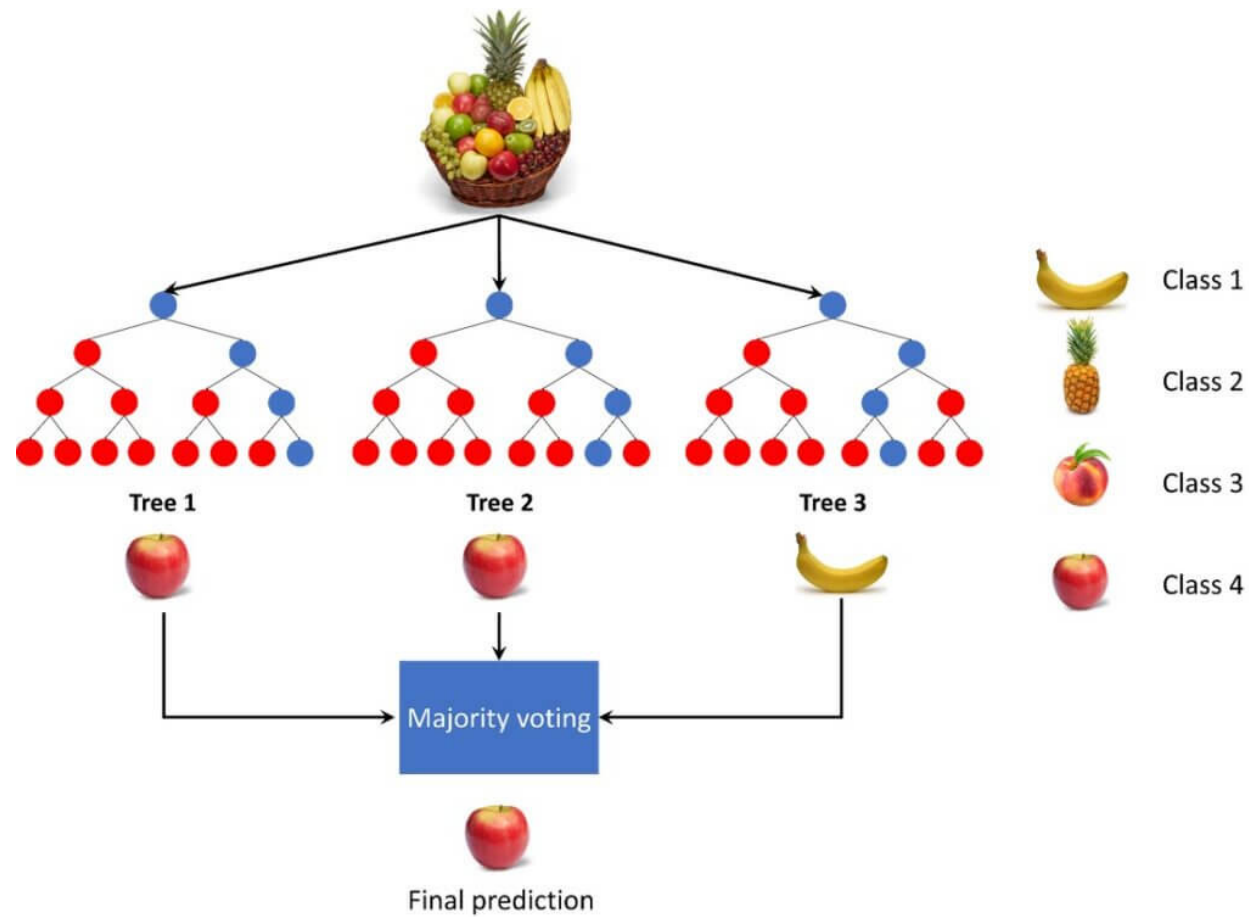
- **High Bias + Low Variance** → **Underfitting** (e.g., Linear Regression, Naïve Bayes).
 - **Low Bias + High Variance** → **Overfitting** (e.g., Deep Decision Trees, KNN with small k).
 - **Balanced Bias-Variance** → **Best Model** (e.g., Random Forest, XGBoost with regularization).
- ◆ **Goal:** Find a balance between **bias and variance to minimize total error**.

Model	Bias	Variance	Best Use Case
Linear Regression	High	Low	If relationships are simple
Decision Tree (max_depth=20)	Low	High	Small dataset, low noise
Random Forest	Medium	Medium	Large dataset, avoids overfitting
XGBoost	Low	Controlled	Best for tabular data with many features

Random Forest Classifier

- Extension of bagging with decision trees
 - Uses random subsets of features, samples
 - Reduces overfitting and variance issues
 - Effective for classification and regression
 - Provides feature importance analysis
 - Handles large datasets with missing values
-

Random Forest Classifier



Random Forest Classifier

1. **Creates multiple Decision Trees** using different random subsets of data.
2. **Each tree votes** on the class label.
3. The final prediction is made based on the **majority vote (classification)** or **average prediction (regression)**.

✓ Why is it called “Random”?

- **Random Samples** → Each tree is trained on a **random subset** of the data.
 - **Random Features** → Each tree **splits on different random features**, improving diversity.
-

Random Forest Classifier

✓ Advantages

- ✓ Handles large datasets well.
- ✓ Resistant to overfitting (unlike individual Decision Trees).
- ✓ Works well with missing values and outliers.
- ✓ Feature importance ranking (helps in feature selection).

✗ Disadvantages

- ✗ Computationally expensive for very large datasets.
 - ✗ Less interpretable than a single decision tree.
 - ✗ Not great for high-dimensional sparse data (e.g., text classification).
-



Gradient Boosting Machines (GBM)

- Boosting method **optimizing weak learners**
 - Reduces loss using **gradient descent**
 - Sequentially **minimizes prediction errors**
 - Each new model **corrects the errors of the previous one**
 - High accuracy but **computationally expensive**
 - Sensitive to **hyperparameter tuning**
-

Gradient Boosting Machines: Steps

1. Train a **weak learner** (usually a **Decision Tree**).
2. Compute the **errors (residuals)**.
3. Train the next tree to **correct the errors**.
4. Repeat the process **several times**, combining weak models into a strong model.

✓ **Boosting focuses on “hard-to-classify” points** by adjusting weights to give more importance to misclassified points.



Gradient Boosting Machines



Advantages

- ✓ **High accuracy** on structured data.
- ✓ **Works well with small to medium datasets.**
- ✓ **Can handle missing values internally.**
- ✓ **Supports classification & regression tasks.**



Disadvantages

- ✗ **Slow training** (compared to Random Forest).
- ✗ **Sensitive to hyperparameters** (learning rate, depth).
- ✗ **Can overfit if not properly tuned.**

XGBoost (Extreme Gradient Boosting)

- Optimized implementation of Gradient Boosting
 - Regularization reduces overfitting significantly
 - Efficient memory and parallel computation
 - Handles missing data automatically
 - Widely used for classification, regression, and ranking tasks and in Kaggle competitions
 - Excellent for structured/tabular data
-



Comparison of XGBoost and GBM

Feature	XGBoost	GBM (Traditional)
Speed	Faster (parallel computing)	Slower
Regularization	L1 & L2 (better generalization)	No built-in
Tree Pruning	Stops unnecessary splits (reduces overfitting)	Splits until max depth
Missing Values Handling	Handles automatically	Requires imputation



XGBoost Algorithm: Steps

- Step 1: Initialize Model
 - Step 2: Compute Residuals
 - Step 3: Train New Trees to Correct Errors
 - Step 4: Update Predictions
 - Step 5: Repeat Until Convergence
-



XGBoost Algorithm: Mathematical Formula

- XGBoost optimizes a loss function using **gradient descent**:

- $F_{m(x)} = F_{\{m-1\}(x)} + \gamma \cdot h_m(x)$

where:

- $F_{m(x)}$ = updated model, $F_{\{m-1\}(x)}$ = previous model.
- γ = learning rate.
- $h_{m(x)}$ = new weak learner trained to minimize error.

 **Regularization (L1 & L2) prevents overfitting:**

- $Loss = \sum (y_i - \widehat{y_i})^2 + \lambda ||w||^2 + \alpha ||w||$

where:

- λ = L2 regularization (Ridge regression).
- α = L1 regularization (Lasso regression).

XGBoost Algorithm

✓ Advantages

- ✓ **High accuracy** compared to Random Forest & GBM.
- ✓ **Faster than traditional GBM** (parallel computing).
- ✓ **Handles missing values automatically.**
- ✓ **Prevents overfitting using L1 & L2 regularization.**

✗ Disadvantages

- ✗ **Sensitive to hyperparameters** (tuning required).
 - ✗ **Computationally expensive for large datasets.**
 - ✗ **Not always the best choice for small datasets.**
-



Evaluation Metrics: Confusion Matrix

		ACTUAL	
		Positive	Negative
PREDICTED	Positive	True Positive (TP) 5	False Positive (FP) 10
	Negative	False Negative (FN) 15	True Negative (TN) 70

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = 0.75$$

$$\text{Recall} = \frac{TP}{TP + FN} = 0.25$$

$$\text{Precision} = \frac{TP}{TP + FP} = 0.33$$

$$\text{F1 Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = 0.28$$



Regression

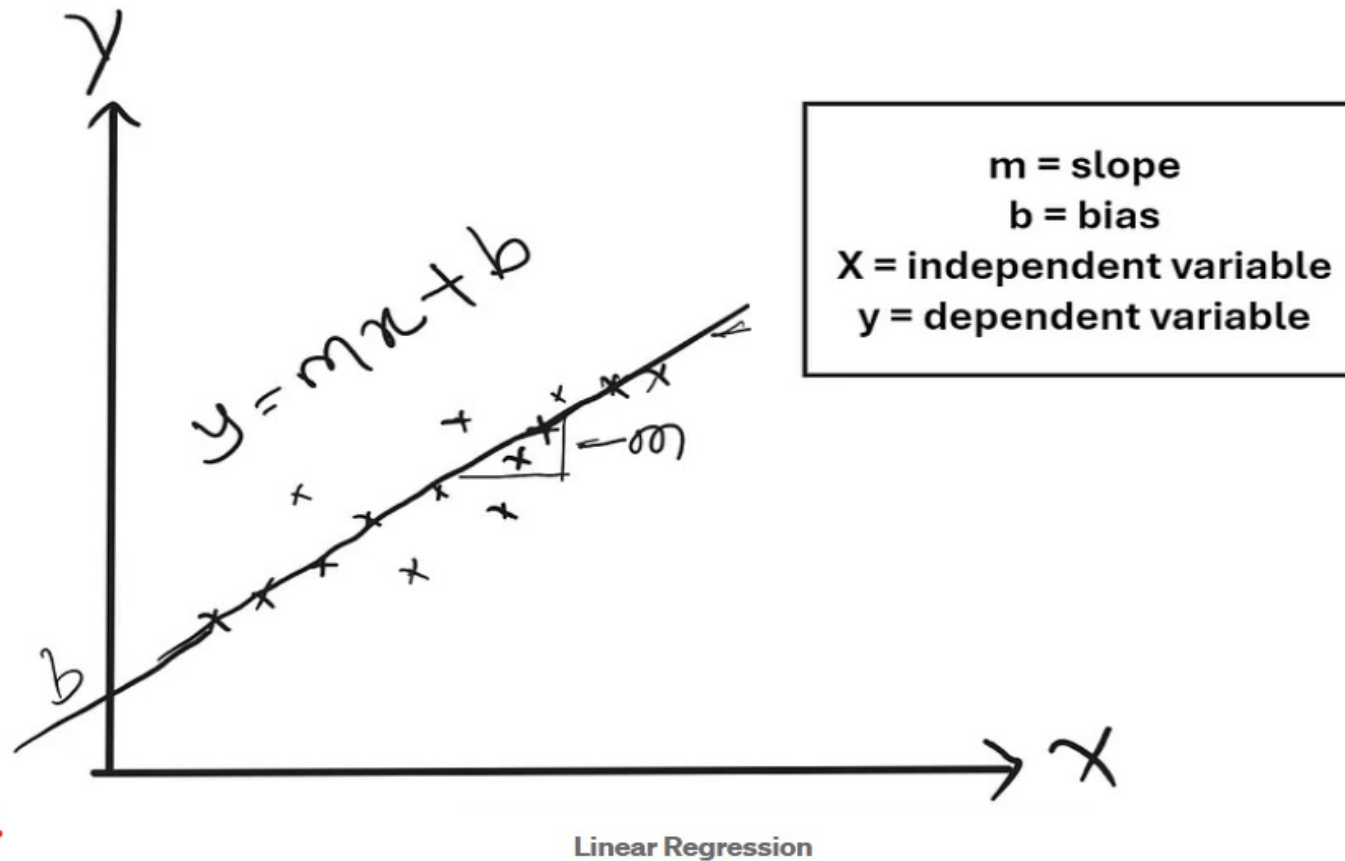


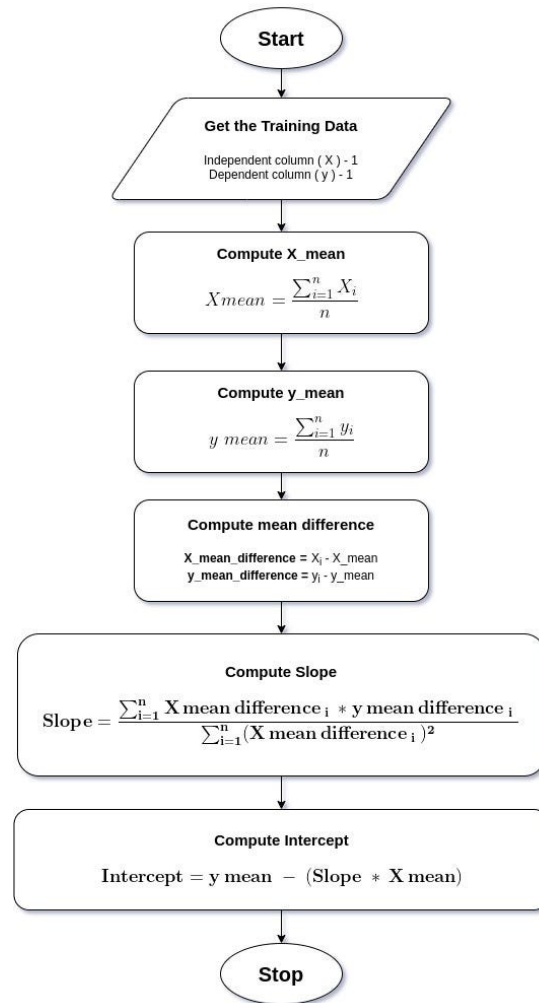


Introduction to Linear Regression

- **Linear regression** models relationships using data.
 - **Dependent variable** predicted via features.
 - Model assumes **linear relationship exists**.
 - Formula: **$y=mx+b$**
 - **Simple** and **multiple regressions** differ.
 - **Applications:** prediction, trend analysis, forecasting.
-

Introduction to Linear Regression



SIMPLE LINEAR REGRESSION

Prediction: Regression Example

- **Problem Statement**

- Last year, five randomly selected students took a math aptitude test before they began their statistics course. The Statistics Department has three questions.
 - What **linear regression equation** best predicts statistics performance, based on math aptitude scores?
 - If a student made an 80 on the aptitude test, what grade would we expect to make in statistics?
 - How well does the **regression equation fit the data**?

How to Find the Regression Equation

- In the table below, the x_i column shows scores on the aptitude test. Similarly, the y_i column shows statistics grades.
- The last two columns show deviations scores - the difference between the student's score and the average score on each test.
- The last two rows show sums and mean scores that we will use to conduct the regression analysis.

Student	x_i	y_i	$(x_i - \bar{x})$	$(y_i - \bar{y})$
1	95	85	17	8
2	85	95	7	18
3	80	70	2	-7
4	70	65	-8	-12
5	60	70	-18	-7
Sum	390	385		
Mean	78	77		

How to Find the Regression Equation

- And for each student, we also need to compute the squares of the deviation scores (the last two columns in the table below).

Student	x_i	y_i	$(x_i - \bar{x})^2$	$(y_i - \bar{y})^2$
1	95	85	289	64
2	85	95	49	324
3	80	70	4	49
4	70	65	64	144
5	60	70	324	49
Sum	390	385	730	630
Mean	78	77		

How to Find the Regression Equation

- And finally, for each student, we need to compute the product of the deviation scores.

Student	x_i	y_i	$(x_i - \bar{x})(y_i - \bar{y})$
1	95	85	136
2	85	95	126
3	80	70	-14
4	70	65	96
5	60	70	126
Sum	390	385	470
Mean	78	77	

How to Find the Regression Equation

- The regression equation is a linear equation of the form: $\hat{y} = b_0 + b_1x$. To conduct a regression analysis, we need to solve for b_0 and b_1 . Computations are shown below. First, we solve for the regression coefficient (b_1):

$$b_1 = \Sigma [(x_i - \bar{x})(y_i - \bar{y})] / \Sigma [(x_i - \bar{x})^2]$$

$$b_1 = 470/730$$

$$b_1 = 0.644$$

- Once we know the value of the regression coefficient (b_1), we can solve for the regression slope (b_0):

$$b_0 = \bar{y} - b_1 * \bar{x}$$

$$b_0 = 77 - (0.644)(78)$$

$$b_0 = 26.768$$

- Therefore, the regression equation is: $\hat{y} = 26.768 + 0.644x$.



How to Use the Regression Equation

- Once you have the regression equation, using it is a snap. Choose a value for the independent variable (x), perform the computation, and you have an estimated value ($\hat{y}$) for the dependent variable.
- In our example, the independent variable is the student's score on the aptitude test. The dependent variable is the student's statistics grade. If a student made an 80 on the aptitude test, the estimated statistics grade ($\hat{y}$) would be:
 - $\hat{y} = b_0 + b_1x$
 - $\hat{y} = 26.768 + 0.644x = 26.768 + 0.644 * 80$
 - $\hat{y} = 26.768 + 51.52 = 78.288$

How to Use the Regression Equation

- Whenever you use a regression equation, you should ask how well the equation fits the data. One way to assess fit is to check the **coefficient of determination**, which can be computed from the following formula.
- $$R^2 = \{ (1 / N) * \Sigma [(x_i - \bar{x}) * (y_i - \bar{y})] / (\sigma_x * \sigma_y) \}^2$$
- where N is the number of observations used to fit the model, Σ is the summation symbol, x_i is the x value for observation i, $\bar{x}$ is the mean x value, y_i is the y value for observation i, $\bar{y}$ is the mean y value, σ_x is the standard deviation of x, and σ_y is the standard deviation of y.

How to Use the Regression Equation

- Computations for the sample problem of this lesson are shown below. We begin by computing the standard deviation of x (σ_x):
- $\sigma_x = \text{sqrt} [\Sigma (x_i - \bar{x})^2 / N]$
- $\sigma_x = \text{sqrt}(730/5) = \text{sqrt}(146) = 12.083$
- Next, we find the standard deviation of y , (σ_y):
- $\sigma_y = \text{sqrt} [\Sigma (y_i - \bar{y})^2 / N]$
- $\sigma_y = \text{sqrt}(630/5) = \text{sqrt}(126) = 11.225$
- And finally, we compute the coefficient of determination (R^2):
- $R^2 = \{ (1 / N) * \Sigma [(x_i - \bar{x}) * (y_i - \bar{y})] / (\sigma_x * \sigma_y) \}^2$
- $R^2 = [(1/5) * 470 / (12.083 * 11.225)]^2$
- $R^2 = (94 / 135.632)^2 = (0.693)^2 = 0.48$

How to Use the Regression Equation

- A coefficient of determination equal to 0.48 indicates that about 48% of the variation in statistics grades (the dependent variable) can be explained by the relationship to math aptitude scores (the independent variable).
- This would be considered a good fit to the data, in the sense that it would substantially improve an educator's ability to predict student performance in statistics class.

Types of Regression

- Linear Regression (**Linear Relationship**)
- Logistic Regression (**Target value either 0 or 1**)
- Ridge Regression (**High correlation b/w Independent variables**)
- Lasso Regression (**Feature selection**)
- Polynomial Regression (**Relationship denoted by N^{th} degree**)
- Bayesian Linear Regression (**Bayes Theorem**)

RANSAC Regression

- RANSAC fits models using outlier exclusion.
 - Stands for Random Sample Consensus Algorithm.
 - Iteratively selects data subsets for fitting.
 - Handles noisy data and extreme values.
 - Builds robust models against outliers.
 - Widely used in computer vision, image processing, and machine learning for tasks like linear regression.
-

RANSAC Regression

- Traditional estimation techniques like Linear Regression assume that **all data points follow a model**. However, in real-world datasets, **outliers can significantly distort the model**.

RANSAC is useful when:

- Your dataset has a **lot of noise or outliers**.
 - You need a **robust estimation of parameters**.
 - A subset of your data **follows a clear pattern**.
-

Steps in RANSAC Algorithm

1. **Randomly select a small subset** of data points.
 2. **Fit a model** to this subset (e.g., a line in linear regression).
 3. **Determine inliers:**
 - Compute the error for all other data points.
 - If a point's error is below a threshold, it is **an inlier**.
 4. **Repeat multiple iterations:**
 - Randomly pick different subsets.
 - Track the model with the most inliers.
 5. **Final Model:**
 - The best model (with the highest inlier count) is chosen as the final model.
-

Steps in RANSAC Algorithm

Feature (Area in Sq. Ft)	Actual Price (₹)	Predicted Price (₹)	Error (Residual)	Inlier/Outlier?
8000	12,500,000	12,600,000	100,000	✓ Inlier
9000	14,000,000	13,500,000	500,000	✓ Inlier
15,000	50,000,000	20,000,000	30,000,000	✗ Outlier
9500	13,000,000	12,900,000	100,000	✓ Inlier

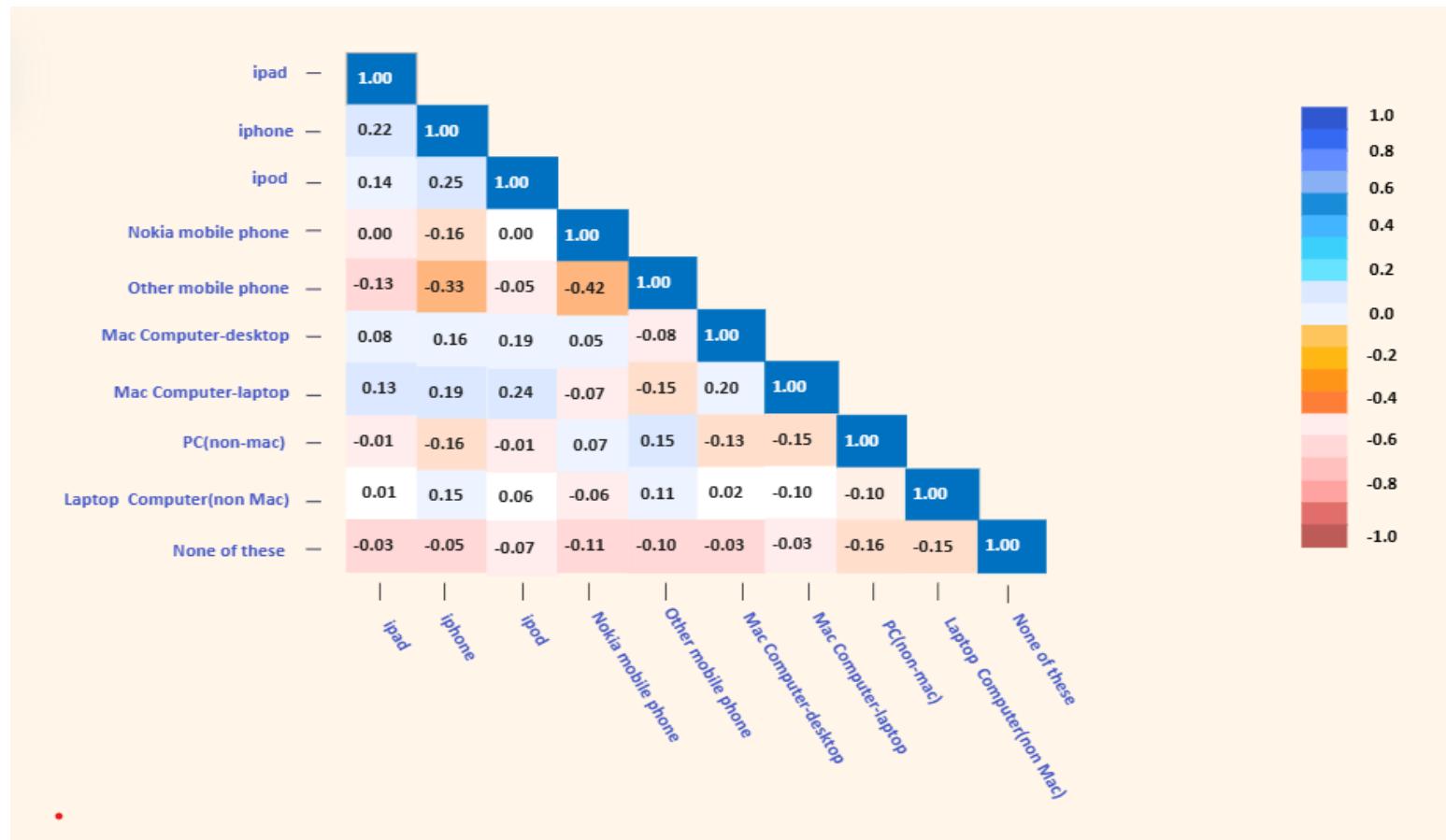
- Since **30M > 1M**, **RANSAC ignores this outlier** and does not let it distort the regression model.

Correlation Matrix Overview

- Displays variable **relationships** as coefficients.
 - Values range: **-1 to +1 correlation**.
 - **Positive values** indicate **direct relationships**.
 - **Negative values** indicate **inverse relationships**.
 - **Zero implies** **no relationship** between variables.
 - Essential for **feature selection, data analysis**.
-



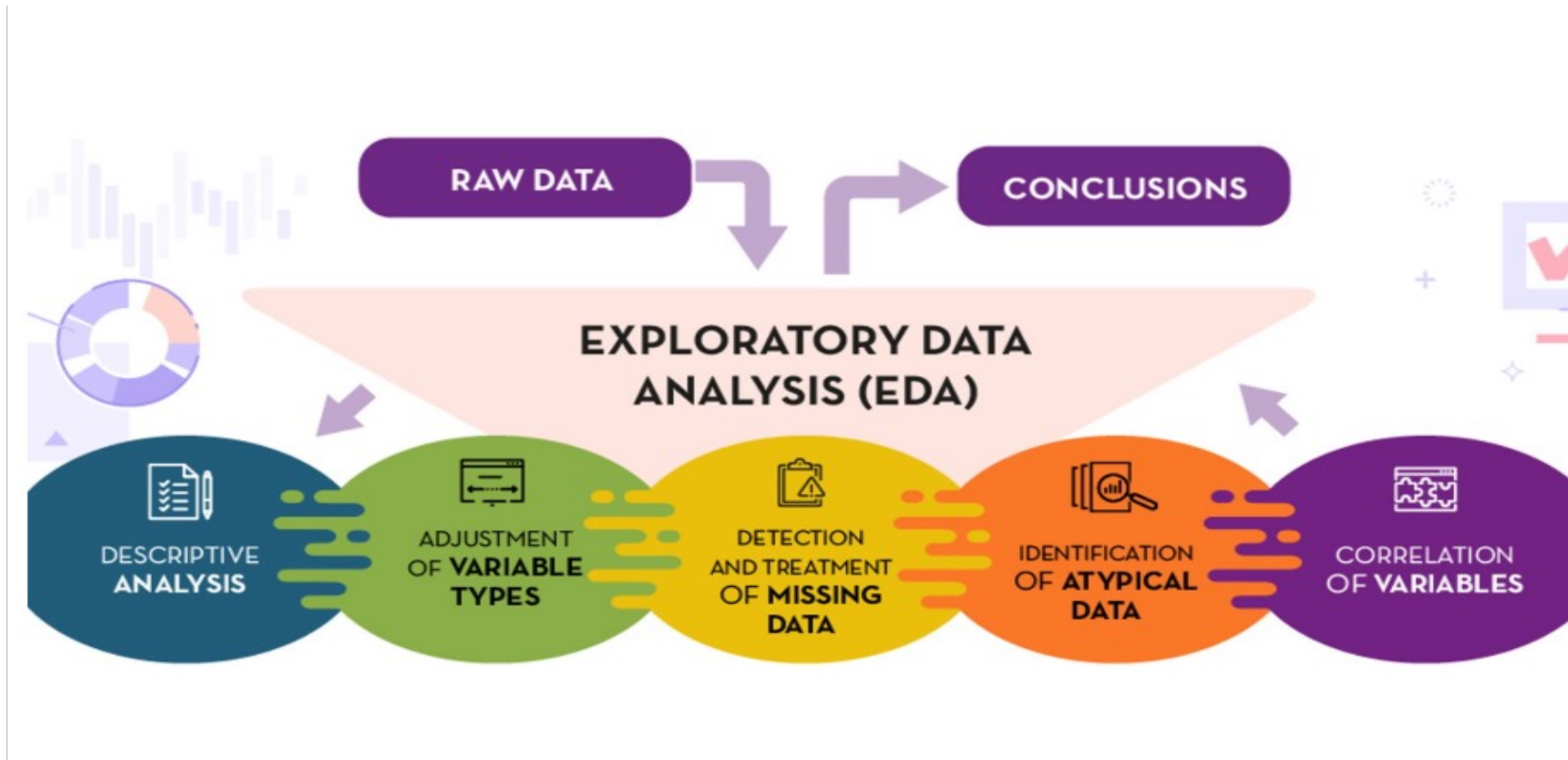
Correlation Matrix Overview



Exploratory Data Analysis (EDA) Importance

- EDA reveals **dataset trends, structures, anomalies**.
 - Visualizations uncover **relationships and distributions**.
 - Statistical summaries describe **central tendencies**.
 - **Scatter plots** and **histograms** show data behaviour.
 - **Outlier detection** identifies irregular data points.
 - Guides **model selection, feature engineering** choices.
-

Exploratory Data Analysis (EDA) Importance



Exploratory Data Analysis (EDA) Steps

- Step 1: Load and Understand the Data

```
import pandas as pd
```

```
df = pd.read_csv("house_prices.csv")
```

```
print(df.head())
```

```
print(df.info()) # Data types, non-null counts
```

```
print(df.describe()) # Summary statistics
```

Exploratory Data Analysis (EDA) Steps

- Step 2: Check for Missing Values

Count of missing values per column

```
print(df.isnull().sum())
```

Fill with median

```
df["Price"].fillna(df["Price"].median(), inplace=True)
```

Exploratory Data Analysis (EDA) Steps

- Step 3: Detect and Handle Outliers

```
import matplotlib.pyplot as plt  
plt.figure(figsize=(8,6))  
sns.boxplot(data=df, x="Price")  
plt.title("Boxplot of House Prices")  
plt.show()
```

Exploratory Data Analysis (EDA): Interquartile Range (IQR)

- Step 3: Detect and Handle Outliers

```
Q1 = df["Price"].quantile(0.25)
```

```
Q3 = df["Price"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
df = df[(df["Price"] >= (Q1 - 1.5 * IQR)) & (df["Price"] <= (Q3 +  
1.5 * IQR))]
```

Exploratory Data Analysis (EDA) Steps

Index	Price (₹)
1	10,000,000
2	11,500,000
3	12,200,000
4	13,000,000
5	15,000,000
6	50,000,000 (Outlier)



Exploratory Data Analysis (EDA): Interquartile Range (IQR)

1. $Q1 = 11,500,000$ (25% of values are below this)
 2. $Q3 = 13,000,000$ (75% of values are below this)
 3. $IQR = Q3 - Q1 = 13,000,000 - 11,500,000 = 1,500,000$
 4. $\text{Lower Bound} = Q1 - (1.5 \times IQR) = 11,500,000 - (1.5 \times 1,500,000) = 9,250,000$
 5. $\text{Upper Bound} = Q3 + (1.5 \times IQR) = 13,000,000 + (1.5 \times 1,500,000) = 15,250,000$
- **Outlier Detected:** The price **50,000,000** is above **15,250,000**, so it gets removed.

Exploratory Data Analysis (EDA) Steps

- Step 4: Visualizing Data Distributions

```
df["Price"].hist(bins=30)
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.title("Distribution of House Prices")
plt.show()
```

Exploratory Data Analysis (EDA) Histogram

- A histogram **groups continuous values into bins (equal intervals)** and **counts the number of occurrences** in each bin.

Possible Outcomes:

1. Normal Distribution (Bell Curve)

- If house prices are **symmetrically distributed**, it means **most houses are priced around a central value**.

2. Right-Skewed Distribution (Positive Skew)

- If the histogram has a **long right tail**, **most houses have lower prices** but some **very expensive** houses exist.

3. Left-Skewed Distribution (Negative Skew)

- If the histogram has a **long left tail**, it means **most houses have higher prices**, but some **very cheap** houses exist.

Exploratory Data Analysis (EDA) Steps

- Step 5: Check Correlations Between Features

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(df.corr(), annot=True, cmap="coolwarm",  
            linewidths=0.5)
```

```
plt.title("Correlation Matrix Heatmap")
```

```
plt.show()
```

Exploratory Data Analysis (EDA) Steps

- Step 6: Feature Selection & Engineering
- Not all features contribute equally; remove **redundant or weak features**.

```
correlation =  
    df.corr()["Price"].abs().sort_values(ascending=False)  
print(correlation)
```

- **High correlation** with target variable (e.g., Area → Price): Keep it!
- **Low correlation** (e.g., FurnishingStatus → Price = 0.01): Consider dropping.

Exploratory Data Analysis (EDA) Steps

- Step 7: Transforming Categorical Variables

```
label_encoders = {}
```

```
categorical_cols = ["mainroad", "guestroom", "basement",  
"hotwaterheating", "airconditioning", "prefarea",  
"furnishingstatus"]
```

```
for col in categorical_cols:
```

```
    le = LabelEncoder()
```

```
    df[col] = le.fit_transform(df[col])
```

```
    label_encoders[col] = le
```

Exploratory Data Analysis (EDA) Steps

- Step 8: Scaling & Normalization

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

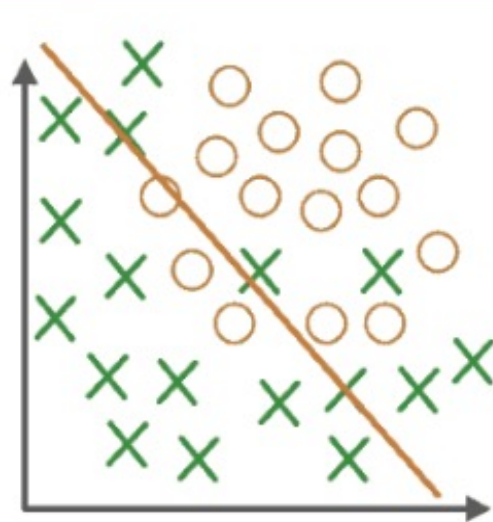
```
X_test_scaled = scaler.transform(X_test)
```



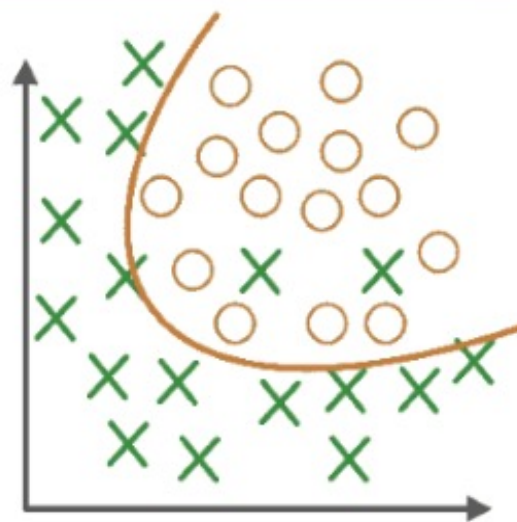
Regularized Methods for Regression

- Regularization penalizes large model coefficients.
 - Prevents overfitting, improves model generalization.
 - Lasso regression uses L1 penalty shrinkage.
 - Ridge regression applies L2 penalty constraint.
 - XGBoost combines both L1, L2 regularizations.
 - Best method depends on dataset properties.
-

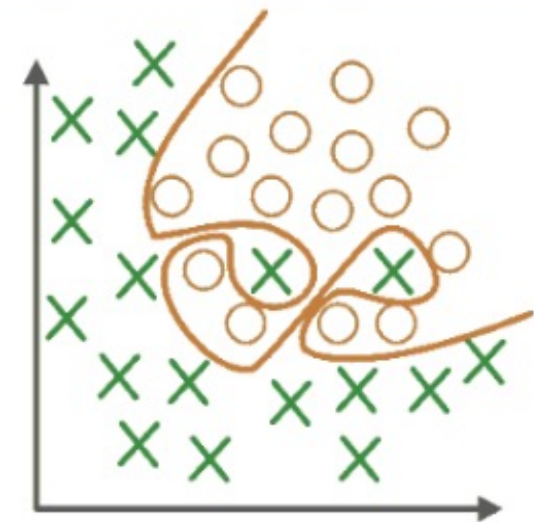
Regularized Methods for Regression



Under-fitting
(too simple to
explain the variance)



Appropriate-fitting



Over-fitting
(forcefitting--too
good to be true)

Regularization

- Regularization is a technique to discourage the complexity of the model. It does this by penalizing the loss function. This helps to solve the overfitting problem.
- **Let's understand how penalizing the loss function helps simplify the model**
- Loss function is the sum of squared difference between the actual value and the predicted value:

$$L(x, y) = \sum_{i=1}^n (y_i - f(x_i))^2$$
$$f(x_i) = h_{\theta}x = \theta_0 + \theta_1x_1 + \theta_2x_2^2 + \theta_3x_3^3 + \theta_4x_4^4$$

Regularization

- As the degree of the input features increases the model becomes complex and tries to fit all the data points.
- When we penalize the weights θ_3 and θ_4 and make them too small, very close to zero. It makes those terms negligible and helps simplify the model.

$$f(x_i) = h_{\theta}x = \theta_0 + \theta_1x_1 + \theta_2x_2^2 + \theta_3x_3^3 + \theta_4x_4^4$$
$$f(x_i) = h_{\theta}x = \theta_0 + \theta_1x_1 + \theta_2x_2^2$$

- Regularization works on assumption that smaller weights generate simpler model and thus helps avoid overfitting.

L1 Normalization

- LASSO regression, L1 regularization, includes a hyper-parameter α times the sum of the absolute value of the coefficients as penalty term in its cost function, shown below (marked in red):

$$\sum_{i=1}^n (Y_i - \hat{b}_0 - \hat{b}_1 X_i - \dots - \hat{b}_p X_p)^2 + \alpha (|\hat{b}_1| + \dots + |\hat{b}_p|)$$

- On the one hand, if we do not apply any penalty (set $\alpha = 0$), the above formula turns into a regular OLS regression, which may overfit.
- On the other hand, the model will probably underfit if we apply a very large penalty (or, a large α value), because we have falsely penalized all coefficients (the most important ones included).

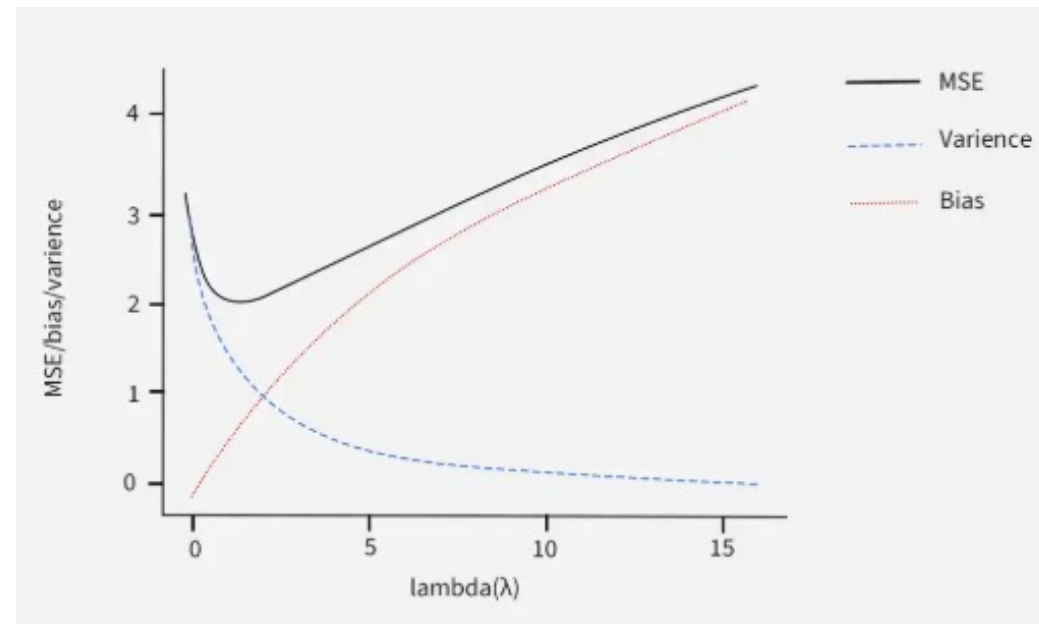
L2 Normalization

- Ridge regression adopts a “**squared magnitude**” of coefficient times lambda as penalty term, shown below.

$$\sum_{i=1}^n (Y_i - \hat{b}_0 - \hat{b}_1 X_i - \dots - \hat{b}_p X_p)^2 + \lambda (\hat{b}_1^2 + \dots + \hat{b}_p^2)$$

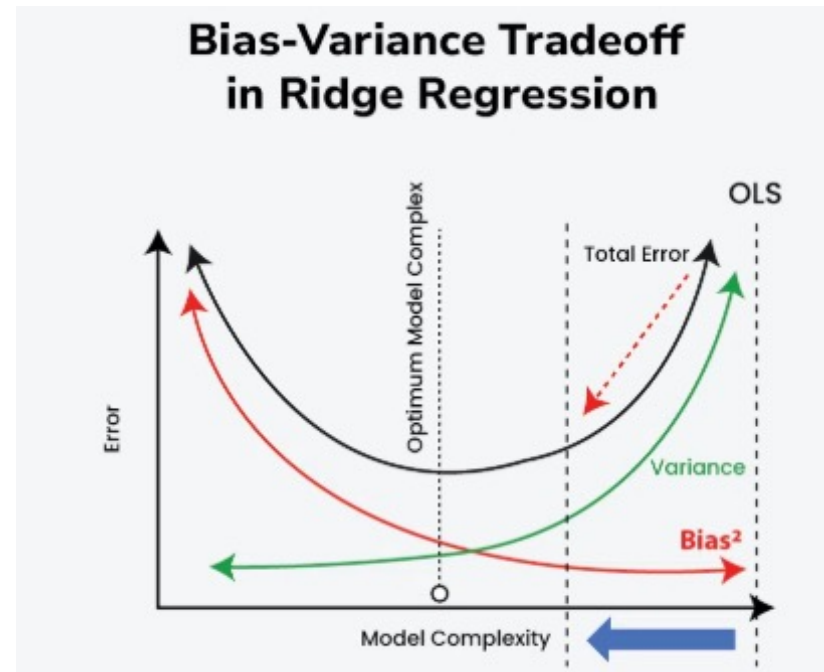
- If **lambda λ is 0**, the formula becomes a regular OLS regression. The **penalty term of the cost function** (marked out in red) increases the biases of the model and makes the fit on the training data worse.
- L2 is called regularization for simplicity**. Instead of shrinking to zero, L2 regularization slows down as the rate goes towards 0. In each iteration, L2 removes a small percentage of weights and so never converges to 0. ⁴⁰

Lasso Regression



- Achieving the **ideal balance** usually entails **minimizing the total prediction error (MSE)** by adjusting the regularization parameter using methods like **cross-validation**.

Ridge Regression



- Ridge regression introduces bias into the estimates to reduce their variance. The mean squared error (MSE) of the ridge estimator can be decomposed into bias and variance components.

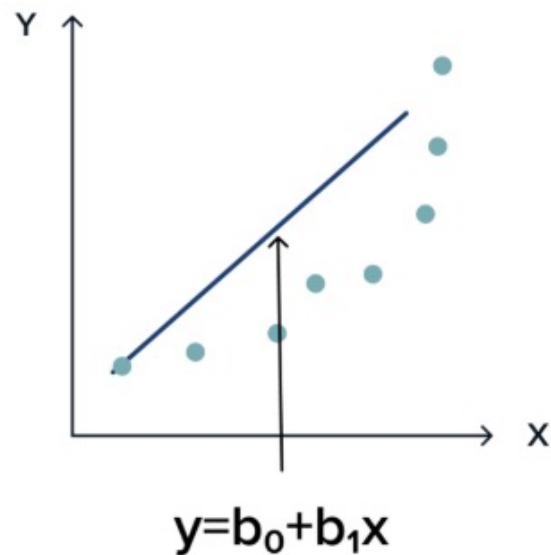


Polynomial Regression Basics

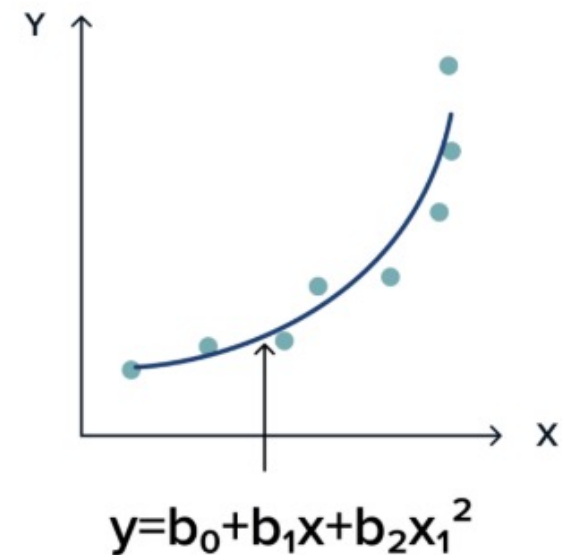
- Fits non-linear relationships using polynomial terms.
 - Transforms features into higher degree powers.
 - Model equation becomes $y = ax^2 + bx + c$
 - Allows fitting curved data distributions flexibly.
 - Overfitting mitigated using regularization techniques.
 - Useful in complex patterns, engineering problems.
-

Polynomial Regression Basics

Simple linear model



Polynomial model



Polynomial Regression Example

If **X** has **two features**:

$$X = [x_1 \ x_2]$$

- **Linear Regression Model:**

$$Y = b_0 + b_1x_1 + b_2x_2$$

- **Polynomial Regression (Degree 2):**

$$Y = b_0 + b_1x_1 + b_2x_2 + b_3x_1^2 + b_4x_2^2 + b_5x_1x_2$$

- This introduces **quadratic terms** (x_1^2 , x_2^2) and **interaction terms** ($x_1 * x_2$).
-

Polynomial Regression Example

- Input Features ($X_{\text{train_scaled}}$ - 2 Features)

Feature 1	Feature 2
1.2	2.5
3.4	1.8

- Transformed Features ($X_{\text{train_poly}}$)

Feature 1	Feature 2	Feature 1 ²	Feature 2 ²	Feature 1 × Feature 2
1.2	2.5	1.44	6.25	3.0
3.4	1.8	11.56	3.24	6.12



More features → More flexibility to fit non-linear patterns.

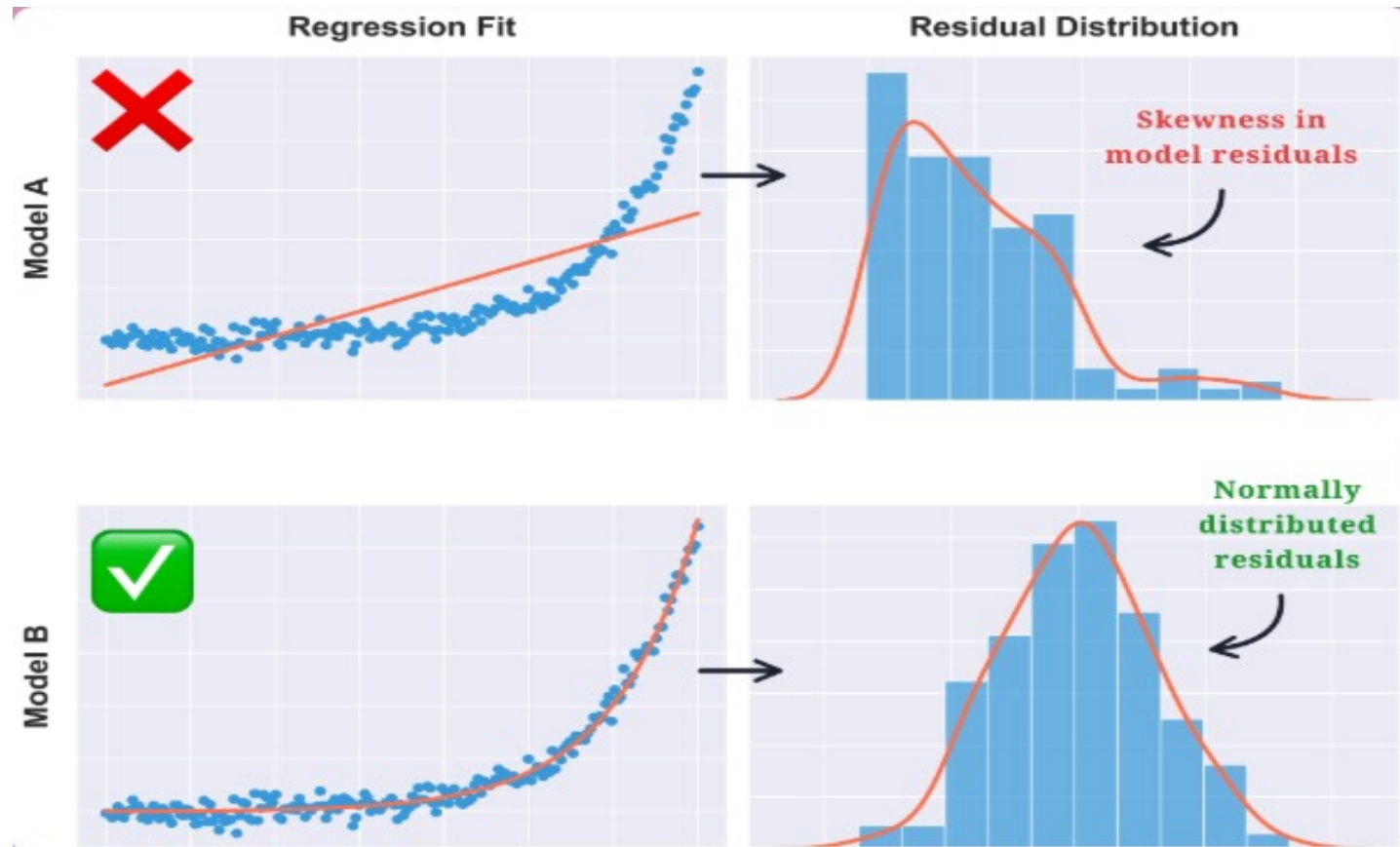


Too many features → Risk of **overfitting** if the degree is too high.

Steps in Polynomial Regression

- **Preprocess data**: standardize or normalize features.
 - **Expand features** into polynomial degree spaces.
 - **Fit linear regression** to transformed features.
 - Evaluate model using **error metrics**.
 - **Choose optimal degree** balancing bias-variance tradeoff.
 - **Apply regularization** if overfitting problems occur.
-

Polynomial Regression Basics



Polynomial Regression

Advantages

- ✓ Captures **non-linear relationships** better than Linear Regression.
- ✓ Works well for **curved** datasets.
- ✓ Provides **better flexibility** for complex patterns

Disadvantages

- ✗ **Overfitting risk:** Higher-degree polynomials may fit noise instead of actual trends.
 - ✗ **Computationally expensive:** More terms increase computation cost.
 - ✗ **Difficult to interpret:** Harder to explain compared to linear models.
-

Regression Applications and Use Cases

- Linear regression predicts sales, trends, economics.
 - RANSAC aids robotics, computer vision modeling.
 - Correlation matrix supports stock price dependencies.
 - Polynomial regression analyzes curved natural patterns.
 - Regularization used in sparse data regressions.
 - EDA essential for all data-driven approaches.
-



Regression-II

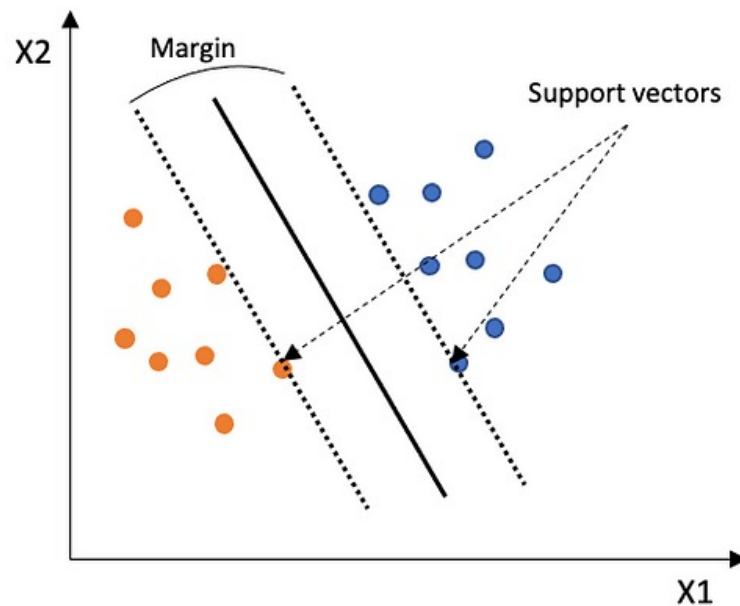




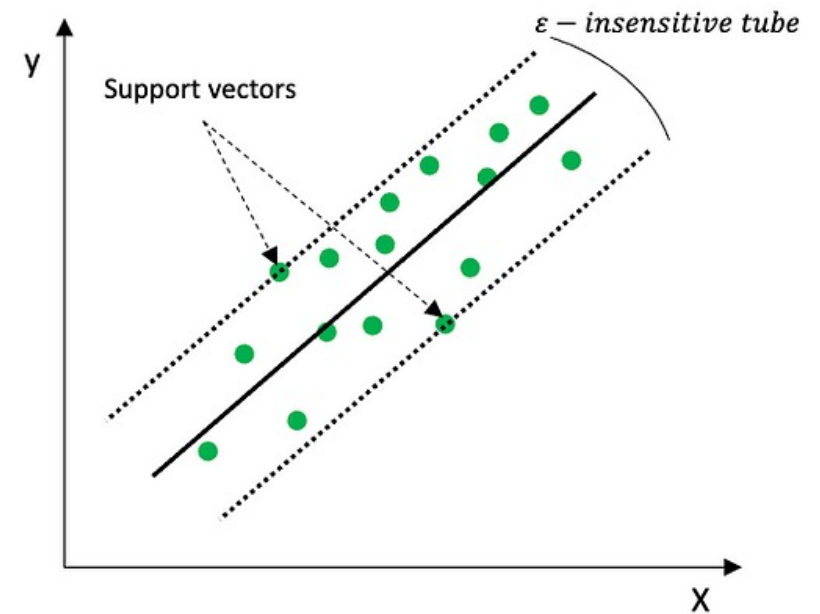
Introduction to Support Vector Machine (SVM) Regressor

- The **Support Vector Machine (SVM) Regressor**, also called **Support Vector Regression (SVR)**, is an extension of the **SVM classification algorithm** used for **predicting continuous values** rather than classifying data.
 - Unlike traditional regression models like Linear Regression, SVR focuses on **finding a function that fits within a margin of tolerance** rather than minimizing the prediction error.
-

Introduction to Support Vector Machine (SVM) Regressor



Classification problem using SVM



Regression problem using SVR

Working of SVR

- Instead of minimizing the residual errors like in Linear Regression, SVR tries to fit as many data points as possible within a margin (ϵ -tube).
 - **Margin (ϵ -tube):** A tolerance range where predictions are considered correct if they fall inside (e.g. predicted values fall within ± 0.1 of actual, they are considered acceptable) .
 - **Support Vectors:** The data points that lie on the margin boundaries (or outside the tube) and influence the model.
 - **Kernel Trick:** SVR can be linear or non-linear using different kernel functions.
-

SVR Kernels

Common **kernel functions**:

- **Linear Kernel** $\rightarrow K(x, y) = x \cdot y$
- **Polynomial Kernel** $\rightarrow K(x, y) = (x \cdot y + c)^d$
- **Radial Basis Function (RBF) Kernel** $\rightarrow K(x, y) = e^{\{-\gamma ||x - y||^2\}}$
(For complex, non-linear relationships, Most commonly used)

Comparison with Linear Regression

Feature	Linear Regression	SVM Regressor (SVR)
Objective	Minimizes MSE	Fits data within an ϵ -margin
Handles Non-Linearity?	No	Yes (using kernels)
Effect of Outliers	High	Lower (due to margin)
Flexibility	Limited	High (tunable parameters)

SVR Formula

- Given input features X and target values Y , SVR aims to find a function:

$$f(X) = wX + b$$

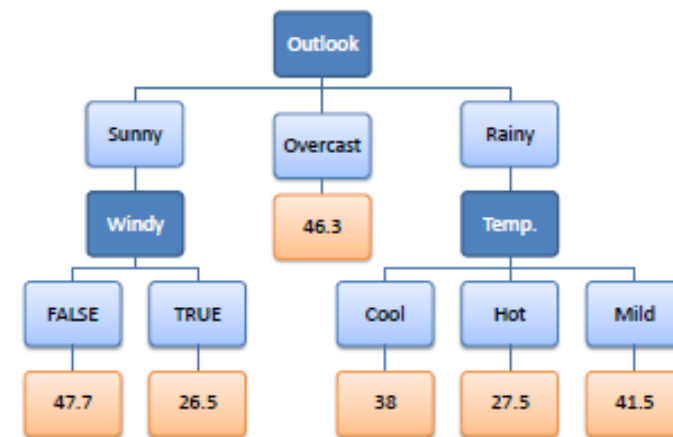
such that the absolute error is within an ϵ -tube:

$$|Y - f(X)| \leq \epsilon$$

Introduction to Decision Tree Regressor

- A **Decision Tree Regressor** is a machine learning algorithm used for **predicting continuous values** (like house prices, temperature, salaries, etc.).
- It builds a **tree-like model of decisions** and predicts values based on **feature-based splits**.

Predictors				Target
Outlook	Temp.	Humidity	Windy	Hours Played
Rainy	Hot	High	False	26
Rainy	Hot	High	True	30
Overcast	Hot	High	False	48
Sunny	Mild	High	False	46
Sunny	Cool	Normal	False	62
Sunny	Cool	Normal	True	23
Overcast	Cool	Normal	True	43
Rainy	Mild	High	False	36
Rainy	Cool	Normal	False	38
Sunny	Mild	Normal	False	48
Rainy	Mild	Normal	True	48
Overcast	Mild	High	True	62
Overcast	Hot	Normal	False	44
Sunny	Mild	High	True	30



Decision Tree Regressor: Example

- Imagine you're trying to **guess a house's price**.
 - You might ask questions like:
 1. Is the **area > 8000 sqft**?
 2. Does it have **more than 2 bathrooms**?
 3. Is it **located on a main road**?
 - Each of **these decisions splits your data** — and eventually **leads to a predicted price**.
 - That's what a decision tree does — **it asks yes/no questions at each node**, and **predicts a value at each leaf**.
-

Decision Tree Regressor: Working

- **Training Phase**

- It splits the data recursively on **feature values to minimize prediction error**.
- At each split, it chooses the **feature** and **threshold** that best reduces the Mean Squared Error (MSE).

- **Prediction Phase**

- For a **new sample**, it travels down the tree based on feature values until **it hits a leaf node**.
 - The **leaf node contains the predicted value** (usually the mean of training values in that node).
-

Decision Tree Regressor

- **Advantages**

- **Easy to Understand:** Works like a flowchart
- **Non-linear Ready:** Can handle complex patterns without any transformation
- **No Need for Scaling:** Handles raw data well

- **Disadvantages**

- **Overfitting:** Learns noise if the tree is too deep
 - **Not Smooth:** Predictions are step-like, not continuous
 - **Sensitive to Data Changes:** Small changes can lead to different trees
-

Random Forest Regressor

- A **Random Forest Regressor** is an **ensemble learning algorithm** that combines the predictions of **multiple decision trees** to make a more **accurate and stable prediction** of a continuous target variable (like house price, temperature, salary, etc.).
 - It's like asking **multiple experts (trees)** and taking the **average of their opinions**.
-

Random Forest Regressor: Working

1. Bootstrapping:

- Randomly selects samples **with replacement** from the original dataset to create multiple **training subsets**.

2. Build Multiple Decision Trees:

- Each subset trains its own **Decision Tree Regressor**.
- At each split, only a **random subset of features** is considered (adds randomness & reduces correlation).

3. Aggregate Predictions:

- For regression: **Takes the average** of all tree predictions.
 - For classification: **Takes the majority vote**.
-

Random Forest Regressor: Example

Let's say 3 trees give these house price predictions:

- Tree 1: ₹12,500,000
- Tree 2: ₹13,000,000
- Tree 3: ₹12,200,000

Random Forest Prediction:

- $\{12,500,000 + 13,000,000 + 12,200,000\} / \{3\} =$
₹12,566,666.67

Random Forest Regressor: Advantages

Feature	Benefit
✓ High Accuracy	Combines multiple models to reduce error
✓ Robust to Overfitting	Trees are less correlated due to randomness
✓ Handles Non-linear Data	No need for feature transformation
✓ Works with Missing/Unscaled Data	Trees are flexible to different input formats
✓ Feature Importance	Tells you which features are most influential

Random Forest Regressor: Disadvantages

Limitation	Impact
✗ Slower	Training & prediction take longer than a single tree
✗ Less Interpretable	Can't visualize like a simple decision tree
✗ Not good for real-time apps	Heavy for low-latency use cases

Introduction to ARIMA

- ARIMA stands for **AutoRegressive Integrated Moving Average**.
- It's a powerful and widely used **statistical model for time series forecasting** — especially when your data has **trend, seasonality, or noise**.
- ARIMA can be used with time-dependent data like:
 - **Stock prices**
 - **Weather predictions**
 - **Sales forecasting**
 - **Electricity demand**
 - **Website traffic**

Introduction to ARIMA

- **ARIMA(p, d, q)**
- Each letter corresponds to a key concept:

Component	Meaning	What it does
AR (p)	AutoRegression	Uses past values to predict future values
I (d)	Integrated	Differencing the data to make it stationary
MA (q)	Moving Average	Uses past forecast errors to improve predictions

Introduction to ARIMA

1. AutoRegressive (AR) – “Regression of the series on itself”

$$Y_t = c + \phi_1 Y_{\{t-1\}} + \phi_2 Y_{\{t-2\}} + \dots + \phi_p Y_{\{t-p\}} + \epsilon_t$$

Example: “Today’s stock price depends on the past 3 days’ prices”

2. Integrated (I) – “Remove trend with differencing”

If your time series has a trend, subtract the previous value:

$$Y'_t = Y_t - Y_{\{t-1\}}$$

- You can difference **once** (d=1), **twice** (d=2), etc., until the data becomes **stationary** (constant mean and variance over time).

3. Moving Average (MA) – “Smooth past errors”

$$Y_t = c + \epsilon_t + \theta_1 \epsilon_{\{t-1\}} + \theta_2 \epsilon_{\{t-2\}} + \dots + \theta_q \epsilon_{\{t-q\}}$$

- Instead of using past values, it uses **past errors** to correct predictions.

Introduction to ARIMA

□ Advantages

- Great for **non-stationary** data
- Can **model complex patterns** (AR + MA)
- **Statistically interpretable**

□ Limitations

- Doesn't handle **seasonality** well (use **SARIMA** instead)
- Assumes **linearity**
- Parameter tuning (p, d, q) needs **trial and error**

Introduction to SARIMA

- **SARIMA** stands for: **Seasonal AutoRegressive Integrated Moving Average**
- It is an **extension of ARIMA** that adds the ability to model **seasonal patterns** in your data.

You use SARIMA when:

- Your time series has a **trend**
- Your time series has **seasonality**
- ARIMA alone isn't enough because ARIMA assumes the **seasonality has been removed**

Introduction to SARIMA

- **SARIMA(p, d, q) × (P, D, Q, s)**

Component	Description
p	Order of AutoRegressive (AR) part
d	Order of Integration (differencing)
q	Order of Moving Average (MA) part
P	Seasonal AR order
D	Seasonal differencing order
Q	Seasonal MA order
s	Seasonal period (e.g., s=12 for monthly data with yearly seasonality)

Introduction to SARIMA

Part	Explanation
AR (p)	Regression on past values (lags)
I (d)	Removes non-stationary trend
MA (q)	Regression on past forecast errors
SAR (P)	Regression on seasonal lags
SI (D)	Seasonal differencing
SMA (Q)	Forecast error smoothing on seasonal lags
s	How often the season repeats

Introduction to SARIMA: Example

Suppose your monthly sales data has this:

- **Trend:** Gradually increasing over time
- **Seasonality:** Repeats every 12 months

You would try:

SARIMA (1, 1, 1) × (1, 1, 1, 12)

That means:

- One regular AR, I, and MA component
- One seasonal AR, I, and MA component
- Seasonality repeats every **12** periods (monthly → annual)

Introduction to SARIMA: Advantages

Benefit	Description
Seasonality	Handles cyclic patterns like sales, weather
Flexibility	Can model both short-term & long-term trends
Based on ARIMA	Familiar for time series experts
Forecasting power	Excellent accuracy if seasonality is present

Introduction to SARIMA: Disadvantages

Limitation	Description
Complex	More parameters to tune
Slower	Takes more time to train
Sensitive	Needs good choice of seasonal period s

Types of Errors: Mean Absolute Error

- **Formula:**

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Interpretation:**

- Average of absolute differences between predictions and actual values.
- Easy to interpret (same unit as target).
- Doesn't penalize large errors heavily.

Types of Errors: Mean Squared Error (MSE)

- **Formula:**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Interpretation:**

- Average of **squared errors**.
- **Penalizes larger errors more than smaller ones** (quadratic growth).
- Useful when **large errors are unacceptable**.

Types of Errors: Mean Squared Logarithmic Error (MSLE)

- **Formula:**

$$\text{MSLE} = \frac{1}{n} \sum_{i=1}^n (\log(1 + y_i) - \log(1 + \hat{y}_i))^2$$

- **Interpretation:**

- Compares **log-transformed** actual vs predicted values.
- Focuses more on **relative differences**, especially for **small numbers**.
- Less sensitive to large absolute errors, **good for exponential or skewed targets**.

Types of Errors: Mean Absolute Percentage Error (MAPE)

- **Formula:**

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

- **Interpretation:**

- Measures error as a **percentage of actual value**
- **Scale-independent**, great for business decisions or time series

Comparisons of Errors

Metric	Penalizes Big Errors	Unit-Sensitive	Good For Small Values	Interpretable as %
MAE	No	Yes	Yes	No
MSE	Yes	Yes	No	No
MSLE	Log-based	No	Yes	No
MAPE	No	No	No	Yes

When to Use What?

If you want to...	Use
Just measure average error	MAE
Heavily punish large errors	MSE
Handle exponential growth or count data	MSLE
Communicate error in percentage terms	MAPE

Explained Variance Score

- The Explained Variance Score measures the **proportion of variance in the target variable that is explained by your model's predictions.**
- It's **similar to R^2 Score**, but with subtle differences.
- **Formula:**

$$\text{Explained Variance} = 1 - \frac{\text{Var}(y - \hat{y})}{\text{Var}(y)}$$

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

Explained Variance Score

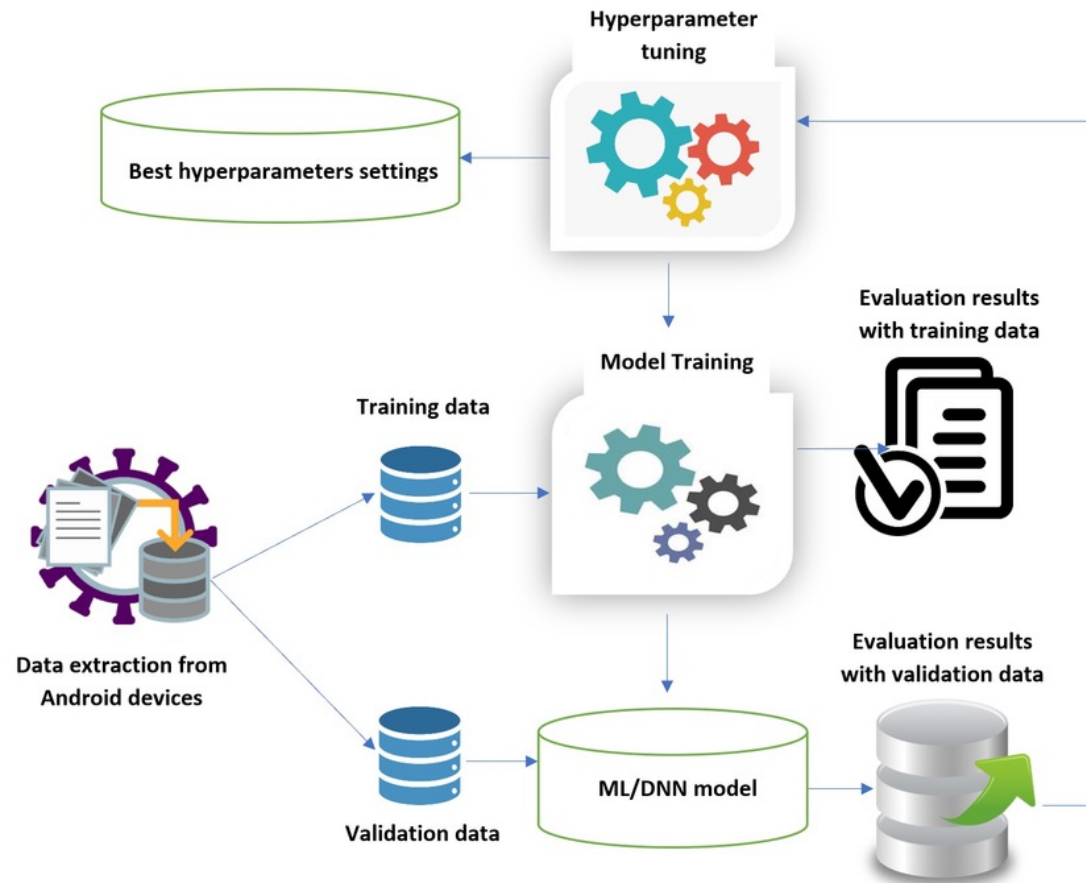
Score	Meaning
1.0	Perfect prediction (no error at all)
0.0	Model predicts no better than a constant
< 0.0	Model performs worse than constant prediction

What's the difference vs R^2 ?

Metric	Key Point
R^2 Score	Measures how much variance is explained vs total variance (mean)
Explained Variance	Measures how much variance is explained vs prediction error

- R^2 penalizes bias, while Explained Variance doesn't.

Model evaluation and hyperparameter tuning



What is Pipeline

- A Pipeline is a tool in scikit-learn that allows you to chain together multiple data processing steps and a model into one streamlined workflow.

Without Pipeline	With Pipeline
Manually preprocess data for train/test	Automates preprocessing consistently
Easy to forget steps during deployment	Keeps steps in order and together
Difficult to avoid data leakage	Prevents leakage automatically
Hard to cross-validate the whole process	Makes full cross-validation possible

What a Pipeline Typically Includes

1. Data Preprocessing

- Imputation
- Scaling
- Encoding

2. Feature Selection or Transformation

- PCA, Polynomial Features

3. Model Training

- Any regressor/classifier
-

Pipeline Examples

Without Pipeline (Manual):

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
model.fit(X_train_scaled, y_train)
```

With Pipeline:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression

pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler()),
    ('regressor', LinearRegression())
])

pipeline.fit(X_train, y_train)

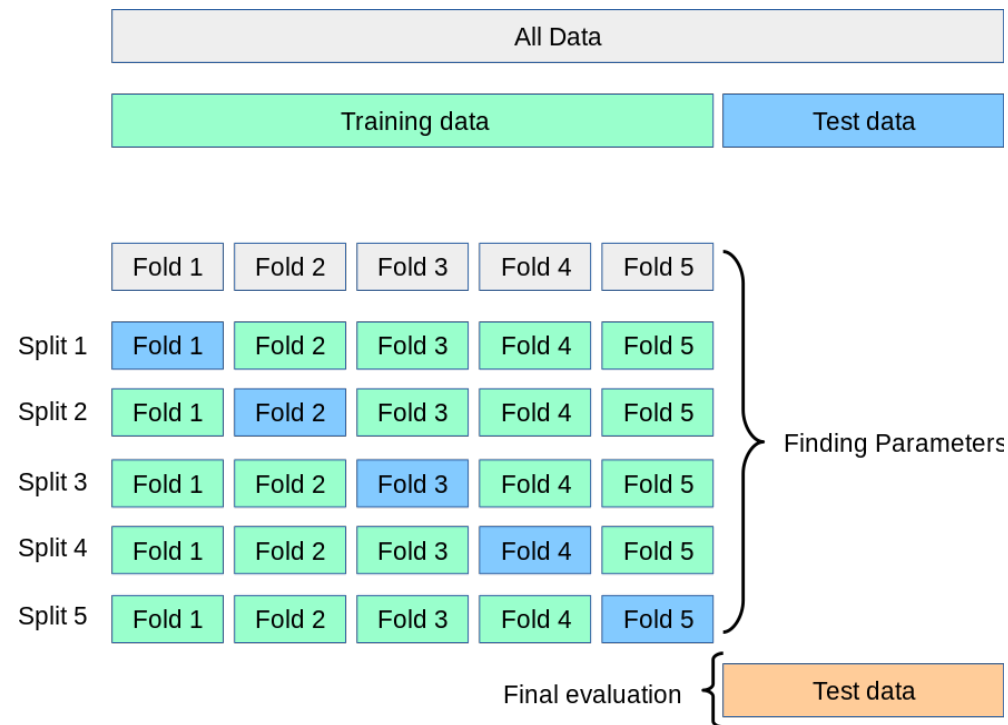
• predictions = pipeline.predict(X_test)
```

Benefits of Using Pipelines

Benefit	Description
✓ Cleaner Code	All steps handled in one object
✓ Reproducibility	Less prone to human error
✓ Cross-validation Ready	Plug into GridSearchCV, <code>cross_val_score</code>
✓ No Data Leakage	Fits transformers only on training data
✓ Easy to Deploy	Export one fitted object using joblib

K-Fold Cross Validation

- **K-fold cross-validation** is a technique used to assess the performance of a machine learning model and ensure that it **generalizes well to unseen data**.
- It helps to **prevent overfitting or underfitting** by evaluating the model on different portions of the dataset.



K-Fold Cross Validation: Working

► Data Splitting:

- The dataset is divided into k equal-sized subsets or “folds”.
- For each fold, a portion is used as the test set, and the remaining $k-1$ folds are used as the training set.

► Training and Evaluation:

- The model is trained on $k-1$ folds and tested on the remaining fold. This process is repeated k times, with each fold taking a turn as the test set.

► Averaging Results:

- After k iterations, we obtain k performance scores (e.g., accuracy, precision, recall, or other metrics) from each fold.
- The final model performance score is computed by taking the average of these k scores. This average provides a more robust estimate of the model’s performance compared to using a single train-test split.

K-Fold Cross Validation: Benefits

► Reduces Bias and Variance:

- By training and testing the model on different parts of the dataset, k-fold cross-validation **reduces the bias and variance associated with model performance**. Each data point is used for both training and validation, which leads to a more comprehensive evaluation.

► Better Use of Data:

- This approach ensures that **every observation in the dataset is used for training and testing**, making full use of the available data, especially helpful when data is limited.

► Consistency and Robustness:

- K-fold cross-validation provides a **more reliable performance estimate** since the model is evaluated on multiple test sets. It's less influenced by the quirks of any single train-test split.

K-Fold Cross Validation: Example

► For a 5-fold cross-validation:

- The data is split into 5 folds.
- The model is trained on 4 folds and tested on the 1 remaining fold.
- This process repeats until each fold has served as the test set once.
- The performance scores from each of the 5 tests are averaged to produce the final model performance estimate.

Learning Curve

- ▶ A learning curve shows how the model's performance changes as we increase the size of the training data.
- ▶ Axes:
 - X-axis: Number of training samples
 - Y-axis: Model score (e.g., accuracy, R^2 , RMSE)
- ▶ Shows:
 - How much the model benefits from more data
 - Whether the model is underfitting or overfitting

Learning Curve Interpretation

Case	Training Score	Validation Score	What It Means
Both low	Low	Low	Underfitting (high bias)
Train high, test low	High	Low	Overfitting (high variance)
Both converge	High/Medium	High/Medium	Good fit

Validation Curve

- ▶ A validation curve shows **how the model's performance changes when you vary a hyperparameter** (e.g., depth of a tree, number of estimators).
- ▶ **Axes:**
 - X-axis: Hyperparameter value (e.g., max_depth, C, n_estimators)
 - Y-axis: Score (training and validation)
- ▶ **Shows:**
 - When a model starts to overfit or underfit as a function of the hyperparameter
 - Helps identify the optimal hyperparameter range

Use Cases

- ▶ Use learning curve to decide:
 - Do I need **more data or a better model?**
- ▶ Use validation curve to decide:
 - Which **hyperparameter gives the best generalization?**

Feature	Learning Curve	Validation Curve
What it varies	Training set size	Hyperparameter
What it helps detect	Bias/variance problems	Optimal hyperparameter
Use case	Data quantity issue	Model tuning

Tuning Hyper Parameters

- Hyper parameters that **cannot be directly learned from the regular training process** are usually fixed before the actual training process begins.
- These parameters express **important properties of the model such as its complexity or how fast it should learn**.
- Some examples of model hyper parameters include:
 - The penalty in Logistic Regression Classifier i.e. L1 or L2 regularization
 - The learning rate for training a neural network.
 - The C and sigma hyper parameters for support vector machines.
 - The k in k-nearest neighbours.
- Models can have many hyper parameters and finding the best combination of parameters can be treated as a search problem. Two best strategies for Hyper parameter tuning are:
 - GridSearchCV
 - RandomizedSearchCV

GridSearchCV

- Machine learning model is evaluated for a **range of hyper parameter values**. This approach is called GridSearchCV, because it searches for best set of hyper parameters from a grid of hyper parameters values.
- For example:** if we want to set two hyper parameters C and Alpha of Logistic Regression Classifier model, with different set of values.
- The gridsearch technique will construct **many versions of the model with all possible combinations of hyperparameters and will return the best one**.

C	0.5	0.701	0.703	0.697	0.696
	0.4	0.699	0.702	0.698	0.702
	0.3	0.721	0.726	0.713	0.703
	0.2	0.706	0.705	0.704	0.701
	0.1	0.698	0.692	0.688	0.675
		0.1	0.2	0.3	0.4
Alpha					

- Drawback:** GridSearchCV will go through all the intermediate combinations of hyper parameters which makes grid search **computationally very expensive**.

RandomizedSearchCV

- RandomizedSearchCV solves the drawbacks of GridSearchCV, as it goes through **only a fixed number of hyper parameter settings**. It moves within the grid in random fashion to find the best set hyper parameters. This approach reduces unnecessary computation.
- RandomizedSearchCV implements a **“fit” and a “score” method**. It also implements “score_samples”, “predict”, “predict_proba”, “decision_function”, “transform” and “inverse_transform”.
- The parameters of the estimator used to apply these methods are optimized by **cross-validated search over parameter settings**.
- In contrast to GridSearchCV, not all parameter values are tried out, but rather a **fixed number of parameter settings is sampled from the specified distributions**. The number of parameter settings that are tried is given by n_iter.